Quantum Rewinding for IOP-Based Succinct Arguments

Alessandro Chiesa
alessandro.chiesa@epfl.ch
EPFL

Marcel Dall'Agnol
dallagnol@princeton.edu
Princeton University

Zijing Di
zijing.di@epfl.ch
EPFL

Ziyi Guan
ziyi.guan@epfl.ch
EPFL

Nicholas Spooner
nspooner@cornell.edu
Cornell University

October 24, 2025

Abstract

We analyze the post-quantum security of succinct interactive arguments constructed from interactive oracle proofs (IOPs) and vector commitment schemes. Specifically, we prove that an interactive variant of the *BCS transformation* is secure in the standard model against quantum adversaries when the vector commitment scheme is collapse binding.

Prior work established the post-quantum security of Kilian's succinct interactive argument, a special case of the BCS transformation for one-message IOPs (i.e., PCPs). That analysis is inherently limited to one message because the reduction, like all prior quantum rewinding reductions, aims to extract classical information (a PCP string) from the quantum argument adversary. Our reduction overcomes this limitation by instead extracting a *quantum algorithm* that implements an IOP adversary; representing such an adversary classically may in general require exponential complexity.

Along the way we define *collapse position binding*, which we propose as the "correct" definition of collapse binding for vector commitment schemes, eliminating shortcomings of prior definitions.

As an application of our results, we obtain post-quantum secure succinct arguments, in the standard model (no oracles), with the *best asymptotic complexity known*.

Keywords: succinct arguments; post-quantum security; quantum rewinding

Contents

1 Introduction

A succinct argument is a protocol in which a prover convinces a verifier of the truth of an NP statement while communicating significantly fewer bits than required to send the witness. Succinct arguments are a fundamental cryptographic primitive with numerous theoretical and practical applications. In this paper we study the post-quantum security of a notable class of succinct interactive arguments in the standard model.

Succinct arguments from PCPs. The first construction of a succinct argument is due to Kilian [Kil92]. Kilian's protocol is constructed from two ingredients: a *probabilistically checkable proof* (PCP), which is an information-theoretic object; and a *vector commitment* [CF13], which is a cryptographic primitive that can be obtained, e.g., from a collision-resistant hash function (via a Merkle commitment scheme). Kilian's protocol was shown to be post-quantum secure in [CMSZ21], almost three decades later, provided that the vector commitment satisfies a standard post-quantum security property called "collapsing" [Unr16b].

Succinct arguments from IOPs. Kilian's succinct argument is often not used due to its poor efficiency (e.g., the concrete cost of known PCP constructions is astronomical [WB15]). However, there is a related family of succinct arguments based on *interactive oracle proofs* (IOPs) [BCS16; RRR16], an interactive generalization of PCPs with significantly better efficiency both asymptotically (e.g., [BCGRS17; RR20; RR22; ACY23]) and concretely (e.g., [BBHR19; GLSTW23; CBBZ23; STW23; HLP24; ACFY24; ZCF24]). Specifically, the BCS transformation [BCS16] enables the construction of succinct *non*-interactive arguments in the random oracle model from any public-coin IOP that satisfies a certain (necessary) strong soundness property.

This leads to a natural interactive relaxation in the standard model (i.e., without oracles or heuristics) that applies to all public-coin IOPs, known as the *interactive BCS protocol* (IBCS protocol). Due to its combination of efficiency, flexibility, and standard-model security, the IBCS protocol plays a key role in constructions of asymptotically efficient succinct arguments [BCG20; RR22; HR22]. All of these works focus on designing efficient *IOPs*; the security of the final succinct argument is then obtained by appealing to general security properties of the IBCS protocol. Despite its widespread prior use, the classical security of the IBCS protocol was only established in [CDGS23].

A further appealing feature of the IBCS protocol is that it is "plausibly" post-quantum secure when instantiated with a post-quantum secure vector commitment (in the sense that there is no obvious quantum attack). However, the post-quantum security of IBCS has never been established based on any cryptographic assumption. Showing this would imply, in particular, that the efficient succinct arguments of [BCG20; RR22; HR22] can be made post-quantum secure under that same assumption.

1.1 Our results

We prove that the IBCS protocol is a succinct interactive argument that is secure against quantum adversaries, when instantiated with any public-coin IOP and "collapsing" vector commitment.

Theorem 1. *Let* IOP *be a public-coin semi-adaptive*[1] *IOP for an* NP *relation* R *with soundness error* ε*, and let* VC *be a collapsing vector commitment scheme. Then* $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ *is an interactive argument for* R *with post-quantum soundness error* $\varepsilon + \mathrm{negl}(\lambda)$*. If* IOP *has post-quantum knowledge soundness error* κ*, then* $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ *has post-quantum knowledge soundness error* $\kappa + \mathrm{negl}(\lambda)$*.*

[1] A semi-adaptive IOP is one where queries to the prover's message in round i do not depend on the answers to queries to prover messages in later rounds; or, equivalently, if the verifier makes no queries to round-j messages for $j < i$. In particular, a non-adaptive IOP is a semi-adaptive IOP. See Remark 2.1 for a discussion.

The theorem is analogous to its classical counterpart [CDGS23]. The differences are that we require a collapsing-type security property of the vector commitment VC (typical of post-quantum security results for interactive protocols [CCY21; CMSZ21; LMS22a; LMS22b]) and that the IOP verifier is semi-adaptive.

The theorem, in the special case of Kilian's protocol (IBCS protocol for a PCP), improves on [CMSZ21]: we do not additionally assume post-quantum position binding (see below) and security holds for every adaptive PCP (an adaptive PCP is a semi-adaptive IOP, and [CMSZ21] considers only non-adaptive PCPs); see Remark 2.1.

Our analysis leading to Theorem 1 is sufficiently careful that we are able to make precise quantitative statements about the soundness and knowledge losses of the IBCS protocol (i.e., the negligible terms above) in terms of the quantum security of VC. See Theorem 6.1 for a formal statement with precise expressions.

Collapsing vector commitments. [CMSZ21] proposes a definition of collapsing for vector commitments. However, their definition has certain shortcomings: most prominently, it does not imply post-quantum position binding, the standard "classical" binding notion for vector commitments. This is in contrast to the collapsing property for standard commitments [Unr16b], which is known to imply the usual definition of post-quantum binding.

In this work we propose a new definition of collapsing for vector commitments, formally incomparable to the one in [CMSZ21], which addresses these shortcomings. In particular, it easily implies post-quantum position binding. We obtain our definition via a new methodology in which we systematically "lift" definitions for standard commitment schemes into corresponding definitions for vector commitments. This framework facilitates simpler proofs: the fact that our collapsing definition implies post-quantum position binding is a direct consequence of two facts: (i) post-quantum position binding is the "lift" of post-quantum binding, and (ii) for standard commitments, collapsing implies post-quantum binding. See Section 2.3.1 for more details.

Asymptotically efficient post-quantum succinct arguments. One desirable feature of the IBCS protocol is that, provided VC runs in linear time, the complexity of the prover and verifier in $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ is essentially the same as in IOP. Linear time vector commitments can be constructed straightforwardly from linear time hash functions, which exist classically under a coding-theoretic assumption [AHIKV17]. The same construction yields a collapsing vector commitment if the underlying hash function is collapsing (see the discussion in Section 2.3.1).[2] Thus, leveraging Theorem 1, we can transform an IOP into a post-quantum succinct argument with similar efficiency. For example, by combining the IOP in [RR22] with a linear-time collapsing vector commitment, we obtain the first post-quantum succinct arguments for boolean circuit satisfiability with a linear-size prover.

Corollary 1. *Suppose that there exist collapsing hash functions computable in linear time. Then for every boolean circuit C there is an $O(\log |C|)$-round post-quantum succinct argument for the relation $R_C \coloneqq \{(\mathbb{x}, \mathbb{w}) : C(\mathbb{x}, \mathbb{w}) = 1\}$ where the argument prover is a circuit of size $O(|C|)$ and the communication complexity is $\mathsf{poly}(\lambda, \log |C|)$.*

Discussion: quantum rewinding. Typical security proofs for succinct arguments in the standard model rely on the ability to *rewind* an adversary. Rewinding a classical adversary is trivially possible: one can take a "snapshot" of the adversary's internal state, and rerun the adversary from that snapshot on different inputs. However, for quantum adversaries, taking such a snapshot is impossible in general due to the no-cloning theorem. This is the notorious *quantum rewinding problem*, and is the key challenge in proving the post-quantum security for succinct arguments (in the standard model).

[2] It is a fascinating open question to determine whether the construction of [AHIKV17] yields a collapsing hash function assuming the quantum hardness of their binary SVP problem.

A sequence of works [Wat09; Unr12; CCY21; CMSZ21; LMS22a; LMS22b] demonstrated that for many protocols of interest one can "transfer" classical rewinding proofs to the quantum setting. However, due to the need to circumvent the no-cloning theorem, quantum rewinding reductions are far more complicated and cumbersome than classical ones. As a result, quantum rewinding algorithms are not a drop-in replacement for classical rewinding; each family of protocols requires a new approach. This work augments the quantum rewinding toolkit with several new techniques that we anticipate will be useful in future security analyses.

Open problems. While Theorem 1 gives a relatively complete picture of the post-quantum security of the IBCS protocol, several interesting questions remain open.

First, Theorem 1 applies to *semi-adaptive* (public-coin) IOPs. This is surprising, as the same caveat does not apply to either the classical analysis of IBCS or the post-quantum analysis of Kilian's protocol (though [CMSZ21] does not consider adaptive PCPs, Theorem 1 works when the IBCS protocol is based on an adaptive PCP). We discuss why our reduction can fail for adaptive (public-coin) IOPs in Remark 2.1.

The second question relates to knowledge soundness. Our reduction shows that the IBCS protocol is a post-quantum argument of knowledge per Definition 3.11 (following a definition proposed by [Unr12]), if the underlying IOP is a proof of knowledge per Definition 3.3 (the analogue of Definition 3.11 for IOPs). Often, however, when using an argument of knowledge as part of a larger protocol, we require a stronger notion of extractability (e.g., witness-extended emulation) to prove security of the overall protocol. While this is already true in the classical setting, it is especially important in the quantum setting, where the unclonability of quantum states places strict requirements on the extractor (e.g., that it must be "state-preserving" [LMS22b]). It remains open whether the IBCS protocol can be shown to satisfy stronger extraction properties.

1.2 Related work

There are few works that establish the security of succinct arguments against quantum adversaries.[3]

In the random oracle model. The first proof of post-quantum security of a succinct argument is due to [CMS19]. They proved that SNARGs (succinct *non-interactive* arguments) obtained via the Micali transformation from a PCP and the BCS transformation from an IOP are secure in the *quantum* random oracle model, provided that the IOP satisfies a stronger notion of soundness called *round-by-round* soundness [CCHLRR18]. In contrast, our result yields post-quantum succinct *interactive* arguments in the *standard* model from standard post-quantum assumptions,[4] and applies to all public-coin IOPs.

In the standard model. [CMSZ21] shows that Kilian's succinct interactive argument, which combines a PCP and a vector commitment, is post-quantum secure (provided the vector commitment is collapsing). This is the first proof of post-quantum security in the standard model for any succinct argument. The tightness of the security reduction was improved in [LMS22b]. In both of these works, the security proof relies on the fact that rewinding takes place only in a single round. In contrast, for the IBCS protocol (based on IOPs), the classical proof requires rewinding in multiple rounds in sequence [CDGS23], as does our Theorem 1.

[LMS22a] showed that the $O(\log n)$-round succinct argument of [BLNS20] ("lattice Bulletproofs") is post-quantum secure assuming the quantum hardness of LWE. As in the classical setting, the security proof works by rewinding in a tree fashion, with each node computed from its children. While the proof technique does rewind across rounds, it relies strongly on the "tree special soundness" of Bulletproofs; in contrast, IBCS is *not* tree special sound. This difference is reflected in the classical rewinding reductions that employ different rewinding schemes ([BLNS20] vs. [CDGS23]). The quantum reductions follow similar templates.

[3] In contrast, there are many works that construct succinct arguments from post-quantum *assumptions* (e.g., certain lattice assumptions), and prove security only against classical adversaries; we do not discuss these.

[4] Even in the classical setting, there are significant barriers to obtaining SNARGs from standard assumptions [GW11].

A key difference between these prior works and the present work is that in the former security is proven by extraction of a *classical string* from the given quantum adversary against the interactive argument: a PCP string in the case of Kilian's protocol, and a commitment opening in the case of lattice Bulletproofs. For IOPs, the underlying soundness guarantee concerns the existence of an *IOP prover strategy* rather than a classical string. As such, our security reduction uses quantum rewinding to extract a *quantum algorithm*: a quantum IOP prover strategy that is almost "as good as" the given quantum argument adversary.

Finally, [Bar+22; MNZ24] construct succinct arguments for QMA with classical verification. Both use Kilian's succinct argument as a subprotocol, relying on the composable analysis in [LMS22b] of its post-quantum security. It may be possible to analyze the IBCS protocol in a similar way; however, it is not known how to achieve this without modifying the protocol (and making non-black-box use of cryptography).

From knowledge assumptions. Several works construct succinct non-interactive arguments from lattice-based (non-falsifiable) knowledge assumptions (e.g., [GMNO18; ISW21; ACLMT22; GNS23]). Since the extractor is assumed rather than constructed, post-quantum security holds under a quantum version of the knowledge assumption. The assumptions underlying the security analyses of these schemes are not well-understood, and many have been shown invalid, either classically [WW23] or quantumly [DFS24]. In this work we use standard (falsifiable) assumptions only: the collapsing property for vector commitments that we use is falsifiable, and can be achieved assuming the quantum hardness of standard LWE (see Section 2.3.1).

2 Techniques

We outline the main ideas underlying our results. In Section 2.1, we review the IBCS protocol. In Section 2.2, we discuss limitations of prior security analyses. In Section 2.3, we sketch how we prove the post-quantum security of the IBCS protocol.

2.1 Review: the IBCS protocol

The IBCS protocol involves two main ingredients.

- An *interactive oracle proof* (IOP) is an interactive protocol between a prover and a verifier in which the prover sends IOP strings as oracles and the verifier sends random messages; after the interaction, the verifier queries the prover oracles at few locations and then accepts or rejects. We let $\mathsf{IOP} = (\mathbf{P}, \mathbf{V})$ be a public-coin IOP for a relation R with alphabet Σ, round complexity k, proof lengths $(\mathsf{L}_i)_{i\in[\mathsf{k}]}$, query complexities $(\mathsf{q}_i)_{i\in[\mathsf{k}]}$, and verifier randomness complexities $(\mathsf{r}_i)_{i\in[\mathsf{k}]}$ (with maximal values $\mathsf{L}_{\mathsf{max}}, \mathsf{q}_{\mathsf{max}}, \mathsf{r}_{\mathsf{max}}$ and total values $\mathsf{L}, \mathsf{q}, \mathsf{r}$).

- A *vector commitment scheme* (VC scheme) is a cryptographic primitive that enables a sender to produce a short commitment to a long vector $m \in \Sigma^L$ and then, subsequently, verifiably open a subvector $m|_Q \in \Sigma^Q$ for any $Q \subseteq [L]$ (without having to open the entire vector). A well-known example is the Merkle commitment scheme. We let $\mathsf{VC} = (\mathsf{VC.Commit}, \mathsf{VC.Open}, \mathsf{VC.Check})$ be a vector commitment scheme.

The formal definitions for an IOP and a VC scheme are in Sections 3.2 and 3.3, respectively. The IBCS protocol combines these two ingredients to construct an interactive argument system where the argument prover sends *commitments* to each IOP oracle and the argument verifier sends the IOP verifier randomness and, after all commitments, the argument prover verifiably opens the queried locations for the argument verifier. In more detail, we denote by $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ the following interactive argument $\mathsf{ARG} = (\mathbb{P}, \mathbb{V})$. The argument prover $\mathbb{P}$ receives an instance $\mathbb{x}$ and a witness $\mathbb{w}$, and the argument verifier $\mathbb{V}$ receives the instance $\mathbb{x}$. Then $\mathbb{P}$ and $\mathbb{V}$ interact, across $\mathsf{k}+1$ rounds (exchanging $2\mathsf{k}+1$ messages), as follows.

1. For every round $i \in [\mathsf{k}]$ of the IOP system:
   (a) $\mathbb{P}$ computes the i-th IOP string $\pi_i \leftarrow \mathbf{P}\big(\mathbb{x}, \mathbb{w}, (r_j)_{j<i}\big)$ and commitment $(\mathsf{cm}_i, \mathsf{aux}_i) \leftarrow \mathsf{VC.Commit}(\pi_i)$ (along with private auxiliary information aux_i, used to compute VC openings), and sends cm_i to $\mathbb{V}$.
   (b) $\mathbb{V}$ samples the i-th IOP verifier randomness $r_i \leftarrow \{0,1\}^{\mathsf{r}_i}$ and sends it to $\mathbb{P}$.
2. $\mathbb{P}$ runs the IOP verifier $\mathbf{V}^{(\pi_i)_{i\in[\mathsf{k}]}}\big(\mathbb{x}, (r_i)_{i\in[\mathsf{k}]}\big)$ to deduce the query sets $(Q_i)_{i\in[\mathsf{k}]}$ (where $Q_i \subseteq [\mathsf{L}_i]$ is the set of queries to π_i), computes opening proofs $\mathsf{pf}_i \leftarrow \mathsf{VC.Open}(Q_i, \mathsf{aux}_i)$, sets $\mathsf{ans}_i := \pi_i(Q_i)$ for each $i \in [\mathsf{k}]$, and sends $((\mathsf{ans}_i, \mathsf{pf}_i))_{i\in[\mathsf{k}]}$ to $\mathbb{V}$.
3. $\mathbb{V}$ performs the following checks.
   (a) $\mathsf{VC.Check}(\mathsf{cm}_i, \mathsf{ans}_i, \mathsf{pf}_i) = 1$ for every $i \in [\mathsf{k}]$ (i.e., all answers have valid openings);
   (b) $\mathbf{V}^{(\mathsf{ans}_i)_{i\in[\mathsf{k}]}}\big(\mathbb{x}, (r_i)_{i\in[\mathsf{k}]}\big) = 1$ (i.e., the IOP verifier accepts the answers).

Above, $\mathbf{V}^{(\mathsf{ans}_i)_{i\in[\mathsf{k}]}}\big(\mathbb{x}, (r_i)_{i\in[\mathsf{k}]}\big)$ is the decision bit of the IOP verifier $\mathbf{V}$, given instance $\mathbb{x}$ and IOP randomness $(r_i)_{i\in[\mathsf{k}]}$, when each query q to the i-th oracle is answered with $\mathsf{ans}_i(q)$. (If $\mathbf{V}$ queries the i-th oracle outside of the domain of ans_i then the decision bit is 0.)

The formal definition of an interactive argument system is in Section 3.4, and the formal description of the IBCS protocol is in Section 3.5.

2.2 Limitations of prior analyses

Below, we review the two prior works most relevant to the post-quantum security of the IBCS protocol: [CDGS23], which gives a classical security analysis of the IBCS protocol; and [CMSZ21], which proves post-quantum security of Kilian's protocol, the special case of IBCS when instantiated with a PCP (rather than an IOP). We then explain the challenges that arise when we attempt to combine the two approaches.

2.2.1 Classical security of the IBCS protocol

[CDGS23] shows that an adversary $\widetilde{\mathbb{P}}$ for the argument verifier $\mathbb{V}$ of $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ can be efficiently converted into an adversary $\widetilde{\mathbf{P}}$ for the IOP verifier $\mathbf{V}$ of IOP that has similar convincing probability. For each $i \in [\mathsf{k}]$, $\widetilde{\mathbf{P}}$ generates the i-th IOP string by *rewinding* $\widetilde{\mathbb{P}}$: the IOP adversary takes a "snapshot" of $\widetilde{\mathbb{P}}$'s internal state at the start of the i-th round and reruns $\widetilde{\mathbb{P}}$ with different verifier challenges on the snapshot, until it gathers enough openings for the i-th commitment to reconstruct a "good" IOP string for round i.

Specifically, [CDGS23] constructs a *reductor* $\mathfrak{R}_{\mathsf{CDGS}}$ (a probabilistic algorithm) that, for a given round i, has oracle access to $\widetilde{\mathbb{P}}(\cdot, \mathsf{aux}_i)$ (where aux_i is $\widetilde{\mathbb{P}}$'s auxiliary state output along with the i-th commitment cm_i), receives as input prior commitments $(\mathsf{cm}_j)_{j \le i}$ and prior verifier randomness $(r_j)_{j<i} \in \{0,1\}^{\mathsf{r}_1} \times \cdots \times \{0,1\}^{\mathsf{r}_{i-1}}$, and outputs a (partially filled) IOP string $\widetilde{\pi}_i$.

$\mathfrak{R}_{\mathsf{CDGS}}^{\widetilde{\mathbb{P}}(\cdot, \mathsf{aux}_i)}\big((\mathsf{cm}_j)_{j \le i}, (r_j)_{j<i}\big)$:

1. Initialize an "empty" IOP string: set $\widetilde{\pi}_i := (\perp)^{\mathsf{L}_i}$.
2. Repeat the following $T = \mathsf{poly}(\lambda)$ times:
   (a) Sample IOP verifier randomness from round i to round k: $(r_i, \ldots, r_{\mathsf{k}}) \leftarrow \{0,1\}^{\mathsf{r}_i + \cdots + \mathsf{r}_{\mathsf{k}}}$.
   (b) Run $\widetilde{\mathbb{P}}((r_j)_{j \in [i,\mathsf{k}]}, \mathsf{aux})$ until the end of the interaction to obtain an answer-opening pair $(\mathsf{ans}_i, \mathsf{pf}_i)$ for the i-th IOP string.
   (c) If $\mathsf{VC.Check}(\mathsf{cm}_i, \mathsf{ans}_i, \mathsf{pf}_i) = 1$, set $\widetilde{\pi}_i(q) := \mathsf{ans}_i(q)$ for all queries q in the domain of ans_i. (If $\widetilde{\pi}_i(q)$ is already set previously, we choose an arbitrary answer for this location.)
3. Output $\widetilde{\pi}_i$.

The reductor $\mathfrak{R}_{\mathsf{CDGS}}$ is used to construct the adversary $\widetilde{\mathbf{P}}$ for IOP: for each round $i \in [\mathsf{k}]$, upon receiving the verifier's message, $\widetilde{\mathbf{P}}$ runs $\mathfrak{R}_{\mathsf{CDGS}}$ with appropriate inputs to obtain the i-th IOP string $\widetilde{\pi}_i$. The analysis in [CDGS23] argues that the success probability of $\widetilde{\mathbf{P}}$ cannot be too different from the success probability of $\widetilde{\mathbb{P}}$, thus bounding the soundness error of the IBCS protocol by the soundness error of its underlying IOP.

2.2.2 Post-quantum security of Kilian's protocol

Known security analyses of Kilian's protocol in the standard model [Kil92; BG08; CDGSY24] rely on rewinding, which is notoriously difficult in the quantum setting due to the no-cloning theorem. [CMSZ21] circumvents this "quantum rewinding" barrier to show post-quantum security of Kilian's protocol. They observe that fully restoring the prover's state is not necessary, rather, it suffices to "repair" the prover's *success probability* after each rewind; then they design a quantum algorithm $\mathsf{RepairState}$ that achieves this. (As mentioned in Section 1.1, the VC scheme used in Kilian's protocol must be *collapsing* for $\mathsf{RepairState}$ to work. We further elaborate on collapsing VC schemes in Section 2.2.3 and Section 2.3.)

With the help of $\mathsf{RepairState}$, [CMSZ21] also constructs a reductor $\mathfrak{R}_{\mathsf{CMSZ}}$ to extract a PCP string; given a commitment cm as input and (unitary) oracle access to $\widetilde{\mathbb{P}}$, the reductor outputs a PCP string $\widetilde{\pi}$. Note that since $\widetilde{\mathbb{P}}$ is a quantum algorithm, its internal state aux is a quantum state. Informally $\mathfrak{R}_{\mathsf{CMSZ}}$ works as follows.

$\mathfrak{R}_{\mathsf{CMSZ}}^{\widetilde{\mathbb{P}}}(\mathsf{cm}, \mathsf{aux})$:

1. Initialize an "empty" PCP string: set $\widetilde{\pi} \coloneqq (\perp)^{\mathsf{L}}$.
2. Let $\mathsf{aux}_1 \coloneqq \mathsf{aux}$.
3. For $i \in [T]$:
   (a) Sample PCP verifier randomness $r \leftarrow \{0,1\}^{\mathsf{r}}$.
   (b) Ask $\widetilde{\mathbb{P}}$ for answers to this randomness: $(\mathcal{A}, \mathcal{O}, \mathsf{aux}'_i) \leftarrow \widetilde{\mathbb{P}}(r, \mathsf{aux}_i)$, where the quantum registers $(\mathcal{A}, \mathcal{O})$ contain (superpositions of) answer-opening pairs $(\mathsf{ans}, \mathsf{pf})$.
   (c) Compute $\mathbb{V}(\mathbb{x}, r, \mathsf{cm}, \mathcal{A}, \mathcal{O})$ (coherently).[5] If the result is 1, measure $\mathcal{A}$ and $\mathcal{O}$ to obtain outcome ans and pf, and set $\widetilde{\pi}(q) \coloneqq \mathsf{ans}(q)$ for all queries q in the domain of ans.
   (d) Repair $\widetilde{\mathbb{P}}$'s state: $\mathsf{aux}_{i+1} \leftarrow \mathsf{RepairState}(\mathsf{aux}'_i)$.
4. Output $\widetilde{\pi}$.

As before, the goal is to show that, with high probability, $\mathfrak{R}_{\mathsf{CMSZ}}$'s output PCP $\widetilde{\pi}$ has similar success probability to $\widetilde{\mathbb{P}}$, which allows us to upper bound the soundness error of ARG via the soundness error of PCP. Specifically, one can show that, for appropriately chosen T, the probability that $\widetilde{\pi} = \pi^\star$ where $\pi^\star$ is a PCP with low success probability (say, $\Pr\left[\mathbf{V}^{\pi^\star}(\mathbb{x}) = 1\right] < \epsilon_{\widetilde{\mathbb{P}}}/20$, where $\epsilon_{\widetilde{\mathbb{P}}}$ is the probability that $\widetilde{\mathbb{P}}$ convinces the verifier $\mathbb{V}$) is exponentially small (in particular $\ll |\Sigma|^{-\mathsf{L}}$). Hence, via a union bound over all $|\Sigma|^{\mathsf{L}}$ possible PCPs, the probability that $\mathfrak{R}_{\mathsf{CMSZ}}$ outputs *any* such PCP is small.

Note that this reductor measures the opening register $\mathcal{O}$ in Item 3c, but it does not use the measurement outcome. This measurement is necessary in the [CMSZ21] analysis as it is used to obtain a contradiction to the *post-quantum position binding* property of the VC scheme in the case that the adversary produces inconsistent openings. In particular, to break this property the adversary must output a pair of classical answer-opening pairs that are inconsistent. As we discuss in detail later on, this proof strategy imposes undesirable consequences on the required post-quantum security properties for the underlying VC scheme.

2.2.3 Challenges in combining [CDGS23] and [CMSZ21]

As one may expect, our main result combines ideas from [CDGS23] and [CMSZ21]. Conceptually speaking, our reductor draws from both prior reductors: we replace the classical rewinding implicitly invoked by the [CDGS23] reductor with the "quantum rewinding" enabled by the state repair algorithm in [CMSZ21].

Carrying out this approach is far from straightforward, however. While adapting the state repair algorithm to this setting is not too difficult, showing that the extracted IOP prover $\widetilde{\mathbf{P}}$ is "as good as" the original argument prover $\widetilde{\mathbb{P}}$ requires new ideas. Here we outline why the approaches taken in [CDGS23] and [CMSZ21] fall short in our setting, and then in Section 2.3 we describe how we resolve these issues.

[CDGS23]: failure of the IID assumption. Recall that RepairState enables a "pseudo-rewinding" (i.e., repairs the state) of a quantum prover $\widetilde{\mathbb{P}}$ so that success probability is preserved. However, the [CDGS23] analysis critically relies on a stronger property of rewinding that is only known to hold in the classical setting.

The [CDGS23] analysis shows that the success probability of the argument adversary $\widetilde{\mathbb{P}}$ and the success probability of the extracted IOP prover $\widetilde{\mathbf{P}}$ cannot differ too much. Towards this end, they observe that, for a given choice of verifier randomness, the argument verifier $\mathbb{V}$'s and the IOP verifier $\mathbf{V}$'s decisions differ only if: (i) $\widetilde{\mathbb{P}}$ gives disagreeing answers to the same query while rewinding; or (ii) the IOP verifier $\mathbf{V}$ queries a location that is "missing" from an extracted IOP string. The first event implies a violation of position binding of the VC scheme in the classical setting (and, as we discuss in Section 2.3, violates a suitable binding notion in the quantum setting). Classically, the second event is bounded by appealing to the fact that

[5] Recall that the reductor $\mathfrak{R}_{\mathsf{CDGS}}$ in [CDGS23] (Section 2.2.1) records the answers whenever VC.Check accepts. Here, one has to check if the argument verifier $\mathbb{V}$ accepts before measuring in order to ensure RepairState works properly.

classical rewinding yields independent and identically distributed (IID) samples. In more detail, let δ_q be the probability that the q-th entry of the oracle is queried by the argument verifier and correctly revealed by the argument adversary. The probability that the q-th entry is "missing" is then, by the IID property, $(1-\delta_q)^T\delta_q \leq 1/T$, where T is the number of rewinds.

In the quantum setting, the state of the adversary *changes after each rewind*, so the samples obtained are no longer IID. Indeed, since the only guarantee we have from the state repair algorithm is that the prover's overall success probability is preserved, we cannot conclude anything about the probability δ_q across rewinds.

[CMSZ21]: failure of the union bound. The above issue arises in all classical security analyses of Kilian's protocol [Kil92; BG08; CDGSY24]. [CMSZ21] circumvents this issue via their novel "union bound" analysis described above, which does not require any assumption on the sample distribution that arises from rewinding except that the overall success probability is large enough.

Unfortunately, the union bound approach turns out to be hopeless in our setting. To achieve a nontrivial bound, we have to set the number of rewinds T to be at least logarithmic in the number of possible strategies we could extract. In the PCP setting, the number of strategies is the number of PCP strings $|\Sigma|^{\mathsf{L}}$, and so it suffices to set $T \approx \mathsf{L} \cdot \log|\Sigma|$, which is polynomial.

In stark contrast, an IOP prover strategy is a function $\{0,1\}^{\mathsf{r}} \to \Sigma^{\mathsf{L}}$, where r is the interaction randomness complexity of the verifier and L is the total length of all IOP strings. The number of such functions is $|\Sigma|^{\mathsf{L}\cdot 2^{\mathsf{r}}}$, which is *doubly exponential* in r.[6] Hence *we would need T to be exponential* to obtain any meaningful guarantee. This would lead to an (unacceptably expensive) exponential-time reduction.

Problems with CMSZ collapsing. As mentioned in Section 2.2.2, the state repair algorithm $\mathsf{RepairState}$ in [CMSZ21] relies on a *collapsing* property of the underlying vector commitment: roughly, for any quantum state $\boldsymbol{\rho}$ which is a superposition of valid answer-opening pairs $(\mathsf{ans}, \mathsf{pf})$, no efficient adversary can distinguish between $\boldsymbol{\rho}$ and the state obtained after measuring $\boldsymbol{\rho}$ in the computational basis.

However, the definition of collapsing VC schemes proposed in [CMSZ21], henceforth referred to as "CMSZ collapsing", has an undesirable feature: *it does not imply post-quantum position binding* (the standard binding notion for VC schemes). Hence, the security of Kilian's protocol in [CMSZ21] requires the underlying VC scheme to satisfy *both* post-quantum position binding *and* CMSZ collapsing. This is in contrast to standard commitment schemes, where collapse binding is known to imply "classical-style" post-quantum binding [Unr16b]; similarly, we would expect the "natural" definition of collapsing for a VC scheme to imply ("classical-style") post-quantum position binding. Note that requiring both properties is sufficient to establish security, but reducing assumptions in a security proof is always more desirable.

CMSZ collapsing does not imply post-quantum position binding because it refers only to superpositions of openings for a *single* subset Q. In contrast, post-quantum position binding requires that it is hard to produce openings for any subsets Q_1, Q_2 that disagree on $Q_1 \cap Q_2$.

A naive attempt to strengthen CMSZ collapsing to refer to superpositions of openings to different subsets fails. This is because CMSZ collapsing requires that a measurement of both the message *and opening* registers of a superposition be undetectable to an attacker. Achieving such a strong guarantee is impossible in this context: an attacker can simply generate a uniform superposition of valid openings to distinct sets Q_1 and Q_2; a measurement of the opening collapses the superposition, which is easily detectable.[7]

A natural attempt to overcome this problem is to only require that measurements of (parts of) the *message* register be undetectable; this is closer to the original definition of collapsing by Unruh (and, looking ahead,

[6] Strictly, not all such functions correspond to valid IOP strategies; however taking this into account would not significantly reduce this number.

[7] Even in the setting of standard commitment schemes, this "strong" collapse binding notion is brittle: any commitment scheme can be rendered insecure with respect to this notion by, for example, adding a single bit to the opening that is not used.

the form our new definition will take). Unfortunately, the security proof of CMSZ relies crucially on the ability to measure openings, in order to appeal to post-quantum position binding.

Thus the "right" definition of collapsing for a VC scheme has remained elusive.

2.3 Our approach

We give an overview of our proof of Theorem 1, which overcomes all of the aforementioned challenges. The formal statement and proof of our main result, including a careful accounting of security losses in the reduction, is presented in Sections 5 and 6.

2.3.1 A new definition of collapse binding for VCs

We propose a new definition for a VC scheme VC, which we call *collapse position binding*, that alone suffices for establishing the security of $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ (i.e., suffices for Theorem 1). In particular, our definition implies post-quantum position binding, overcoming the limitations of CMSZ collapsing discussed in Section 2.2.3 (as well as other limitations discussed in Remark 4.3).

Recall that in the opening phase of a vector commitment scheme, the committer sends a pair $(\mathsf{ans}, \mathsf{pf})$, where $\mathsf{ans}\colon Q \to \Sigma$ for some query set $Q \subseteq [L]$ and pf is an opening proof. Informally, a VC scheme is collapse position binding if, for every index i, given a superposition of valid openings $(\mathsf{ans}, \mathsf{pf})$ such that $\mathsf{ans}(i) \neq \bot$, no efficient quantum algorithm can distinguish whether $\mathsf{ans}(i)$ is measured. This must hold even for superpositions over many *different* query sets Q containing i. This requirement is incomparable to CMSZ collapsing: that definition requires the stronger guarantee that measuring all of $(\mathsf{ans}, \mathsf{pf})$ is undetectable, but only for superpositions of openings of a single query set Q.

Definition 1. *A vector commitment scheme* $\mathsf{VC} = (\mathsf{VC.Commit}, \mathsf{VC.Open}, \mathsf{VC.Check})$ *is* **collapse position binding** *if every polynomial-size quantum circuit A can distinguish the cases $b \in \{0,1\}$ in the experiment* $\mathsf{CollapseBindingExp}(b)$ *(defined below) with only negligible probability.*

$\mathsf{CollapseBindingExp}(b)$*:*

1. *Ask A to output a commitment* cm*, location* $\mathsf{idx} \in [L]$*, and quantum registers $(\mathcal{A}, \mathcal{O})$ containing a superposition supported on answer-opening pairs* $(\mathsf{ans}, \mathsf{pf})$ *for which* idx *is in the domain of* ans*.*
2. *Compute* $\mathsf{VC.Check}(\mathsf{cm}, \mathcal{A}, \mathcal{O})$*, aborting if the result is* 0*. If $b = 1$, measure $\mathcal{A}$ at coordinate* idx*.*
3. *Output $A(\mathcal{A}, \mathcal{O})$.*

The definition above is obtained via a novel "lifting" of security definitions for commitment schemes into their positional equivalents for vector commitment schemes. Given a vector commitment scheme VC, we derive for every position $i \in [L]$ a (not necessarily hiding) bit commitment scheme CM_i as follows. Committing to a bit b consists of a VC commitment to the vector $b \cdot e_i$ (the vector with b at position i and 0 everywhere else); opening consists of a VC opening to *any* subset containing i. We show that post-quantum position binding for VC is equivalent to post-quantum binding for CM_i for every i. Similarly, our Definition 1 is designed so that we can prove its equivalence to *collapse binding* for CM_i for every i (per the collapse binding definition in [Unr16a] for bit commitment schemes). Since collapse binding implies post-quantum binding for bit commitments, we can immediately deduce via lifting that Definition 1 implies post-quantum position binding.

Our collapse position binding definition is attainable: it is straightforward to prove that a Merkle commitment scheme based on a collapsing hash function family ([Unr16b]) is a collapse position binding vector commitment scheme. ([CMSZ21] shows that if the hash function family is collapsing then the Merkle

commitment scheme is CMSZ collapsing; their analysis can be adapted to our definition.) In turn, collapsing hash function families are known to exist assuming the quantum hardness of standard LWE [Unr16a].

2.3.2 Unlocking multiple rounds via a random stopping time

The classical security analysis of $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ sketched in Section 2.2.1 relies on the fact that the argument adversary $\widetilde{\mathbb{P}}$ has the same local behavior for every query $q \in [\mathsf{L}_i]$ after each rewind, because its state is *identical*. However, in quantum rewinding, we have *no control over $\widetilde{\mathbb{P}}$'s state beyond its success probability*. Preservation of the success probability does not rule out an argument adversary $\widetilde{\mathbb{P}}$ that adopts a different strategy after each rewind. As discussed in Section 2.2.3, [CMSZ21] circumvents this issue for Kilian's protocol via an analysis that *cannot* be applied to $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$.

We resolve this problem by employing a *random stopping time*, as follows. The total number of query positions $q \in [\mathsf{L}_i]$ filled in by the reductor $\mathfrak{R}$ is (trivially) at most L_i. As a countermeasure against adversaries that change strategies across rewindings, we modify the reductor $\mathfrak{R}$ so that it rewinds the argument adversary $\widetilde{\mathbb{P}}$ a random $\mathsf{t} \leftarrow [T]$ number of times (for some T set appropriately). We prove that the probability that there is a missing position $q \in [\mathsf{L}_i]$, using this random stopping time, is upper bounded by L_i/T.

We note that the random stopping time only needs to appear in the analysis rather than the reductor $\mathfrak{R}$ itself, as rewinding more than t times can only improve our success probability. However, since we do not know how to benefit from these additional rewindings, we directly incorporate the random stopping time in the reductor $\mathfrak{R}$ itself.

2.3.3 Our quantum reductor

We wish to transform an argument adversary $\widetilde{\mathbb{P}}$ for $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ into an IOP adversary $\widetilde{\mathbf{P}}$ for IOP (which uses $\widetilde{\mathbb{P}}$ as a black-box). Note that $\widetilde{\mathbb{P}}$ is a quantum algorithm that outputs $\mathsf{k}+1$ messages (commitments $\mathsf{cm}_1, \ldots, \mathsf{cm}_\mathsf{k}$ in the first k rounds and openings in the $(\mathsf{k}+1)$-th round) whereas $\widetilde{\mathbf{P}}$ is a quantum algorithm that outputs k messages (IOP strings $\widetilde{\pi}_1, \ldots, \widetilde{\pi}_\mathsf{k}$). This transformation is enabled by a quantum reductor $\mathfrak{R}$ that, for any given $i \in [\mathsf{k}]$, is tasked with producing $\widetilde{\pi}_i$, given the first i commitments $(\mathsf{cm}_j)_{j \le i}$, the first $i-1$ verifier randomness $(r_j)_{j<i}$, the first $i-1$ IOP strings $(\widetilde{\pi}_j)_{j<i}$ and an auxiliary prover state aux_j for after $\widetilde{\mathbb{P}}$ outputs the i-th commitment.

Specifically, adapting our collapse position binding definition (Section 2.3.1) and applying the random stopping time trick (Section 2.3.2), our reductor $\mathfrak{R}$ (in the i-th round) informally works as follows.

$\mathfrak{R}^{\widetilde{\mathbb{P}}}\big((\mathsf{cm}_j)_{j \le i}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j<i}, \mathsf{aux}_i\big)$:

1. Initialize an "empty" IOP string: set $\widetilde{\pi}_i \coloneqq (\bot)^{\mathsf{L}_i}$.
2. Set $\mathsf{aux}_{i,1} \coloneqq \mathsf{aux}_i$.
3. Sample the random stopping time $\mathsf{t} \leftarrow [T]$.
4. For $\ell \in [\mathsf{t}]$:
   (a) Sample IOP verifier randomness from round i to round k: $(r_i, \ldots, r_\mathsf{k}) \leftarrow \{0,1\}^{\mathsf{r}_i + \cdots + \mathsf{r}_\mathsf{k}}$.
   (b) Run $\widetilde{\mathbb{P}}$ on the concatenation of the random strings: $((\mathcal{C}_j)_{j>i}, ((\mathcal{A}_j, \mathcal{O}_j))_{j \ge i}, \mathsf{aux}'_{i,\ell}) \leftarrow \widetilde{\mathbb{P}}((r_j)_{j \in [\mathsf{k}]}, \mathsf{aux}_{i,\ell})$, where for every j, the quantum register $\mathcal{C}_j$ contains (superpositions of) the j-th commitment, and the quantum registers $(\mathcal{A}_j, \mathcal{O}_j)$ contain (superpositions of) query-answer pairs $(\mathsf{ans}_j, \mathsf{pf}_j)$ for the j-th IOP string.
   (c) Compute $\mathbb{V}(\mathbb{x}, (r_j)_{j \in [\mathsf{k}]}, (\widetilde{\pi}_j)_{j<i}, (\mathsf{cm}_j)_{j \le i}, (\mathcal{C}_j)_{j>i}, ((\mathcal{A}_j, \mathcal{O}_j))_{j \ge i})$ (coherently). If the result is 1, measure $\mathcal{A}_i$ to obtain ans_i and set $\widetilde{\pi}_i(q) \coloneqq \mathsf{ans}_i(q)$ for all q in the domain of ans_i.[8]

[8] We further optimize this step in Section 5: we only measure $\mathcal{A}_i$ if $\mathcal{A}_i$ contains any new queries that are not already set in $\widetilde{\pi}_i$, i.e.,

(d) Repair $\widetilde{\mathbb{P}}$'s state: $\mathsf{aux}_{i,\ell+1} \leftarrow \mathsf{RepairState}(\mathsf{aux}'_{i,\ell})$.

5. Output $\widetilde{\pi}_i$.

We prove Theorem 1 via a hybrid analysis, which "interpolates" between the argument system ARG and its underlying IOP one round at a time. More precisely, we define a sequence of hybrid protocols $(\mathsf{H}_i)_{i\in[\mathsf{k}]}$ where H_i has the IBCS transformation applied only to the last $\mathsf{k}-i$ rounds of the IOP.

The i-th hybrid is an interactive protocol $\mathsf{H}_i = (\widetilde{\mathcal{P}}_i, \mathcal{V}_i)$ constructed via our reductor $\mathfrak{R}$ in the first i rounds. $\widetilde{\mathcal{P}}_i$ uses $\mathfrak{R}$ to construct the IOP strings $(\widetilde{\pi}_j)_{j\in[i]}$ and, in the remaining rounds, $\widetilde{\mathcal{P}}_i$ runs the argument adversary $\widetilde{\mathbb{P}}$; the hybrid verifier $\mathcal{V}_i$ accordingly receives IOP strings in rounds $j \in [i]$ and VC commitments in rounds $j > i$ (to be opened after the interaction as needed). Note that H_0 corresponds to the IBCS protocol $(\widetilde{\mathbb{P}}, \mathbb{V})$ and $\mathsf{H}_{\mathsf{k}} = (\widetilde{\mathcal{P}}_{\mathsf{k}}, \mathcal{V}_{\mathsf{k}})$ corresponds to the IOP $(\widetilde{\mathbf{P}}, \mathbf{V})$.

The advantage of this perspective is that to move from a H_i-adversary $\widetilde{\mathcal{P}}_i$ to a H_{i+1}-adversary $\widetilde{\mathcal{P}}_{i+1}$, we only need to rewind the H_i-adversary in round $i+1$. Using the "quantum game" framework [CMSZ21] (recalled in Section 3.6), we show that the pre- and post-repair states have similar success probability, which allows the H_i-adversary to be rewound (up to T times) without a significant drop in success probability.

This leaves to prove that the success probabilities of hybrid adversaries $\widetilde{\mathcal{P}}_i$ and $\widetilde{\mathcal{P}}_{i+1}$ are close. Recall that they only differ in the $(i+1)$-th round: $\widetilde{\mathcal{P}}_i$ answers with the argument adversary $\widetilde{\mathbb{P}}$'s answer-opening pairs, while $\widetilde{\mathcal{P}}_{i+1}$ answers with the IOP string $\widetilde{\pi}_{i+1}$ extracted by the reductor $\mathfrak{R}$. Therefore, if $\mathcal{V}_i$'s decision differs from $\mathcal{V}_{i+1}$'s then either: (i) $\mathfrak{R}$ fails to fill in a query location $q \in [\mathsf{L}_i]$ queried by $\mathcal{V}_{i+1}$; or (ii) $\widetilde{\mathbb{P}}$ outputs two different answers for the same query location. By the random stopping time argument outlined in Section 2.3.2, the first event happens with small probability.

The second event is analogous to a violation of post-quantum position binding. However, post-quantum position binding does not suffice here. Recall that post-quantum position binding requires that no efficient quantum adversary can find two inconsistent classical openings for the same classical commitment. To adhere to our collapse position binding definition (Definition 1, which does not guarantee that measuring $\mathcal{O}$ is undetectable), the reductor $\mathfrak{R}$ used to construct the VC adversary cannot measure the opening register $\mathcal{O}$ (as opposed to the reductor outlined in Section 2.2.2). Instead, as an intermediate step, we introduce a property implied by collapse position binding called *unique-message position binding*, derived from the analogous property for standard commitments [LMS22b] via our lifting from Section 2.3.1. (More precisely, by lifting the implication of unique-message binding for commitments from collapse binding [LMS22b].) Unique-message position binding allows us to demonstrate a binding violation while leaving the opening register in superposition. We conclude that the second event happens with small probability unless $\widetilde{\mathbb{P}}$ violates unique-message position binding, and therefore unless it violates collapse position binding.

Discussion of our security bounds. The proof outline above leads to the bound in Theorem 1: the post-quantum soundness error ϵ_{ARG} of $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ is at most $\epsilon_{\mathsf{IOP}} + \mathrm{negl}(\lambda)$, where ϵ_{IOP} is the soundness error of IOP (and similarly for knowledge soundness). In fact, we obtain the following concrete bound for security parameter λ against t_{ARG}-size adversaries against $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$:

$$\epsilon_{\mathsf{ARG}}(\lambda, t_{\mathsf{ARG}}) \leq \epsilon_{\mathsf{IOP}} + \sum_{i\in[\mathsf{k}]} \mathsf{L}_i \cdot \mathsf{q}_i \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, t_{\mathsf{VCCollapse}}) + \epsilon \quad \text{where} \quad t_{\mathsf{VCCollapse}} = \mathsf{poly}\left(\frac{\mathsf{L}}{\epsilon}\right) \cdot (t_{\mathsf{ARG}} + t_{\mathsf{Repair}}) \ .$$

Above, $\epsilon_{\mathsf{VCCollapse}}(\lambda, t_{\mathsf{VCCollapse}})$ is the collapse position binding error of VC (see Definition 1) for security parameter λ against $t_{\mathsf{VCCollapse}}$-size quantum adversaries, and t_{Repair} is the running time of the repair state procedure $\mathsf{RepairState}$. The parameter $\epsilon > 0$ is a rewinding tradeoff (more rewinds implies smaller error).

we make at most L_i measurements inside this loop.

For comparison, the classical soundness error of $\mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ ([CDGS23; CGKY25]) is bounded as

$$\epsilon_{\mathsf{ARG}}(\lambda, t_{\mathsf{ARG}}) \leq \epsilon_{\mathsf{IOP}} + \mathsf{k} \cdot \epsilon_{\mathsf{VC}}(\lambda, t_{\mathsf{VC}}) + \epsilon \quad \text{where} \quad t_{\mathsf{VC}} = O\left(\frac{\mathsf{L}}{\epsilon} \cdot t_{\mathsf{ARG}}\right) \ ,$$

and there are barriers to noticeably improving this bound [CDGSY24].

We highlight the differences between the bounds in the classical and quantum settings.

- $\mathsf{k} \cdot \epsilon_{\mathsf{VC}}(\lambda, t_{\mathsf{VC}})$ vs. $\sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot \mathsf{q}_i \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, t_{\mathsf{VCCollapse}})$: The error term $\mathsf{k} \cdot \epsilon_{\mathsf{VC}}(\lambda, t_{\mathsf{VC}})$ in the classical analysis is because the position binding of VC can be violated for any of the commitments $(\mathsf{cm}_i)_{i \in [\mathsf{k}]}$. In the quantum case, we pay for $\epsilon_{\mathsf{VCCollapse}}$ every time we measure the answer register. Since, for every $i \in [\mathsf{k}]$, we make at most $\mathsf{L}_i \cdot \mathsf{q}_i$ measurements, overall we obtain the term $\sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot \mathsf{q}_i \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, t_{\mathsf{VCCollapse}})$.
- t_{VC} vs. $t_{\mathsf{VCCollapse}}$: The running time t_{VC} of the VC adversary in the classical analysis is the running time t_{ARG} of the adversary $\widetilde{\mathbb{P}}$ times the number $\frac{\mathsf{L}}{\epsilon}$ of rewindings. In the quantum case, the rewind and repair process can fail; roughly, to obtain $\frac{\mathsf{L}}{\epsilon}$ valid rewindings, we need to rewind and repair for $\mathsf{poly}\left(\frac{\mathsf{L}}{\epsilon}\right)$ times. Thus, the running time $t_{\mathsf{VCCollapse}}$ of the VC adversary is (roughly) $t_{\mathsf{ARG}} + t_{\mathsf{Repair}}$ times $\mathsf{poly}\left(\frac{\mathsf{L}}{\epsilon}\right)$.

Reducing the gap between the quantum analysis and classical analysis (in terms of the respective security properties of the VC scheme) remains a challenging open question. In particular, improving the efficiency of the quantum rewinding subroutine would considerably improve the efficiency of the security reduction.

Remark 2.1 (adaptive verifiers)**.** Surprisingly, and contrary to [CDGS23], our quantum reduction does not work for public-coin IOPs whose verifier is *adaptive* (each query may depend on the answers to prior queries). Indeed, an adaptive verifier may, in particular, choose its queries to a proof string π_i based on the answers to queries to a *later* proof string π_j ($j > i$). In our quantum reduction, this means that the register holding Q_i (i.e., the domain of ans_i) may be *entangled* with the register containing ans_j for $j > i$, so measuring Q_i may cause ans_j to collapse. This is problematic, because at this point $\widetilde{\mathbb{P}}$ has yet to send a commitment to ans_j, and so we cannot appeal to the collapsing property of VC to argue that this measurement is undetectable.

Nevertheless we are able to establish security for a limited, yet expressive, form of adaptivity. Specifically, our quantum reduction works for *semi-adaptive* verifiers, which are adaptive verifiers for which queries to a proof string cannot depend on later proof strings (but can depend on earlier proof strings); see Definition 3.5. This class of verifiers captures notable special cases.

- *Non-adaptive IOPs* (queries by IOP verifier depend only on the instance and verifier randomness). Essentially all IOP constructions of practical interest are non-adaptive IOPs.
- *Adaptive PCPs* (the PCP verifier can adaptively query the PCP string with no restrictions). Hence our results extends [CMSZ21], whose analysis implicitly assumes the ability to compute query sets from (only) the verifier's randomness, and thus is limited to non-adaptive PCPs.

It remains a fascinating open question to determine whether this indicates that the IBCS protocol is *not* post-quantum secure for some adaptive-verifier IOP, or if it is merely a limitation of current techniques.

3 Preliminaries

Relations. A relation R is a set of pairs $(\mathbb{x}, \mathbb{w})$ where $\mathbb{x}$ is an instance and $\mathbb{w}$ is a witness. The corresponding language $L(R)$ is the set of instances $\mathbb{x}$ for which there exists a witness $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in R$.

Sets, strings, sequences. We define $[i] \coloneqq \{1, 2, \ldots, i\}$ and $[i, j] \coloneqq \{i, i+1, \ldots, j\}$. We write $s \leftarrow S$ to denote a uniformly random sample s drawn from a set S. We assume that sets $S \subseteq [i]$ are represented via a canonical (e.g., increasing) sequence of their elements. Strings are sequences of symbols in a finite alphabet Σ. We denote by $|\mathbb{x}|$ the length of string $\mathbb{x}$ and by $|C|$ the size of a circuit C (besides $|S|$ for the size of set S). We usually distinguish nested sequences syntactically, e.g., $(x_1, x_2, \ldots, x_i) \neq \big(x_1, (x_2, \ldots, x_i)\big)$; however we sometimes abuse notation denoting the concatenation $(r_1, \ldots, r_i, r'_1, \ldots, r'_j)$ of strings $(r_\ell)_{\ell \in [i]}$ and $(r'_\ell)_{\ell \in [j]}$ by $\big((r_\ell)_{\ell \in [i]}, (r'_\ell)_{\ell \in [j]}\big)$.

Negligible functions. We denote the *security parameter* by λ. A function $f \colon \mathbb{N} \to [0, 1]$ is negligible, denoted $f(\lambda) = \mathrm{negl}(\lambda)$, if $f(\lambda) = o(\lambda^{-k})$ for all $k \in \mathbb{N}$ (f is asymptotically smaller than any inverse polynomial). A probability is *overwhelming* if is at least $1 - f(\lambda)$ for a negligible function f. We often abuse notation, writing $\mathrm{negl}(\lambda)$ to denote negligible functions and $1 - \mathrm{negl}(\lambda)$ for overwhelming ones.

Functions. We represent partial functions $f \colon [\ell] \to Y$ defined at $X \subseteq [\ell]$ by a sequence $(y_1, y_2, \ldots, y_\ell) \in (Y \cup \{\bot\})^\ell$ where $y_i = f(i)$ if $i \in X$ and $y_i = \bot$ if $i \notin X$ (the symbol $\bot \notin Y$ indicates f is undefined at i).

The support of a partial function f is $\mathsf{supp}(f) \coloneqq \{x \in [\ell] : f(x) \neq \bot\}$. We sometimes abuse notation writing $f \colon X \to Y$ when $X = \mathsf{supp}(f)$; or writing $f = y|_X$ when $y \in Y^\ell$ and $X \subseteq [\ell]$ for the partial function such that $f(i) = y_i$ when $i \in X$. For a subset $S \subseteq \mathsf{supp}(f)$ with elements $s_1 < s_2 < \cdots < s_{|S|}$, we write $f(S)$ to denote the vector $(f(s_1), \ldots, f(s_n))$. To distinguish partial from total functions $g \colon [\ell] \to Y$, we reserve $g \in Y^\ell$ for the latter.

Given a partial function $f \colon X \to Y$ defined at $X \subseteq [\ell]$, we denote by $f[z \to y]$ the function $g \colon X \cup \{z\} \to Y$ that is consistent with f over X (i.e. such that $g(x) = f(x)$ for all $x \in X$) and such that $g(z) = y$. We also use $f[z \to \bot]$ to denote $f|_{X \setminus \{z\}}$, consistent with the representation of partial functions.

Decision problems, algorithms, protocols. We denote by $A^y(x)$ the output of an algorithm A with *oracle* access to function/circuit or string y and *explicit* access to x; when A is randomized and its internal randomness is r, we also write $A^y(x, r)$. Equivalently, an execution of A (with oracle and explicit access as before) that yields output z is denoted $z \leftarrow A^y(x)$. Additionally, A may also output auxiliary information reserved for itself in future runs or iterations (e.g., of an interactive protocol); if so, the auxiliary information is denoted $(z, \mathtt{aux}) \leftarrow A^y(x)$ (as the last output). We highlight that $\mathtt{aux}$ may include quantum information if A is a quantum algorithm (which we usually represent as a quantum state $\boldsymbol{\rho}$ when access to A is black-box; or as $\mathtt{aux} = (\mathtt{aux}', \boldsymbol{\rho})$ with $\mathtt{aux}'$ classical when A is constructed explicitly).

We often refer to decision problems and algorithms with binary output; we associate the bit 0 with false and 1 with true, and also allow such algorithms to abort by outputting a symbol $\mathtt{Abort}$. If f is a partial function and A queries a point outside of the domain of x (which evaluates to $\bot$), we assume A outputs 0.

Interactive protocols are denoted with angled brackets: if A and B receive inputs x and y, respectively, and interact to produce output z, we write $z \leftarrow \langle A(x), B(y) \rangle$.

3.1 Quantum preliminaries and notation

Quantum information. A (pure) *quantum state* is a vector $|\psi\rangle$ in a complex Hilbert space $\mathcal{H}$ with $\||\psi\rangle\| = 1$; in this work, $\mathcal{H}$ is finite-dimensional. We denote by $\boldsymbol{S}(\mathcal{H})$ the space of Hermitian operators on $\mathcal{H}$. A *density matrix* is a Hermitian operator $\boldsymbol{\rho} \in \boldsymbol{S}(\mathcal{H})$ with $\mathsf{Tr}(\boldsymbol{\rho}) = 1$. A density matrix represents a probabilistic mixture of pure states (a mixed state); the density matrix corresponding to the pure state $|\psi\rangle$ is $|\psi\rangle\langle\psi|$. We

divide a Hilbert space into *registers*, e.g., $\mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2$; we sometimes slightly abuse notation and write $(\mathcal{H}_1, \mathcal{H}_2)$ for their tensor product. (For notational consistency, we use boldface greek letters for mixed states and calligraphic for quantum registers.) We sometimes write $\boldsymbol{\rho}_{\mathcal{H}_1}$ to specify that $\boldsymbol{\rho} \in \boldsymbol{S}(\mathcal{H}_1)$.

A unitary operation is represented by a complex matrix U satisfying $UU^\dagger = \mathbf{I}$. The operation U transforms the pure state $|\psi\rangle$ to the pure state $U|\psi\rangle$, and the density matrix $\boldsymbol{\rho}$ to the density matrix $U\boldsymbol{\rho}U^\dagger$; the latter mapping is sometimes denoted $\boldsymbol{\rho}' \leftarrow U(\boldsymbol{\rho})$ (where $\boldsymbol{\rho}' = U\boldsymbol{\rho}U^\dagger$). We often omit identity operators when applying a unitary U that acts on $\mathcal{H}_1$ to $\mathcal{H}_1 \otimes \mathcal{H}_2$; i.e., we use U to denote $U \otimes \mathbf{I}$ (where $\mathbf{I}$ acts on $\mathcal{H}_2$).

A *projector* Π is a Hermitian operator ($\Pi^\dagger = \Pi$) such that $\Pi^2 = \Pi$. A *projective measurement* is a collection of projectors $\mathsf{P} = (\Pi_i)_{i \in S}$ such that $\sum_{i \in S} \Pi_i = \mathbf{I}$. This implies that $\Pi_i \Pi_j = 0$ for distinct i and j in S. Applying a projective measurement to a pure state $|\psi\rangle$ yields outcome $i \in S$ with probability $p_i = \|\Pi_i |\psi\rangle\|^2$, and the post-measurement state is $|\psi_i\rangle = \Pi_i |\psi\rangle / \sqrt{p_i}$. We sometimes refer to the post-measurement state $|\psi_i\rangle = \Pi_i |\psi\rangle / \sqrt{p_i}$ as the result of applying $\mathsf{P} = (\Pi_i)_{i \in S}$ to $|\psi\rangle$ and *post-selecting* (i.e. conditioning) on outcome i. A state $|\psi\rangle$ is an *eigenstate* of P if it is an eigenstate of every Π_i.

A two-outcome projective measurement is called a *binary projective measurement*, and is written as $\mathsf{P} = (\Pi, \mathbf{I} - \Pi)$, where Π is associated with the outcome 1, and $\mathbf{I} - \Pi$ is associated with the outcome 0.

General (non-unitary) evolution of a quantum state can be represented via a *completely-positive trace-preserving (CPTP)* map $\mathbf{T} \colon \boldsymbol{S}(\mathcal{H}) \to \boldsymbol{S}(\mathcal{H}')$. We omit the precise definition of these maps; we only use the facts that they are trace-preserving (for every $\boldsymbol{\rho} \in \boldsymbol{S}(\mathcal{H})$ it holds that $\mathsf{Tr}(\mathbf{T}(\boldsymbol{\rho})) = \mathsf{Tr}(\boldsymbol{\rho})$) and linear.

For every CPTP map $\mathbf{T} \colon \boldsymbol{S}(\mathcal{H}) \to \boldsymbol{S}(\mathcal{H})$ there exists a *unitary dilation* U that operates on an expanded Hilbert space $\mathcal{H} \otimes \mathcal{L}$ and satisfies $\mathbf{T}(\boldsymbol{\rho}) = \mathsf{Tr}_{\mathcal{L}}(U(\boldsymbol{\rho} \otimes |0\rangle\langle 0|^{\mathcal{L}})U^\dagger)$. This is not necessarily unique; however, if $\mathbf{T}$ is described as a circuit then there is a dilation $U_{\mathbf{T}}$ represented by a circuit of size $O(|\mathbf{T}|)$.

For Hilbert spaces $\mathcal{A}$ and $\mathcal{B}$, the *partial trace* over $\mathcal{B}$ is the unique CPTP map $\mathbf{Tr}_{\mathcal{B}} \colon \boldsymbol{S}(\mathcal{A} \otimes \mathcal{B}) \to \boldsymbol{S}(\mathcal{A})$ such that $\mathbf{Tr}_{\mathcal{B}}(\boldsymbol{\rho}_{\mathcal{A}} \otimes \boldsymbol{\rho}_{\mathcal{B}}) = \mathbf{Tr}(\boldsymbol{\rho}_{\mathcal{B}})\boldsymbol{\rho}_{\mathcal{A}}$ for every $\boldsymbol{\rho}_{\mathcal{A}} \in \boldsymbol{S}(\mathcal{A})$ and $\boldsymbol{\rho}_{\mathcal{B}} \in \boldsymbol{S}(\mathcal{B})$. We say that the partial trace of $\boldsymbol{\rho}_{\mathcal{A}\mathcal{B}}$ after *tracing out* system $\mathcal{B}$ is the density matrix $\boldsymbol{\rho}_{\mathcal{A}}$.

A *general measurement* is a CPTP map $\mathbf{M} \colon \boldsymbol{S}(\mathcal{H}) \to \boldsymbol{S}(\mathcal{H} \otimes \mathcal{Y})$, where $\mathcal{Y}$ is an ancilla register holding a classical outcome. Specifically, given measurement operators $\{M_i\}_{i=1}^N$ such that $\sum_{i=1}^N M_i M_i^\dagger = \mathbf{I}$ and a basis $\{|i\rangle\}_{i=1}^N$ for $\mathcal{Y}$, $\mathbf{M}(\boldsymbol{\rho}) \coloneqq \sum_{i=1}^N (M_i \boldsymbol{\rho} M_i^\dagger \otimes |i\rangle\langle i|_{\mathcal{Y}})$. We sometimes implicitly discard the outcome register. A projective measurement is simply a general measurement where the M_i are projectors. A measurement induces a probability distribution over its outcomes given by $\Pr[i] = \mathsf{Tr}(|i\rangle\langle i|_{\mathcal{Y}} \mathbf{M}(\boldsymbol{\rho}))$; we denote sampling from this distribution by $i \leftarrow \mathbf{M}(\boldsymbol{\rho})$.

Quantum algorithms. We consider quantum algorithms with classical and quantum inputs and (classical and quantum) outputs. An algorithm A that receives input register $\mathcal{X}$ and outputs register $\mathcal{Y}$ (corresponding to some CPTP map $\boldsymbol{S}(\mathcal{X}) \to \boldsymbol{S}(\mathcal{Y})$) is denoted $\mathcal{Y} \leftarrow A(\mathcal{X})$. If its input and output states are $\boldsymbol{\rho} \in \boldsymbol{S}(\mathcal{X})$ and $\boldsymbol{\sigma} \in \boldsymbol{S}(\mathcal{Y})$, respectively, we write $\boldsymbol{\sigma} \leftarrow A(\boldsymbol{\rho})$; when A is an interactive algorithm, we may also write $\mathcal{Y}' \leftarrow A(\boldsymbol{\rho})$ when $\mathcal{Y} = \mathcal{Y}' \otimes \mathcal{Y}''$ (and A only sends register $\mathcal{Y}'$ to the other party) or $\boldsymbol{\sigma} \leftarrow A(\mathcal{X}')$ when $\mathcal{X} = \mathcal{X}' \otimes \mathcal{X}''$ (and A retains register $\mathcal{X}''$ from past interaction).

Quantum algorithms may also compute functions in superposition, or do so as a subroutine. Given a function $f \colon X \to Y$ and quantum registers $\mathcal{X}, \mathcal{Y}$ with bases $\{|x\rangle\}_{x \in X}, \{|y\rangle\}_{y \in Y}$ for their respective Hilbert spaces, we use $f(\mathcal{X})$ to denote a coherent application of f: applying the unitary $|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle$ on $\mathcal{X} \otimes \mathcal{Y}$ and outputting the register $\mathcal{Y}$. We assume an embedding of Y into $\{0,1\}^{\lceil \log_2 |Y| \rceil}$, so the operation $\oplus$ is well-defined.

We call a *measurement of* $f(\mathcal{X})$, denoted $y \leftarrow f(\mathcal{X})$, the coherent application of f followed by measuring $\mathcal{Y}$ then uncomputing f (and outputting the measurement outcome y). When $\mathcal{X} = \otimes_{i \in [\ell]} \mathcal{X}_i$, we use $\mathcal{X}_i$ to denote the i-th register and $\mathcal{X}(i)$ to denote a measurement of the function $(x_1, \ldots, x_\ell) \mapsto x_i$ so as to avoid ambiguity. We sometimes slightly abuse notation, writing $f^{\mathcal{Y}}(\mathcal{X})$ to denote $f(\mathcal{X}, \mathcal{Y})$.

A *quantum adversary* is a family of quantum circuits $\{\mathsf{Adv}_\lambda\}_{\lambda\in\mathbb{N}}$ represented classically via some standard universal gate set. A quantum adversary has *polynomial size* if there exist a polynomial p and $\lambda_0 \in \mathbb{N}$ such that $|\mathsf{Adv}_\lambda| \leq p(\lambda)$ for every $\lambda > \lambda_0$ (quantum adversaries have classical non-uniform advice).

A *quantum prover* $\mathbb{P}$ in a k-round interactive protocol is described by a sequence of k unitaries $(U_{\mathbb{P}_1}, U_{\mathbb{P}_1}, \ldots, U_{\mathbb{P}_k})$, corresponding to the unitary dilations of the algorithm applied by $\mathbb{P}$ at each round (to its internal state and verifier message).

Black-box access. A circuit $\mathcal{C}$ with black-box access to a unitary U, denoted $\mathcal{C}^U$, is a standard quantum circuit with special gates that act as U and $U^\dagger$. We also use $\mathcal{C}^{\mathbf{T}}$ to denote black-box access to a map $\mathbf{T}$, which we interpret as $\mathcal{C}^{U_{\mathbf{T}}}$ for a unitary dilation $U_{\mathbf{T}}$ of $\mathbf{T}$; all of our results are independent of the choice of dilation. This allows, for example, the "partial application" of a projective measurement, and the implementation of a general measurement via a projective measurement on a larger space.

3.2 Interactive oracle proofs

An interactive oracle proof (IOP) system [BCS16; RRR16] is a generalization of probabilistically checkable proofs to multiple rounds. An IOP system for a relation R is defined as follows.

Definition 3.1 (Completeness)**.** $\mathsf{IOP} = (\mathbf{P}, \mathbf{V})$ *for a relation R has* **perfect completeness** *if for every instance-witness pair $(\mathbb{x}, \mathbb{w}) \in R$,*

$$\Pr\left[\langle \mathbf{P}(\mathbb{x}, \mathbb{w}), \mathbf{V}(\mathbb{x})\rangle = 1\right] = 1 \ .$$

Definition 3.2 (Soundness)**.** $\mathsf{IOP} = (\mathbf{P}, \mathbf{V})$ *for a relation R has* **soundness error** ϵ_{IOP} *if for every auxiliary state $\boldsymbol{\xi}$, and quantum circuit $\widetilde{\mathbf{P}}$,*

$$\Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & (\mathbb{x}, \mathbf{aux}) \leftarrow \widetilde{\mathbf{P}}(\boldsymbol{\xi}) \\ \wedge\, \mathbb{x} \notin L(R) & b \leftarrow \left\langle \widetilde{\mathbf{P}}(\mathbf{aux}), \mathbf{V}(\mathbb{x})\right\rangle \\ \wedge\, b = 1 & \end{array}\right] \leq \epsilon_{\mathsf{IOP}}(n) \ .$$

Definition 3.3 (Knowledge soundness)**.** $\mathsf{IOP} = (\mathbf{P}, \mathbf{V})$ *for a relation R has* **(adaptive) knowledge soundness error** κ_{IOP} **with extraction circuit size bound** $t_{\mathbf{E}}$ *if there exists a quantum circuit $\mathbf{E}$ such that for every instance size bound $n \in \mathbb{N}$, auxiliary input state $\boldsymbol{\xi}$, and quantum circuit $\widetilde{\mathbf{P}}$,*

$$\begin{aligned} &\Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & (\mathbb{x}, \mathbf{aux}) \leftarrow \widetilde{\mathbf{P}}(\boldsymbol{\xi}) \\ \wedge\, (\mathbb{x}, \mathbb{w}) \in R & \mathbb{w} \leftarrow \mathbf{E}^{\widetilde{\mathbf{P}}}(\mathbb{x}, \mathbf{aux}) \end{array}\right] \\ &\geq \Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & (\mathbb{x}, \mathbf{aux}) \leftarrow \widetilde{\mathbf{P}}(\boldsymbol{\xi}) \\ \wedge\, b = 1 & b \leftarrow \left\langle \widetilde{\mathbf{P}}(\mathbf{aux}), \mathbf{V}(\mathbb{x})\right\rangle \end{array}\right] - \kappa_{\mathsf{IOP}}(n) \ ; \end{aligned}$$

moreover, $\mathbf{E}$ has circuit size $t_{\mathbf{E}}(n)$.

We consider several efficiency measures for an IOP.

- The *round complexity* k is the number of interaction rounds (back-and-forth interactions) between the IOP prover and IOP verifier.
- The *proof alphabet* Σ is the alphabet over which each IOP string is written.
- The *proof length* L is the total number of alphabet symbols across all IOP strings sent by the IOP prover; moreover, L_i is the length of the proof sent by $\mathbf{P}$ in round i and $\mathsf{L}_{\mathsf{max}} \coloneqq \max_{i\in[\mathsf{k}]}\{\mathsf{L}_i\}$;

- The *query complexity* $\mathsf{q} \leq \mathsf{L}$ is the total number of queries that the IOP verifier makes to any IOP string (each query specifies a round and a location in the IOP string of that round, and is answered by the corresponding symbol in the IOP string); moreover, q_i is the number of queries made by $\mathbf{V}$ in round i and $\mathsf{q}_{\mathsf{max}} := \max_{i \in [\mathsf{k}]} \{\mathsf{q}_i\}$ (we assume that $\mathbf{V}$ always queries exactly q_i coordinates in round i, which can be ensured by adding dummy queries);
- The *randomness complexity* r is the number of random bits used by the IOP verifier; moreover, r_i is the number of random bits used by $\mathbf{V}$ in round i.

Any efficiency measure may be a function of the instance $\mathbb{x}$ (e.g., as is common, of the instance size $|\mathbb{x}|$).

Partial proof strings. We allow IOP provers to output partial proof strings $\pi_i \colon [\mathsf{L}_i] \to \Sigma$ which may be undefined at some $q \in [\mathsf{L}_i]$ (where $\pi_i(q) = \perp$). If the IOP verifier $\mathbf{V}$ queries any such coordinate, it outputs 0 (see Section 3).

Public-coin IOPs. We focus on IOPs that are *public-coin*, which means that the IOP verifier is a public-coin interactive algorithm: the IOP verifier sends a uniformly random message, independent of any other messages.

Definition 3.4. *$\mathsf{IOP} = (\mathbf{P}, \mathbf{V})$ for a relation R is* **public-coin** *if, for every $i \in [\mathsf{k}]$, the i-th message of the IOP verifier $\mathbf{V}$ is a fresh uniform random string r_i of some length r_i (which may depend on the instance).*

Queries in a public-coin IOP can be postponed until after the interaction, so the two-step experiment $(\mathbb{x}, \mathbf{aux}) \leftarrow \widetilde{\mathbf{P}}(\boldsymbol{\xi})$ followed by $b \leftarrow \langle \widetilde{\mathbf{P}}(\mathbf{aux}), \mathbf{V}(\mathbb{x}) \rangle$ can be written as

$$\left[\begin{array}{l} (\mathbb{x}, \mathbf{aux}_0) \leftarrow \widetilde{\mathbf{P}}(\boldsymbol{\xi}) \\ (\widetilde{\pi}_1, \mathbf{aux}_1) \leftarrow \widetilde{\mathbf{P}}(\mathbf{aux}_0) \\ r_1 \leftarrow \{0,1\}^{\mathsf{r}_1} \\ \text{For } i \in [\mathsf{k}-1] \setminus \{1\} : \\ \quad (\widetilde{\pi}_i, \mathbf{aux}_i) \leftarrow \widetilde{\mathbf{P}}(r_{i-1}, \mathbf{aux}_{i-1}) \\ \quad r_i \leftarrow \{0,1\}^{\mathsf{r}_i} \\ \widetilde{\pi}_{\mathsf{k}} \leftarrow \widetilde{\mathbf{P}}(r_{\mathsf{k}-1}, \mathbf{aux}_{\mathsf{k}-1}) \\ r_{\mathsf{k}} \leftarrow \{0,1\}^{\mathsf{r}_{\mathsf{k}}} \\ b \leftarrow \mathbf{V}^{(\pi_i)_{i \in [\mathsf{k}]}}(\mathbb{x}, (r_i)_{i \in [\mathsf{k}]}) \end{array}\right] = \left[\begin{array}{l} (\mathbb{x}, \mathbf{aux}_0) \leftarrow \widetilde{\mathbf{P}}(\boldsymbol{\xi}) \\ r_0 := () \\ \text{For } i \in [\mathsf{k}] : \\ \quad (\widetilde{\pi}_i, \mathbf{aux}_i) \leftarrow \widetilde{\mathbf{P}}(r_{i-1}, \mathbf{aux}_{i-1}) \\ \quad r_i \leftarrow \{0,1\}^{\mathsf{r}_i} \\ b \leftarrow \mathbf{V}^{(\pi_i)_{i \in [\mathsf{k}]}}(\mathbb{x}, (r_i)_{i \in [\mathsf{k}]}) \end{array}\right] .$$

Note that objects appearing only on the right-hand side for the sake of compactness (i.e., r_0 and $\mathbf{aux}_{\mathsf{k}}$) are set to $()$, the empty string.

Semi-adaptive public-coin IOPs. A public-coin IOP is *semi-adaptive* if, for every $i \in [\mathsf{k}]$, queries made by the IOP verifier to the i-th proof string π_i cannot depend on query answers for later proof strings $(\pi_j)_{j>i}$.

Definition 3.5. *$\mathsf{IOP} = (\mathbf{P}, \mathbf{V})$ is* **semi-adaptive** *if there exists an algorithm* Query *that, for every round $i \in [\mathsf{k}]$ and index $\ell \in [\mathsf{q}_i]$, determines $\mathbf{V}$'s ℓ-th query position to the i-th IOP string, with the following syntax.*

$\mathsf{Query}^{\pi_i}(\mathbb{x}, (r_j)_{j \in [\mathsf{k}]}, (\mathsf{ans}_j)_{j<i}, \ell) \to q_{i,\ell}$*: On input instance $\mathbb{x}$, verifier randomness $(r_j)_{j \in [\mathsf{k}]}$, past-round answers $(\mathsf{ans}_j)_{j<i}$, and query index $\ell \in [\mathsf{q}_i]$, and given oracle access to an IOP string π_i,* Query *outputs $q_{i,\ell} \in [\mathsf{L}_i]$, the ℓ-th query to the i-th IOP string.*

$\mathsf{Query}^{\pi_i}(\mathbb{x}, (r_j)_{j \in [\mathsf{k}]}, (\mathsf{ans}_j)_{j<i}, \ell)$ *outputs* `Abort` *if any previous query by $\mathbf{V}$ returns $\perp$ (i.e., $\mathsf{ans}_j(q_{j,m}) = \perp$ for some $j < i$ and $m \in [\mathsf{q}_j]$, or $\pi_i(q_{i,m}) = \perp$ for some $m < \ell$).*

Additionally, we define $\mathsf{QuerySet}^{\pi_i}\big(\mathbb{x}, (r_j)_{j \in [\mathsf{k}]}, (\mathsf{ans}_j)_{j<i}\big)$ *as the set of queries to the i-th IOP string:*

$$\mathsf{QuerySet}^{\pi_i}\big(\mathbb{x}, (r_j)_{j \in [\mathsf{k}]}, (\mathsf{ans}_j)_{j<i}\big) := \big\{\mathsf{Query}^{\pi_i}(\mathbb{x}, (r_j)_{j \in [\mathsf{k}]}, (\mathsf{ans}_j)_{j<i}, \ell)\big\}_{\ell \in [\mathsf{q}_i]} .$$

Remark 3.6. One may consider the following, seemingly natural, alternative syntax for Query (and for the natural extension to QuerySet):

$$\mathsf{Query}^{(\pi_j)_{j\le i}}(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, \ell) \to q_{i,\ell} \ .$$

However, such a definition does not prevent fully adaptive behavior: Query may make a query q to π_j (for some $j < i$) such that $q \notin \mathsf{QuerySet}^{(\pi_\ell)_{\ell\le j}}(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]})$ while determining queries to π_i.

3.3 Vector commitment schemes

A (static) *vector commitment scheme* [CF13] over alphabet Σ is a tuple of algorithms

$$\mathsf{VC} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Check})$$

with the following syntax.

- $\mathsf{VC.Gen}(1^\lambda, L) \to \mathsf{pp}$: On input a security parameter $\lambda \in \mathbb{N}$ and message size bound $L \in \mathbb{N}$, VC.Gen samples public parameter pp.
- $\mathsf{VC.Commit}(\mathsf{pp}, m) \to (\mathsf{cm}, \mathsf{aux})$: On input a public parameter pp and a message $m \in \Sigma^L$, VC.Commit produces a commitment cm and the corresponding auxiliary state aux.
- $\mathsf{VC.Open}(\mathsf{pp}, Q, \mathsf{aux}) \to \mathsf{pf}$: On input a public parameter pp, a query set $Q \subseteq [L]$ and an auxiliary state aux, VC.Open outputs an opening proof string pf attesting that $m|_Q$ is the restriction of m to Q.
- $\mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}, \mathsf{ans}, \mathsf{pf}) \to \{0, 1\}$: On input a public parameter pp, a commitment cm, a partial function $\mathsf{ans}\colon Q \to \Sigma$ and an opening proof string pf, VC.Check determines if pf is a valid proof for ans being a restriction to Q of the message whose commitment is cm.

Definition 3.7 (Completeness)**.** $\mathsf{VC} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Check})$ *has* **perfect completeness** *if for every security parameter $\lambda \in \mathbb{N}$, message $m \in \Sigma^L$, and query set $Q \subseteq [L]$,*

$$\Pr\left[\ \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}, m|_Q, \mathsf{pf}) = 1 \ \middle|\ \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{VC.Gen}(1^\lambda, L) \\ (\mathsf{cm}, \mathsf{aux}) \leftarrow \mathsf{VC.Commit}(\mathsf{pp}, m) \\ \mathsf{pf} \leftarrow \mathsf{VC.Open}(\mathsf{pp}, Q, \mathsf{aux}) \end{array}\right] = 1 \ .$$

Definition 3.8 (Post-quantum position binding)**.** $\mathsf{VC} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Check})$ *has* **post-quantum position binding error** $\epsilon_{\mathsf{VCBinding}}$ *if for every security parameter $\lambda \in \mathbb{N}$, message length $L \in \mathbb{N}$, query set size $s \in [L]$, auxiliary input state $\boldsymbol{\xi}$, adversary size bound $t_{\mathsf{VCBinding}} \in \mathbb{N}$, and $t_{\mathsf{VCBinding}}$-size quantum circuit $A_{\mathsf{VCBinding}}$,*

$$\Pr\left[\begin{array}{l} |\mathsf{supp}(\mathsf{ans})| = |\mathsf{supp}(\mathsf{ans}')| = s \\ \wedge\ \exists\, i \in \mathsf{supp}(\mathsf{ans}) \cap \mathsf{supp}(\mathsf{ans}') : \mathsf{ans}(i) \neq \mathsf{ans}'(i) \\ \wedge\ \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}, \mathsf{ans}, \mathsf{pf}) = 1 \\ \wedge\ \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}, \mathsf{ans}', \mathsf{pf}') = 1 \end{array} \middle|\ \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{VC.Gen}(1^\lambda, L) \\ \begin{pmatrix} \mathsf{cm}, \\ \mathsf{ans}, \mathsf{ans}', \\ \mathsf{pf}, \mathsf{pf}' \end{pmatrix} \leftarrow A_{\mathsf{VCBinding}}(\mathsf{pp}, s, \boldsymbol{\xi}) \end{array}\right]$$
$$\le \epsilon_{\mathsf{VCBinding}}(\lambda, L, s, t_{\mathsf{VCBinding}}) \ .$$

Moreover, we say "$A_{\mathsf{VCBinding}}$ wins" as shorthand to refer to the above event.

3.4 Interactive arguments

A (post-quantum) interactive argument for a relation R is a tuple of algorithms $\mathsf{ARG} = (\mathbb{G}, \mathbb{P}, \mathbb{V})$ that satisfies the following properties.

Definition 3.9 (Perfect completeness)**.** $\mathsf{ARG} = (\mathbb{G}, \mathbb{P}, \mathbb{V})$ *for a relation R has* **perfect completeness** *if for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, public parameter $\mathsf{pp} \in \mathbb{G}(1^\lambda, n)$, and instance-witness pair $(\mathbb{x}, \mathbb{w}) \in R$ with $|\mathbb{x}| \leq n$,*

$$\Pr\left[\langle \mathbb{P}(\mathsf{pp}, \mathbb{x}, \mathbb{w}), \mathbb{V}(\mathsf{pp}, \mathbb{x})\rangle = 1\right] = 1 \ .$$

Definition 3.10 (Adaptive soundness)**.** $\mathsf{ARG} = (\mathbb{G}, \mathbb{P}, \mathbb{V})$ *for a relation R has* **(adaptive) soundness error** ϵ_{ARG} *if for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, circuit size bound $t_{\mathsf{ARG}} \in \mathbb{N}$, auxiliary input state $\boldsymbol{\xi}$, and t_{ARG}-size quantum circuit $\widetilde{\mathbb{P}}$,*

$$\Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ \wedge\, \mathbb{x} \notin L(R) & (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ \wedge\, b = 1 & b \leftarrow \left\langle \widetilde{\mathbb{P}}(\mathsf{aux}), \mathbb{V}(\mathsf{pp}, \mathbb{x})\right\rangle \end{array}\right] \leq \epsilon_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) \ .$$

Definition 3.11 (Knowledge soundness)**.** *An interactive argument $\mathsf{ARG} = (\mathbb{G}, \mathbb{P}, \mathbb{V})$ for a relation R has* **(adaptive) knowledge soundness error** κ_{ARG} **with extraction circuit size bound** $t_{\mathcal{E}}$ *if there exists a quantum circuit $\mathcal{E}$ such that for every security parameter $\lambda \in \mathbb{N}$, instance size bound $n \in \mathbb{N}$, auxiliary input state $\boldsymbol{\xi}$, circuit size bound $t_{\mathsf{ARG}} \in \mathbb{N}$, and t_{ARG}-size quantum circuit $\widetilde{\mathbb{P}}$,*

$$\begin{aligned} &\Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ \wedge\, (\mathbb{x}, \mathbb{w}) \in R & (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ & \mathbb{w} \leftarrow \mathcal{E}^{\widetilde{\mathbb{P}}}(\mathbb{x}, \mathsf{aux}) \end{array}\right] \\ &\geq \Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ \wedge\, b = 1 & (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ & b \leftarrow \left\langle \widetilde{\mathbb{P}}(\mathsf{aux}), \mathbb{V}(\mathsf{pp}, \mathbb{x})\right\rangle \end{array}\right] - \kappa_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) \ ; \end{aligned}$$

moreover, the circuit size of $\mathcal{E}$ is $t_{\mathcal{E}}(\lambda, n, t_{\mathsf{ARG}})$.

Public-coin interactive arguments. We focus on interactive arguments that are public-coin.

Definition 3.12. *A k-round $\mathsf{ARG} = (\mathbb{G}, \mathbb{P}, \mathbb{V})$ for a relation R is* **public-coin** *if, for every round $i \in [k]$, the i-th message of the argument verifier is an independent uniform random string r_i of a prescribed length r_i (which may depend on the instance). In particular, at the end of any round of the protocol, the argument prover knows all the randomness sampled by the argument verifier until then.*

Let pm_i denote the prover's message in the i-th round. In the public-coin case, $b \leftarrow \langle \widetilde{\mathbb{P}}(\mathsf{aux}_0), \mathbb{V}(\mathsf{pp}, \mathbb{x})\rangle$ can be written as

$$\left[\begin{array}{l} r_0 \coloneqq () \\ \text{For } i \in [k]: \\ \quad (\mathsf{pm}_i, \mathsf{aux}_i) \leftarrow \widetilde{\mathbb{P}}(r_{i-1}, \mathsf{aux}_{i-1}) \\ \quad r_i \leftarrow \{0,1\}^{\mathsf{r}_i} \\ b \leftarrow \mathbb{V}(\mathsf{pp}, \mathbb{x}, (r_i)_{i\in[k]}, (\mathsf{pm}_i)_{i\in[k]}) \end{array}\right] \ .$$

Remark 3.13. We highlight that, as our results center around the construction of IOP provers $\widetilde{\mathbf{P}}$ with black-box access to argument provers $\widetilde{\mathbb{P}}$, we do not assume any structure on the latter's auxiliary information aux_i; therefore, we usually denote it as an arbitrary quantum state $\boldsymbol{\rho}_i$. In particular, Definitions 3.9 to 3.12 can be stated equivalently with $\boldsymbol{\rho}$ in place of aux. (As can Definitions 3.1 to 3.4 with $\boldsymbol{\rho}$ in place of $\mathbf{aux}$; but we maintain the auxiliary information notation for IOP provers, whose structure is $\mathbf{aux} = (\mathbf{aux}', \mathsf{aux})$ with $\mathbf{aux}'$ classical and $\mathsf{aux} = \boldsymbol{\rho}$ used for executions of $\widetilde{\mathbb{P}}$.)

3.5 The IBCS protocol

We describe the interactive argument $(\mathbb{P}, \mathbb{V}) \coloneqq \mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ [CDGS23].

Construction 3.14. The argument generator $\mathbb{G}$ receives as input a security parameter $\lambda \in \mathbb{N}$ and an instance size bound $n \in \mathbb{N}$, and works as follows.

$\mathbb{G}(\lambda, n)$:
1. Sample public parameters for the VC scheme: $\mathsf{pp}_{\mathsf{VC}} \leftarrow \mathsf{VC.Gen}\big(1^\lambda, \mathsf{L}_{\mathsf{max}}(n)\big)$.
2. Output $\mathsf{pp} \coloneqq \mathsf{pp}_{\mathsf{VC}}$.

The argument prover $\mathbb{P}$ receives as input the public parameter pp, an instance $\mathbb{x}$, and a witness $\mathbb{w}$, and the argument verifier $\mathbb{V}$ receives as input the public parameter pp and the instance $\mathbb{x}$. Then $\mathbb{P}$ and $\mathbb{V}$ interact as follows.

1. **$\mathbb{P}$'s commitments.**
   For $i \in [\mathsf{k}]$:
   (a) $\mathbb{P}$'s i-th commitment.
      i. Compute the i-th IOP string $\pi_i \in \Sigma^{\mathsf{L}_i}$ and (classical) auxiliary state:
$$(\pi_i, \mathbf{aux}_i) \leftarrow \begin{cases} \mathbf{P}(\mathbb{x}, \mathbb{w}) & \text{if } i = 1 \\ \mathbf{P}(r_{i-1}, \mathbf{aux}_{i-1}) & \text{if } i > 1 \end{cases}.$$
      ii. Compute a VC commitment to the IOP string: $(\mathsf{cm}_i, \mathsf{aux}_i) \leftarrow \mathsf{VC.Commit}(\mathsf{pp}, \pi_i)$.
      iii. Send cm_i to $\mathbb{V}$.
   (b) $\mathbb{V}$'s i-th challenge.
      i. Sample the i-th IOP verifier randomness $r_i \leftarrow \{0,1\}^{\mathsf{r}_i}$.
      ii. Send r_i to $\mathbb{P}$.
2. **$\mathbb{P}$'s response.**
   (a) Run the IOP verifier $\mathbf{V}^{(\pi_i)_{i\in[\mathsf{k}]}}(\mathbb{x}, (r_i)_{i\in[\mathsf{k}]})$ to deduce its query sets $(Q_i)_{i\in[\mathsf{k}]}$, where $Q_i \subseteq [\mathsf{L}_i]$ is the set of entries of π_i queried by $\mathbf{V}$.
   (b) For every $i \in [\mathsf{k}]$, set $\mathsf{ans} \coloneqq \pi_i|_{Q_i}$ and compute an opening proof $\mathsf{pf}_i \leftarrow \mathsf{VC.Open}(\mathsf{pp}, Q_i, \mathsf{aux}_i)$.
   (c) Send $\big((\mathsf{ans}_i, \mathsf{pf}_i)\big)_{i\in[\mathsf{k}]}$ to $\mathbb{V}$.
3. **$\mathbb{V}$'s decision.**
   (a) Check that $\mathbf{V}^{(\mathsf{ans}_i)_{i\in[\mathsf{k}]}}\big(\mathbb{x}, (r_i)_{i\in[\mathsf{k}]}\big) = 1$.
   (b) Check that $\mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}_i, \mathsf{ans}_i, \mathsf{pf}_i) = 1$ for every $i \in [\mathsf{k}]$.
   (c) Output 1 if all checks pass; output 0 otherwise.

The protocol has $\mathsf{k}+1$ rounds and $2\mathsf{k}+1$ messages: the first k simulate the IOP, and in the last $\mathbb{P}$ sends the query set assignments along with their opening proofs. Moreover, the protocol is public coin because the verifier's messages consist of random strings. We comment on the protocol's efficiency measures:

- the generator communicates $|\mathsf{pp}_{\mathsf{VC}}|$ bits to the prover and verifier;
- the prover-to-verifier communication consists of $\sum_{i\in[\mathsf{k}]}(|\mathsf{cm}_i|+\mathsf{q}_i\cdot(\log \mathsf{L}_i+\log|\Sigma|)+|\mathsf{pf}_i|)$ bits (encoding partial functions as coordinate-symbol pairs for coordinates in its support);
- the verifier-to-prover communication consists of $\mathsf{r}=\sum_{i\in[\mathsf{k}]}\mathsf{r}_i$ bits;
- the time complexity of the argument generator is $t_{\mathsf{VC.Gen}}$;
- the time complexity of the argument prover is $t_{\mathbf{P}}+\mathsf{k}\cdot(t_{\mathsf{VC.Commit}}+t_{\mathsf{VC.Open}})+t_{\mathbf{V}}$;
- the time complexity of the argument verifier is $t_{\mathbf{V}}+\mathsf{k}\cdot t_{\mathsf{VC.Check}}$.

3.6 Quantum rewinding

We recall useful definitions and claims about quantum rewinding from [CMSZ21].

Definition 3.15. *A* **quantum game** $\mathsf{G}=(C,Z,f)$ *consists of a challenge set C, answer set Z, and winning predicate $f\colon C\times Z\to\{0,1\}$. Let $\mathcal{Z}$ denote an answer register (i.e., a quantum register with basis states indexed by elements of Z) and $\mathcal{I}$ an arbitrary additional register.*

A quantum strategy for G *is a pair* $(\mathsf{Strat},\boldsymbol{\rho})$ *where* $\mathsf{Strat}=\{\mathsf{Strat}_r\}_{r\in C}$ *is a collection of unitaries acting on $\mathcal{Z}\otimes\mathcal{I}$ and $\boldsymbol{\rho}$ is a quantum state on the same registers. A play of the quantum game* G *with quantum strategy* $(\mathsf{Strat},\boldsymbol{\rho})$ *consists of the following experiment.*

1. *Sample a challenge $r\leftarrow C$.*
2. *Apply the unitary Strat_r to $\boldsymbol{\rho}$, obtaining $\boldsymbol{\rho}'\leftarrow\mathsf{Strat}_r(\boldsymbol{\rho})$ (in the same registers $\mathcal{Z}\otimes\mathcal{I}$).*
3. *Measure the predicate f coherently on r and $\mathcal{Z}$, obtaining outcome $b\leftarrow f(r,\mathcal{Z})$.*
4. *Output b.*

The **value of** $(\mathsf{Strat},\boldsymbol{\rho})$ *is the probability that playing* Strat *starting from $\boldsymbol{\rho}\in\boldsymbol{S}(\mathcal{Z}\otimes\mathcal{I})$ leads the above experiment to output $b=1$:*

$$\omega_{\mathsf{G}}(\mathsf{Strat},\boldsymbol{\rho}):=\Pr\left[b=1\;\middle|\;\begin{array}{l}r\leftarrow C\\ \boldsymbol{S}(\mathcal{Z}\otimes\mathcal{I})\ni\boldsymbol{\rho}'\leftarrow\mathsf{Strat}_r(\boldsymbol{\rho})\\ b\leftarrow f(r,\mathcal{Z})\end{array}\right].$$

Construction 3.16. Let $\mathsf{G}=(C,Z,f)$ be a quantum game with answer register $\mathcal{Z}$ and internal register $\mathcal{I}$. We describe a quantum algorithm $\mathsf{AlgRewind}_{\mathsf{G}}$ that plays G multiple times based on two quantum procedures, $\mathsf{ValEst}_{\mathsf{G}}$ and $\mathsf{Repair}_{\mathsf{G}}$, constructed in [CMSZ21].

- $\mathsf{ValEst}_{\mathsf{G}}\big(\mathsf{Strat},(\mathcal{Z},\mathcal{I}),\gamma,\delta\big)\to\big(p,(\mathcal{Z},\mathcal{I})\big)$: Let $\boldsymbol{\rho}$ denote the initial state on register $\mathcal{Z}\otimes\mathcal{I}$. On input a quantum strategy $(\mathsf{Strat},\boldsymbol{\rho})$ and error parameters γ,δ, the algorithm $\mathsf{ValEst}_{\mathsf{G}}$ outputs a probability p and a state $\boldsymbol{\rho}'$ on $\mathcal{Z}\otimes\mathcal{I}$.

  The outputs satisfy $\mathbb{E}[p]=\omega_{\mathsf{G}}(\mathsf{Strat},\boldsymbol{\rho})$ and $\omega_{\mathsf{G}}(\mathsf{Strat},\boldsymbol{\rho}')\approx p$.

- $\mathsf{Repair}_{\mathsf{G}}\big(\mathsf{Strat},(\mathcal{Z},\mathcal{I}),\gamma,\delta,p,b,r\big)\to(\mathcal{Z},\mathcal{I})$: Let $\boldsymbol{\rho}$ denote the initial state on register $\mathcal{Z}\otimes\mathcal{I}$. On input a player strategy $(\mathsf{Strat},\boldsymbol{\rho})$, error parameters γ and δ, probability $p\in[0,1]$, bit $b\in\{0,1\}$, and challenge $r\in C$, $\mathsf{Repair}_{\mathsf{G}}$ outputs a state $\boldsymbol{\rho}'$ on $\mathcal{Z}\otimes\mathcal{I}$.

  If $\boldsymbol{\rho}$ is the state resulting from running $\mathsf{ValEst}_{\mathsf{G}}$ followed by one play of G, and p, b, r are the probability output by $\mathsf{ValEst}_{\mathsf{G}}$, the bit output by G and the challenge r sampled in the game, then state $\boldsymbol{\rho}'$ satisfies $\omega_{\mathsf{G}}(\mathsf{Strat},\boldsymbol{\rho}')\approx p$.

The algorithm $\mathsf{AlgRewind}_{\mathsf{G}} := (\mathsf{RewindInit}_{\mathsf{G}}, \mathsf{RewindStrat}_{\mathsf{G}}, \mathsf{RewindRepair}_{\mathsf{G}})$ is a tuple of three algorithms. On input a quantum algorithm Strat, registers $(\mathcal{Z},\mathcal{I})$, and parameters $\mathsf{N} \in \mathbb{N}, \epsilon \in [0,1]$, $\mathsf{RewindInit}_{\mathsf{G}}$ initializes parameters, estimates an initial probability of the input strategy and outputs auxiliary information $\mathbf{aux}$. On input a challenge r and $\mathbf{aux}$, $\mathsf{RewindStrat}_{\mathsf{G}}$ execute the player's strategy, and outputs register $\mathcal{Z}$ along with the updated $\mathbf{aux}$. On input register $\mathcal{Z}$, bit b and auxiliary information $\mathbf{aux}$, $\mathsf{RewindRepair}_{\mathsf{G}}$ repairs the quantum state on registers $\mathcal{Z} \otimes \mathcal{I}$ and outputs updated $\mathbf{aux}$. They operate as follows.

- $\mathsf{RewindInit}_{\mathsf{G}}\big(\mathsf{Strat}, (\mathcal{Z},\mathcal{I}), \mathsf{N}, \epsilon\big)$:
  1. Set error parameters $\gamma := \frac{\epsilon}{2\mathsf{N}+2} \in [0,1]$ and $\delta := \frac{\epsilon^2}{256\mathsf{N}^2} \in [0,1]$.
  2. Initialize an internal counter $i := 0$.
  3. Run $\big(p_0, (\mathcal{Z},\mathcal{I})\big) \leftarrow \mathsf{ValEst}_{\mathsf{G}}\big(\mathsf{Strat}, (\mathcal{Z},\mathcal{I}), \gamma, \delta\big)$.
  4. Output $\mathbf{aux} := \big(\mathsf{Strat}, \gamma, \delta, i, p_i, (\mathcal{Z},\mathcal{I})\big)$.
- $\mathsf{RewindStrat}_{\mathsf{G}}(r, \mathbf{aux})$:
  1. Parse $\mathbf{aux}$ as $\big(\mathsf{Strat}, \gamma, \delta, i, p_i, (\mathcal{Z},\mathcal{I})\big)$.
  2. Run $(\mathcal{Z},\mathcal{I}) \leftarrow \mathsf{Strat}_r(\mathcal{Z},\mathcal{I})$.
  3. Set $r_i := r$.
  4. Set $\mathbf{aux} := (\mathsf{Strat}, \gamma, \delta, i, p_i, r_i, \mathcal{I})$.
  5. Output $(\mathcal{Z}, \mathbf{aux})$.
- $\mathsf{RewindRepair}_{\mathsf{G}}(\mathcal{Z}, b, \mathbf{aux})$:
  1. Parse $\mathbf{aux}$ as $(\mathsf{Strat}, \gamma, \delta, i, p_i, r_i, \mathcal{I})$.
  2. Run $(\mathcal{Z},\mathcal{I}) \leftarrow \mathsf{Strat}^{\dagger}_{r_i}(\mathcal{Z},\mathcal{I})$.
  3. Run $(\mathcal{Z},\mathcal{I}) \leftarrow \mathsf{Repair}_{\mathsf{G}}\big(\mathsf{Strat}, (\mathcal{Z},\mathcal{I}), \gamma, \delta, p_i, b, r_i\big)$.
  4. Increment the internal counter: $i := i + 1$.
  5. Run $(p_i, (\mathcal{Z},\mathcal{I})) \leftarrow \mathsf{ValEst}_{\mathsf{G}}\big(\mathsf{Strat}, (\mathcal{Z},\mathcal{I}), \gamma, \delta\big)$.
  6. Output $\mathbf{aux} := \big(\mathsf{Strat}, \gamma, \delta, i, p_i, (\mathcal{Z},\mathcal{I})\big)$.

Construction 3.17. Let $\mathsf{G} = (C, Z, f)$ be a quantum game and $\mathsf{AlgRewind}_{\mathsf{G}}$ be the algorithm in Construction 3.16. For every player strategy $(\mathsf{Strat}, \boldsymbol{\rho})$, parameters N and ϵ, and number of iterations i, we define the following rewinding procedure.

$\mathsf{Rewind}_{\mathsf{G}}\big(\mathsf{Strat}, \mathsf{N}, \epsilon, i, (\mathcal{Z},\mathcal{I})\big)$:
1. Run $\mathbf{aux}_0 \leftarrow \mathsf{RewindInit}_{\mathsf{G}}\big(\mathsf{Strat}, \mathsf{N}, \epsilon, (\mathcal{Z},\mathcal{I})\big)$.
2. For $j \in [i]$:
   (a) Sample $r \leftarrow C$.
   (b) Run $(\mathcal{Z}, \mathbf{aux}'_j) \leftarrow \mathsf{RewindStrat}_{\mathsf{G}}(r, \mathbf{aux}_{j-1})$.
   (c) Measure $b \leftarrow f(r, \mathcal{Z})$.
   (d) Run $\mathbf{aux}_j \leftarrow \mathsf{RewindRepair}_{\mathsf{G}}(\mathcal{Z}, b, \mathbf{aux}'_j)$.

The circuit size of $\mathsf{Rewind}_{\mathsf{G}}\big(\mathsf{Strat}, \mathsf{N}, \epsilon, i, (\mathcal{Z},\mathcal{I})\big)$ is

$$t_{\mathsf{Rewind}}(\mathsf{N}, \epsilon, t_f, t_{\mathsf{Strat}}) = \mathsf{poly}(\mathsf{N}/\epsilon) \cdot (t_f + t_{\mathsf{Strat}}) \ , \tag{1}$$

where t_f and t_{Strat} are the gate complexities of the game predicate f and unitary Strat, respectively.

Later in this paper, we also use $\mathsf{Rewind}_{\mathsf{G}}\big(\mathsf{Strat}, \boldsymbol{\rho}, \mathsf{N}, \epsilon, i\big)$ to denote the same rewinding procedure and run $\mathsf{AlgRewind}_{\mathsf{G}}\big(\mathsf{Strat}, \boldsymbol{\rho}, \mathsf{N}, \epsilon\big)$ in Step 1 of Construction 3.17. Recall from Section 3.1 that it is equivalent to have registers or quantum state on the registers as inputs to quantum algorithms.

Claim 3.18 ([CMSZ21, Claim 4.15])**.** *Let* $\mathsf{G} = (C, Z, f)$ *be a quantum game and* $(\mathsf{Strat}, \boldsymbol{\rho}^{(0)})$ *be a quantum strategy for* G*. Fix* $\epsilon \in [0,1]$ *and* $\mathsf{N} \in \mathbb{N}$*. For every* $i \in [\mathsf{N}]$*, the quantum sate* $\boldsymbol{\rho}^{(i)}$ *on* $(\mathcal{Z}, \mathcal{I})$ *after running* $\mathsf{Rewind}_{\mathsf{G}}\big(\mathsf{Strat}, \mathsf{N}, \epsilon, i, (\mathcal{Z}, \mathcal{I})\big)$ *(Construction 3.17) satisfies*

$$\omega_{\mathsf{G}}(\mathsf{Strat}, \boldsymbol{\rho}^{(i)}) \geq \omega_{\mathsf{G}}(\mathsf{Strat}, \boldsymbol{\rho}^{(0)}) - \epsilon \ .$$

4 Security properties for vector commitment schemes

We assume that VC schemes satisfy *collapse position binding*, a new property that we introduce in Definition 4.1. A similar property was introduced in [CMSZ21], and in Remark 4.3 we explain the differences between the two definitions. We show that our definition of collapse position binding implies *unique-message position binding* (Definition 4.4), which implies post-quantum position binding (Definition 3.8). collapse position binding and unique-message position binding are the two main properties that we use in the security analysis of the IBCS protocol in Section 5.

Definition 4.1 (collapse position binding)**.** $\mathsf{VC} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Check})$ *has* **collapse position binding error** $\epsilon_{\mathsf{VCCollapse}}$ *if, for every security parameter* $\lambda \in \mathbb{N}$*, message length* $L \in \mathbb{N}$*, query set size* $s \in [L]$*, auxiliary input state* $\boldsymbol{\xi}$*, adversary size bound* $t_{\mathsf{VCCollapse}} \in \mathbb{N}$*, and* $t_{\mathsf{VCCollapse}}$*-size quantum circuit* $A_{\mathsf{VCCollapse}}$*,*

$$\left| \begin{array}{l} \Pr\left[\mathsf{VCCollapseExp}(0, \lambda, L, s, A_{\mathsf{VCCollapse}}, \boldsymbol{\xi}) = 1\right] \\ -\Pr\left[\mathsf{VCCollapseExp}(1, \lambda, L, s, A_{\mathsf{VCCollapse}}, \boldsymbol{\xi}) = 1\right] \end{array} \right| \leq \epsilon_{\mathsf{VCCollapse}}(\lambda, L, s, t_{\mathsf{VCCollapse}}) \ .$$

For $b \in \{0,1\}$*, the collapse position binding experiment is defined as follows.*

$\mathsf{VCCollapseExp}(b, \lambda, L, s, A_{\mathsf{VCCollapse}}, \boldsymbol{\xi})$:

1. *Sample public parameters* $\mathsf{pp} \leftarrow \mathsf{VC.Gen}(1^\lambda, L)$ *for the VC scheme.*
2. *Run* $(\mathsf{cm}, \mathsf{idx}, \mathcal{A}, \mathcal{O}) \leftarrow A_{\mathsf{VCCollapse}}(\mathsf{pp}, \boldsymbol{\xi})$ *to obtain a classical commitment* cm *and coordinate* $\mathsf{idx} \in [L]$*, as well as quantum registers* $(\mathcal{A}, \mathcal{O})$ *that store superpositions of partial functions* ans *and opening proofs* pf.
3. *Measure* $(\mathcal{A}(\mathsf{idx}) \neq \bot) \wedge (|\mathsf{supp}(\mathcal{A})| = s) \wedge \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}, \mathcal{A}, \mathcal{O})$ *and output* `Abort` *if the outcome is* 0*. (Recall from Section 3.1 that* $f(\mathcal{X})$ *is the coherent application of the function* f *on register* $\mathcal{X}$ *onto an ancilla followed by measuring the ancilla, uncomputing* f *and outputting the measurement outcome. This step applies* $f(\mathsf{pp}, \mathsf{cm}, s, \mathsf{idx}, \mathsf{ans}, \mathsf{pf}) := (\mathsf{ans}(\mathsf{idx}) \neq \bot) \wedge (|\mathsf{supp}(\mathsf{ans})| = s) \wedge \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}, \mathsf{ans}, \mathsf{pf})$*.)*
4. *If* $b = 1$*, measure* $\mathcal{A}(\mathsf{idx})$ *(and discard the outcome* $\mathsf{ans}(\mathsf{idx})$*).*
5. *Output* $b' \leftarrow A_{\mathsf{VCCollapse}}(\mathcal{A}, \mathcal{O})$.

Note that the condition above is equivalent to

$$\left| \Pr\left[\mathsf{VCCollapseExp}(b, \lambda, L, s, A_{\mathsf{VCCollapse}}, \boldsymbol{\xi}) = b \mid b \leftarrow \{0,1\}\right] - \frac{1}{2} \right| \leq \frac{1}{2} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, L, s, t_{\mathsf{VCCollapse}}).$$

Remark 4.2 (Monotonicity of $\epsilon_{\mathsf{VCCollapse}}$)**.** We assume hereafter that the collapse position binding error $\epsilon_{\mathsf{VCCollapse}}$ is monotone in each coordinate in the natural direction:

- $\epsilon_{\mathsf{VCCollapse}}(\cdot, L, s, t_{\mathsf{VCCollapse}})$ is non-increasing (larger security parameters decrease an adversary's success);
- $\epsilon_{\mathsf{VCCollapse}}(\lambda, \cdot, s, t_{\mathsf{VCCollapse}})$ is non-decreasing (detecting the measurement of some location in a string is easier than detecting in a substring);
- $\epsilon_{\mathsf{VCCollapse}}(\lambda, L, \cdot, t_{\mathsf{VCCollapse}})$ is non-decreasing (detecting a measurement in a set is easier than detecting one in a subset); and
- $\epsilon_{\mathsf{VCCollapse}}(\lambda, L, s, \cdot)$ is non-decreasing (the success of an adversary increases with its computational power).

The last condition is trivially satisfied, while the first should also hold in any reasonable commitment scheme. The remaining two are natural (and satisfied in the case of Merkle trees); in any case, otherwise

one may replace, in our computations, expressions of the type $\epsilon_{\mathsf{VCCollapse}}(\lambda, L_{\mathsf{Max}}, s_{\mathsf{Max}}, t_{\mathsf{VCCollapse}})$, when $L_{\mathsf{Max}} := \max_i \{L_i\}$ and $s_{\mathsf{Max}} := \max_j \{s_j\}$, with

$$\max_{i,j} \{\epsilon_{\mathsf{VCCollapse}}(\lambda, L_i, s_j, t_{\mathsf{VCCollapse}})\} \quad .$$

Remark 4.3. Our definition of collapse position binding for VC differs from the prior definition in [CMSZ21].

First, [CMSZ21] imposes a stronger requirement that measurement of the *opening* register $\mathcal{O}$ be undetectable to an efficient adversary, analogous to the notion of "strong" collapse binding for (standard) commitments introduced by [CCY21]. As discussed in [LMS22b], this makes the property fragile, as adding a single dummy bit to the opening of an otherwise secure scheme is enough to violate strong collapse binding. Our definition captures undetectability of measurements of the message register $\mathcal{M}$ alone, and thus more closely corresponds to the standard notion of collapse binding.

Second, [CMSZ21] restricts attention to adversaries that output a classical query set Q and produce a superposition of openings on Q. Our definition considers a wider class of adversaries: those that output a single index idx and a superposition of openings on potentially *many* sets Q that contain idx. While this modification makes our definition more restrictive, it implies post-quantum position binding, which need not be assumed separately (unlike in [CMSZ21]).

In light of these features, we consider this definition to be the "correct" VC analogue of collapse binding. Moreover, as we show (in Lemma 4.12), this definition arises by considering the standard collapse binding guarantee applied to a certain natural (plain) commitment scheme induced by VC.

Definition 4.4 (Unique-message position binding)**.** $\mathsf{VC} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Check})$ *has* **unique-message position binding error** $\epsilon_{\mathsf{VCUniq}}$ *if, for every security parameter $\lambda \in \mathbb{N}$, message length $L \in \mathbb{N}$, query set size $s \in [L]$, auxiliary input state $\boldsymbol{\xi}$, adversary size bound $t_{\mathsf{VCUniq}} \in \mathbb{N}$, and t_{VCUniq}-size quantum circuit A_{VCUniq},*

$$\Pr\left[\mathsf{VCUniqExp}(\lambda, L, s, A_{\mathsf{VCUniq}}, \boldsymbol{\xi}) = 1\right] \leq \epsilon_{\mathsf{VCUniq}}(\lambda, L, s, t_{\mathsf{VCUniq}}) \quad .$$

The unique-message position binding experiment is defined as follows.

$\mathsf{VCUniqExp}(\lambda, L, s, A_{\mathsf{VCUniq}}, \boldsymbol{\xi})$*:*

1. *Sample public parameters* $\mathsf{pp} \leftarrow \mathsf{VC.Gen}(1^\lambda, L)$ *for the VC scheme.*
2. *Run* $(\mathsf{cm}, \mathsf{idx}, m, \mathcal{A}, \mathcal{O}) \leftarrow A_{\mathsf{VCUniq}}(\mathsf{pp}, \boldsymbol{\xi})$ *to obtain a classical commitment* cm*, index* $\mathsf{idx} \in [L]$*, and message $m \in \Sigma$, as well as quantum registers $(\mathcal{A}, \mathcal{O})$ that store a superposition of functions* ans *and opening proofs* pf*.*
3. *Measure* $(\mathcal{A}(\mathsf{idx}) = \bot) \wedge (|\mathsf{supp}(\mathcal{A})| = s-1) \wedge \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}, \mathcal{A}[\mathsf{idx} \to m], \mathcal{O})$ *and output* `Abort` *if the outcome is* 0*. (This step applies* $f(\mathsf{pp}, \mathsf{cm}, s, \mathsf{idx}, \mathsf{ans}, \mathsf{pf}, m) := (\mathsf{ans}(\mathsf{idx}) = \bot) \wedge (|\mathsf{supp}(\mathsf{ans})| = s-1) \wedge \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}, \mathsf{ans}[\mathsf{idx} \to m], \mathsf{pf})$ *coherently.)*
4. *Run* $(m', \mathcal{A}, \mathcal{O}) \leftarrow A_{\mathsf{VCUniq}}(\mathcal{A}, \mathcal{O})$ *to obtain another classical message $m' \in \Sigma$ as well as superpositions of answers and openings in $(\mathcal{A}, \mathcal{O})$. Output* `Abort` *if $m' = m$.*
5. *Measure* $(\mathcal{A}(\mathsf{idx}) = \bot) \wedge (|\mathsf{supp}(\mathcal{A})| = s-1) \wedge \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}, \mathcal{A}[\mathsf{idx} \to m'], \mathcal{O})$*. Output* 1 *if the measurement outcome is* 1*; output* 0 *otherwise.*

We say "A_{VCUniq} wins*" as shorthand for the event* $\mathsf{VCUniqExp}(\lambda, L, s, A_{\mathsf{VCUniq}}, \boldsymbol{\xi}) = 1$.

Lemma 4.5. *Let* VC *be a vector commitment scheme with collapse position binding error $\epsilon_{\mathsf{VCCollapse}}$, unique-message position binding error $\epsilon_{\mathsf{VCUniq}}$, and post-quantum position-binding error $\epsilon_{\mathsf{VCBinding}}$. Then, for every security parameter $\lambda \in \mathbb{N}$, message length $L \in \mathbb{N}$, query set size $s \in [L]$, and adversary size bound $t_{\mathsf{VCBinding}} \in \mathbb{N}$,*

$$\epsilon_{\mathsf{VCBinding}}(\lambda, L, s, t_{\mathsf{VCBinding}}) \leq \epsilon_{\mathsf{VCUniq}}(\lambda, L, s, t_{\mathsf{VCUniq}}) \leq 4 \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, L, s, t_{\mathsf{VCCollapse}}) \quad ,$$

where $t_{\mathsf{VCCollapse}} = O(t_{\mathsf{VCUniq}})$ *and* $t_{\mathsf{VCUniq}} = O(t_{\mathsf{VCBinding}})$.

In Section 4.1 we present a transformation from a VC to a commitment scheme and connect the properties we introduced for VCs to their analogue for commitment schemes. In Section 4.2 we prove Lemma 4.5.

4.1 From VC schemes to commitment schemes

A *commitment scheme* over alphabet Σ is a tuple of algorithms

$$\mathsf{CM} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Check})$$

with the following syntax.

- $\mathsf{CM.Gen}(1^\lambda) \to \mathsf{pp}$: On input a security parameter $\lambda \in \mathbb{N}$, CM.Gen samples public parameter pp.
- $\mathsf{CM.Commit}(\mathsf{pp}, m) \to (\mathsf{cm}, \omega)$: On input a public parameter pp and a message $m \in \Sigma$, CM.Commit produces a commitment cm and the corresponding opening ω.
- $\mathsf{CM.Check}(\mathsf{pp}, \mathsf{cm}, m, \omega) \to \{0,1\}$: On input a public parameter pp, an opening ω, a commitment cm, and a message m, CM.Check checks if cm is a valid commitment to m.

We state several properties for a commitment scheme CM.

Definition 4.6 (Completeness)**.** $\mathsf{CM} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Check})$ *has* **perfect completeness** *if for every security parameter* $\lambda \in \mathbb{N}$ *and message* $m \in \Sigma$*,*

$$\Pr\left[\ \mathsf{CM.Check}(\mathsf{pp}, \mathsf{cm}, m, \omega) = 1 \ \middle|\ \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{CM.Gen}(1^\lambda) \\ (\mathsf{cm}, \omega) \leftarrow \mathsf{CM.Commit}(\mathsf{pp}, m) \end{array} \right] = 1 \ .$$

Definition 4.7 (post-quantum binding)**.** $\mathsf{CM} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Check})$ *has* **post-quantum binding error** $\epsilon_{\mathsf{CMBinding}}$ *if for every security parameter* $\lambda \in \mathbb{N}$*, auxiliary input state* $\boldsymbol{\xi}$*, adversary size bound* $t_{\mathsf{CMBinding}} \in \mathbb{N}$*, and* $t_{\mathsf{CMBinding}}$*-size quantum circuit* $A_{\mathsf{CMBinding}}$*,*

$$\Pr\left[\begin{array}{l} m \neq m' \\ \wedge\ \mathsf{CM.Check}(\mathsf{pp}, \mathsf{cm}, m, \omega) = 1 \\ \wedge\ \mathsf{CM.Check}(\mathsf{pp}, \mathsf{cm}, m', \omega') = 1 \end{array} \middle|\ \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{CM.Gen}(1^\lambda) \\ (\mathsf{cm}, m, m', \omega, \omega') \leftarrow A_{\mathsf{CMBinding}}(\mathsf{pp}, \boldsymbol{\xi}) \end{array} \right] \leq \epsilon_{\mathsf{CMBinding}}(\lambda, t_{\mathsf{CMBinding}}) \ .$$

Moreover, we say "$A_{\mathsf{CMBinding}}$ *wins" as shorthand to refer to the above event.*

Definition 4.8 (Collapse binding)**.** $\mathsf{CM} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Check})$ *has* **collapse binding error** $\epsilon_{\mathsf{CMCollapse}}$ *if for every security parameter* $\lambda \in \mathbb{N}$*, auxiliary input state* $\boldsymbol{\xi}$*, adversary size bound* $t_{\mathsf{CMCollapse}} \in \mathbb{N}$*, and* $t_{\mathsf{CMCollapse}}$*-size quantum circuit* $A_{\mathsf{CMCollapse}}$*,*

$$\left|\begin{array}{l} \Pr\left[\mathsf{CMCollapseExp}(0, \lambda, A_{\mathsf{CMCollapse}}, \boldsymbol{\xi}) = 1\right] \\ -\Pr\left[\mathsf{CMCollapseExp}(1, \lambda, A_{\mathsf{CMCollapse}}, \boldsymbol{\xi}) = 1\right] \end{array}\right| \leq \epsilon_{\mathsf{CMCollapse}}(\lambda, t_{\mathsf{CMCollapse}}) \ .$$

For $b \in \{0,1\}$*, the collapse binding experiment is defined as follows.*

$\mathsf{CMCollapseExp}(b, \lambda, A_{\mathsf{CMCollapse}}, \boldsymbol{\xi})$*:*

1. *Sample public parameters* $\mathsf{pp} \leftarrow \mathsf{CM.Gen}(1^\lambda)$.
2. *Run* $(\mathsf{cm}, \mathcal{M}, \mathcal{W}) \leftarrow A_{\mathsf{CMCollapse}}(\mathsf{pp}, \boldsymbol{\xi})$ *to obtain a classical commitment* cm *as well as quantum registers* $(\mathcal{M}, \mathcal{W})$ *that store superpositions of messages* m *and openings* ω*, respectively.*

3. *Measure* $\mathsf{CM.Check}(\mathsf{pp}, \mathsf{cm}, \mathcal{M}, \mathcal{W})$ *and output output* $\texttt{Abort}$ *if the outcome is* 0.
4. *If* $b = 1$*, measure* $\mathcal{M}$ *in the standard basis (discarding the outcome* m*).*
5. *Output* $b' \leftarrow A_{\mathsf{CMCollapse}}(\mathcal{M}, \mathcal{W})$.

Definition 4.9 (Unique-message binding)**.** $\mathsf{CM} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Check})$ *has* **unique-message binding error** $\epsilon_{\mathsf{CMUniq}}$ *if for every security parameter* $\lambda \in \mathbb{N}$*, auxiliary input state* $\boldsymbol{\xi}$*, adversary size bound* $t_{\mathsf{CMUniq}} \in \mathbb{N}$*, and* t_{CMUniq}*-size quantum circuit* A_{CMUniq}*,*

$$\Pr\left[\mathsf{CMUniqExp}(\lambda, A_{\mathsf{CMUniq}}, \boldsymbol{\xi}) = 1\right] \leq \epsilon_{\mathsf{CMUniq}}(\lambda, t_{\mathsf{CMUniq}}) \ .$$

The unique-message binding experiment is defined as follows.

$\mathsf{CMUniqExp}(\lambda, A_{\mathsf{CMUniq}}, \boldsymbol{\xi})$*:*

1. *Sample public parameters* $\mathsf{pp} \leftarrow \mathsf{CM.Gen}(1^\lambda)$.
2. *Run* $(\mathsf{cm}, m, \mathcal{W}) \leftarrow A_{\mathsf{CMUniq}}(\mathsf{pp}, \boldsymbol{\xi})$ *to obtain a classical commitment* cm *and message* m*, as well as a register* $\mathcal{W}$ *that stores a superposition of openings* ω*.*
3. *Measure* $\mathsf{CM.Check}(\mathsf{pp}, \mathsf{cm}, m, \mathcal{W})$ *and output* $\texttt{Abort}$ *if the outcome is* 0*.*
4. *Run* $(m', \mathcal{W}) \leftarrow A_{\mathsf{CMUniq}}(\mathcal{W})$ *to obtain another classical message* m' *and a superposition of openings* ω *in* $\mathcal{W}$*. Output* $\texttt{Abort}$ *if* $m' = m$*.*
5. *Measure* $\mathsf{CM.Check}(\mathsf{pp}, \mathsf{cm}, m', \mathcal{W})$ *and output* 1 *if the outcome is* 1*; output* 0 *otherwise.*

Moreover, we say "A_{CMUniq} wins*" as shorthand for the event* $\mathsf{CMUniqExp}(\lambda, A_{\mathsf{CMUniq}}, \boldsymbol{\xi}) = 1$.

Construction 4.10. Given a vector commitment $\mathsf{VC} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Check})$, $L \in \mathbb{N}$ and $s \in [L]$, we construct the following commitment scheme $\mathsf{CM} = \mathsf{CM}^{(\mathsf{VC}, L, s)}$.

- $\mathsf{CM.Gen}(1^\lambda)$:
  1. Output $\mathsf{pp} \leftarrow \mathsf{VC.Gen}(1^\lambda, L)$.
- $\mathsf{CM.Commit}(\mathsf{pp}, m)$:
  1. Let $\alpha \in \Sigma$ be an arbitrary symbol and $\mathsf{idx} \in [L]$ an arbitrary coordinate; set $m_{\mathsf{VC}} \in \Sigma^L$ as $m_{\mathsf{VC}}(\mathsf{idx}) = m$ and $m_{\mathsf{VC}}(i) = \alpha$ for every $i \neq \mathsf{idx}$. Compute $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{aux}) \leftarrow \mathsf{VC.Commit}(\mathsf{pp}, m_{\mathsf{VC}})$.
  2. Let Q be an arbitrary set of size $s - 1$ such that $\mathsf{idx} \notin Q$. Run $\mathsf{pf} \leftarrow \mathsf{VC.Open}(\mathsf{pp}, Q \cup \{\mathsf{idx}\}, \mathsf{aux})$.
  3. Output $(\mathsf{cm} := (\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx}), \omega := (\mathsf{ans}, \mathsf{pf}))$, where $\mathsf{ans}\colon Q \cup \{\mathsf{idx}\} \to \Sigma$ is the partial function such that $\mathsf{ans}(i) = \alpha$ for all $i \in Q$ and $\mathsf{ans}(\mathsf{idx}) = \bot$.
- $\mathsf{CM.Check}(\mathsf{pp}, \mathsf{cm}, m, \omega)$:
  1. Parse cm as $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx})$ and ω as $(\mathsf{ans}, \mathsf{pf})$ where ans is a partial function with domain in $[L]$.
  2. Output $(\mathsf{ans}(\mathsf{idx}) = \bot) \wedge (|\mathsf{supp}(\mathsf{ans})| = s - 1) \wedge \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}_{\mathsf{VC}}, \mathsf{ans}[\mathsf{idx} \to m], \mathsf{pf})$.

Lemma 4.11. *If* VC *has post-quantum position binding error* $\epsilon_{\mathsf{VCBinding}}$*, then, for every security parameter* $\lambda \in \mathbb{N}$*, message length* $L \in \mathbb{N}$*, query set size* $s \in [L]$*, and adversary size bound* $t_{\mathsf{CMBinding}} \in \mathbb{N}$*, the post-quantum binding error of* $\mathsf{CM}^{(\mathsf{VC}, L, s)}$ *satisfies*

$$\epsilon_{\mathsf{CMBinding}}(\lambda, t_{\mathsf{CMBinding}}) \leq \epsilon_{\mathsf{VCBinding}}(\lambda, L, s, t_{\mathsf{VCBinding}}) \quad \textit{where} \quad t_{\mathsf{VCBinding}} = O(t_{\mathsf{CMBinding}}) \ .$$

Conversely, if $\mathsf{CM}^{(\mathsf{VC}, L, s)}$ *has post-quantum binding error* $\epsilon_{\mathsf{CMBinding}}$*, then for every* $\lambda \in \mathbb{N}$ *and* $t_{\mathsf{VCBinding}} \in \mathbb{N}$*,*

$$\epsilon_{\mathsf{VCBinding}}(\lambda, L, s, t_{\mathsf{VCBinding}}) \leq \epsilon_{\mathsf{CMBinding}}(\lambda, t_{\mathsf{CMBinding}}) \quad \textit{where} \quad t_{\mathsf{CMBinding}} = O(t_{\mathsf{VCBinding}}) \ .$$

Proof. Fix $L \in \mathbb{N}$, $s \in [L]$ and $\mathsf{CM} = \mathsf{CM}^{(\mathsf{VC},L,s)}$. We define a bijection between adversaries $A_{\mathsf{VCBinding}}$ for VC and adversaries $A_{\mathsf{CMBinding}}$ for CM as follows:

- $A_{\mathsf{VCBinding}}(\mathsf{pp}, \boldsymbol{\xi})$:
  1. Run $(\mathsf{cm}, m, m', \omega, \omega') \leftarrow A_{\mathsf{CMBinding}}(\mathsf{pp}, \boldsymbol{\xi})$.
  2. Parse cm as $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx})$, ω as $(\mathsf{ans}, \mathsf{pf})$ and ω' as $(\mathsf{ans}', \mathsf{pf}')$. Output `Abort` if $\mathsf{ans}(\mathsf{idx}) \neq \bot$ or $\mathsf{ans}'(\mathsf{idx}) \neq \bot$.
  3. Output $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{ans}[\mathsf{idx} \to m], \mathsf{ans}'[\mathsf{idx} \to m'], \mathsf{pf}, \mathsf{pf}')$.
- $A_{\mathsf{CMBinding}}(\mathsf{pp}, \boldsymbol{\xi})$:
  1. Run $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{ans}, \mathsf{ans}', \mathsf{pf}, \mathsf{pf}') \leftarrow A_{\mathsf{VCBinding}}(\mathsf{pp}, \boldsymbol{\xi})$.
  2. Let $\mathsf{idx} \in \mathsf{supp}(\mathsf{ans}) \cap \mathsf{supp}(\mathsf{ans}')$ such that $\mathsf{ans}(\mathsf{idx}) \neq \mathsf{ans}'(\mathsf{idx})$; if none exists, output `Abort`. Set $m \coloneqq \mathsf{ans}(\mathsf{idx})$ and $m' \coloneqq \mathsf{ans}'(\mathsf{idx})$.
  3. Set $\mathsf{cm} \coloneqq (\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx})$, $\omega \coloneqq (\mathsf{ans}[\mathsf{idx} \to \bot], \mathsf{pf})$ and $\omega' \coloneqq (\mathsf{ans}'[\mathsf{idx} \to \bot], \mathsf{pf}')$.
  4. Output $(\mathsf{cm}, m, m', \omega, \omega')$.

The circuit size $t_{\mathsf{VCBinding}}$ of $A_{\mathsf{VCBinding}}$ is $O(t_{\mathsf{CMBinding}})$, as it only parses the output of $A_{\mathsf{VCBinding}}$ and augments answers by a single symbol. Similarly, $t_{\mathsf{CMBinding}} = O(t_{\mathsf{VCBinding}})$. Finally, since

$$\begin{aligned}&\mathsf{CM.Check}(\mathsf{pp}, \mathsf{cm}, m, \omega)\\ &= (\mathsf{ans}(\mathsf{idx}) = \bot) \wedge (|\mathsf{supp}(\mathsf{ans})| = s - 1) \wedge \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}_{\mathsf{VC}}, \mathsf{ans}[\mathsf{idx} \to m], \mathsf{pf})\end{aligned}$$

by the definition of CM (and likewise for m' and ω'), it follows from Definition 3.8 that $A_{\mathsf{VCBinding}}$ wins if and only if $A_{\mathsf{CMBinding}}$ wins. In particular, $\epsilon_{\mathsf{VCBinding}}(\lambda, L, s, t_{\mathsf{VCBinding}}) = \epsilon_{\mathsf{CMBinding}}(\lambda, t_{\mathsf{CMBinding}})$. □

4.2 Collapse position binding implies unique-message position binding

Lemma 4.12. *Let* VC *be a vector commitment with collapse position binding error* $\epsilon_{\mathsf{VCCollapse}}$*. For every security parameter* $\lambda \in \mathbb{N}$*, message length* $L \in \mathbb{N}$*, query set size* $s \in [L]$ *and adversary size bound* $t_{\mathsf{CMCollapse}} \in \mathbb{N}$*, the commitment scheme* $\mathsf{CM} = \mathsf{CM}^{(\mathsf{VC},L,s)}$ *has collapse binding error*

$$\epsilon_{\mathsf{CMCollapse}}(\lambda, t_{\mathsf{CMCollapse}}) \leq \epsilon_{\mathsf{VCCollapse}}(\lambda, L, s, t_{\mathsf{VCCollapse}}) \quad \textit{where} \quad t_{\mathsf{VCCollapse}} = O(t_{\mathsf{CMCollapse}}) \ .$$

Conversely, if $\mathsf{CM}^{(\mathsf{VC},L,s)}$ *has collapse binding error* $\epsilon_{\mathsf{CMCollapse}}$*, then for every* $t_{\mathsf{VCCollapse}} \in \mathbb{N}$*,*

$$\epsilon_{\mathsf{VCCollapse}}(\lambda, L, s, t_{\mathsf{VCCollapse}}) \leq \epsilon_{\mathsf{CMCollapse}}(\lambda, t_{\mathsf{CMCollapse}}) \quad \textit{where} \quad t_{\mathsf{CMCollapse}} = O(t_{\mathsf{VCCollapse}}) \ .$$

Proof. We define the following bijection between adversaries $A_{\mathsf{VCCollapse}}$ against VC and adversaries $A_{\mathsf{CMCollapse}}$ against CM.

- $A_{\mathsf{VCCollapse}}(\mathsf{pp}, \boldsymbol{\xi})$:
  1. Run $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx}, \mathcal{M}, \mathcal{W}) \leftarrow A_{\mathsf{CMCollapse}}(\mathsf{pp}, \boldsymbol{\xi})$ to obtain a classical commitment-index pair $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx})$ and registers $(\mathcal{M}, \mathcal{W})$ where $\mathcal{M}$ contains a superposition of messages m and $\mathcal{W}$ is parsed as $(\mathcal{A}, \mathcal{O})$, containing superpositions of partial functions ans and opening proofs pf, respectively.
  2. Measure $\mathcal{A}(\mathsf{idx}) = \bot$ and output `Abort` if the outcome is 0.
  3. Swap $\mathcal{M}$ with $\mathcal{A}_{\mathsf{idx}}$ and output $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx}, \mathcal{A}, \mathcal{O})$ where $\mathcal{A}_{\mathsf{idx}}$ denotes the idx-th register of $\mathcal{A}$. (Note that this is an implementation of the operation $\mathcal{A}[\mathsf{idx} \to \mathcal{M}]$ which doesn't coherently copy the register $\mathcal{M}$, whose inverse is Step 1 of $A_{\mathsf{VCCollapse}}(\mathcal{A}, \mathcal{O})$.)

$A_{\mathsf{VCCollapse}}(\mathcal{A}, \mathcal{O})$:

1. Prepare a register $\mathcal{M}$ with state $|\bot\rangle$ and swap it with $\mathcal{A}_{\mathsf{idx}}$.
2. Set $\mathcal{W} \coloneqq (\mathcal{A}, \mathcal{O})$, and output $b' \leftarrow A_{\mathsf{CMCollapse}}(\mathcal{M}, \mathcal{W})$.

- $A_{\mathsf{CMCollapse}}(\mathsf{pp}, \boldsymbol{\xi})$:
  1. Run $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx}, \mathcal{A}, \mathcal{O}) \leftarrow A_{\mathsf{VCCollapse}}(\mathsf{pp}, \boldsymbol{\xi})$ to obtain a classical commitment-index pair $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx})$ and registers $(\mathcal{A}, \mathcal{O})$ that contain superpositions of partial functions ans and opening proofs pf, respectively.
  2. Measure $\mathcal{A}(\mathsf{idx}) \neq \bot$ and output $\mathtt{Abort}$ if the outcome is 0.
  3. Prepare a register $\mathcal{M}$ with state $|\bot\rangle$ and swap it with $\mathcal{A}_{\mathsf{idx}}$.
     Set $\mathcal{W} \coloneqq (\mathcal{A}, \mathcal{O})$ and output $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx}, \mathcal{M}, \mathcal{W})$.

$A_{\mathsf{CMCollapse}}(\mathcal{M}, \mathcal{W})$:

1. Parse $\mathcal{W}$ as $(\mathcal{A}, \mathcal{O})$, swap $\mathcal{M}$ with $\mathcal{A}_{\mathsf{idx}}$ and output $b' \leftarrow A_{\mathsf{VCCollapse}}(\mathcal{A}, \mathcal{O})$.

By construction of CM, $A_{\mathsf{VCCollapse}}$ outputs 1 if and only if $A_{\mathsf{CMCollapse}}$ outputs 1. In particular, we have $\epsilon_{\mathsf{CMCollapse}}(\lambda, t_{\mathsf{CMCollapse}}) = \epsilon_{\mathsf{VCCollapse}}(\lambda, L, s, t_{\mathsf{VCCollapse}})$ with $t_{\mathsf{VCCollapse}} = O(t_{\mathsf{CMCollapse}})$ and $t_{\mathsf{CMCollapse}} = O(t_{\mathsf{VCCollapse}})$ in the first and second constructions, respectively. □

Lemma 4.13 ([LMS22b, Lemma 4.2])**.** *For every security parameter $\lambda \in \mathbb{N}$, adversary size bound $t_{\mathsf{CMUniq}} \in \mathbb{N}$, and t_{CMUniq}-size quantum circuit A_{CMUniq},*

$$\epsilon_{\mathsf{CMUniq}}(\lambda, t_{\mathsf{CMUniq}}) \leq 4 \cdot \epsilon_{\mathsf{CMCollapse}}(\lambda, t_{\mathsf{CMCollapse}}) \quad \textit{where} \quad t_{\mathsf{CMCollapse}} = O(t_{\mathsf{CMUniq}}) \ .$$

Lemma 4.14. *Let VC be a vector commitment scheme with unique-message position binding error $\epsilon_{\mathsf{VCUniq}}$. Consider the commitment scheme $\mathsf{CM}^{(\mathsf{VC}, L, s)}$ constructed from VC as in Construction 4.10 with security parameter $\lambda \in \mathbb{N}$, message length $L \in \mathbb{N}$ and query set size $s \in [L]$. Then, $\mathsf{CM}^{(\mathsf{VC}, L, s)}$ has unique-message binding error $\epsilon_{\mathsf{CMUniq}}$ satisfying*

$$\epsilon_{\mathsf{CMUniq}}(\lambda, t_{\mathsf{CMUniq}}) \leq \epsilon_{\mathsf{VCUniq}}(\lambda, L, s, t_{\mathsf{VCUniq}}) \quad \textit{where} \quad t_{\mathsf{VCUniq}} = O(t_{\mathsf{CMUniq}}) \ .$$

Conversely, if $\mathsf{CM}^{(\mathsf{VC}, L, s)}$ has unique-message position binding error $\epsilon_{\mathsf{CMUniq}}$, then VC has unique-binding error $\epsilon_{\mathsf{VCUniq}}$ satisfying

$$\epsilon_{\mathsf{VCUniq}}(\lambda, L, s, t_{\mathsf{VCUniq}}) \leq \epsilon_{\mathsf{CMUniq}}(\lambda, t_{\mathsf{CMUniq}}) \quad \textit{where} \quad t_{\mathsf{CMUniq}} = O(t_{\mathsf{VCUniq}}) \ .$$

Proof. We define a bijection between unique-message binding adversaries A_{CMUniq} against $\mathsf{CM}^{(\mathsf{VC}, L, s)}$ and unique-message position binding adversaries A_{VCUniq} against VC as follows.

- $A_{\mathsf{CMUniq}}(\mathsf{pp}, \boldsymbol{\xi})$:
  1. Run $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx}, m, \mathcal{A}, \mathcal{O}) \leftarrow A_{\mathsf{VCUniq}}(\mathsf{pp}, \boldsymbol{\xi})$ to obtain a classical commitment-index-symbol triple $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx}, m)$ and registers $(\mathcal{A}, \mathcal{O})$ that contain superpositions of partial functions ans and opening proofs pf.
  2. Set $\mathsf{cm} \coloneqq (\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx})$, $\mathcal{W} \coloneqq (\mathcal{A}, \mathcal{O})$ and output $(\mathsf{cm}, m, \mathcal{W})$.

$A_{\mathsf{CMUniq}}(\mathcal{W})$:

1. Parse $\mathcal{W}$ as $(\mathcal{A}, \mathcal{O})$ and run $(m', \mathcal{A}, \mathcal{O}) \leftarrow A_{\mathsf{VCUniq}}(\mathcal{A}, \mathcal{O})$ to receive another classical message m' and quantum registers $(\mathcal{A}, \mathcal{O})$.

2. Set $\mathcal{W} := (\mathcal{A}, \mathcal{O})$ and output $(m', \mathcal{W})$.

- $A_{\mathsf{VCUniq}}(\mathsf{pp}, \boldsymbol{\xi})$:
  1. Run $(\mathsf{cm}, m, \mathcal{W}) \leftarrow A_{\mathsf{CMUniq}}(\mathsf{pp}, \boldsymbol{\xi})$, parsing cm as $(\mathsf{cm}_{\mathsf{VC}}, \mathsf{idx})$ and $\mathcal{W}$ as $(\mathcal{A}, \mathcal{O})$.
  2. Output $(\mathsf{cm}, \mathsf{idx}, m, \mathcal{A}, \mathcal{O})$.

  $A_{\mathsf{VCUniq}}(\mathcal{A}, \mathcal{O})$:
  1. Set $\mathcal{W} := (\mathcal{A}, \mathcal{O})$ and run $(m', \mathcal{W}) \leftarrow A_{\mathsf{CMUniq}}(\mathcal{W})$.
  2. Output $(m', \mathcal{A}, \mathcal{O})$.

By construction of CM, A_{CMUniq} wins if and only if A_{VCUniq} does. In particular, we have $\epsilon_{\mathsf{CMUniq}}(\lambda, t_{\mathsf{CMUniq}}) = \epsilon_{\mathsf{VCUniq}}(\lambda, L, s, t_{\mathsf{VCUniq}})$ with $t_{\mathsf{CMUniq}} = O(t_{\mathsf{VCUniq}})$ and $t_{\mathsf{VCUniq}} = O(t_{\mathsf{CMUniq}})$ in the first and second constructions, respectively. □

Lemma 4.15. *Let* VC *be a vector commitment scheme with unique-message position binding error* $\epsilon_{\mathsf{VCUniq}}$*. For every security parameter* $\lambda \in \mathbb{N}$*, message length* $L \in \mathbb{N}$*, query set size* $s \in [L]$*, and adversary size bound* $t_{\mathsf{VCBinding}} \in \mathbb{N}$*,* VC *has post-quantum position-binding error* $\epsilon_{\mathsf{VCBinding}}$ *satisfying*

$$\epsilon_{\mathsf{VCBinding}}(\lambda, L, s, t_{\mathsf{VCBinding}}) \leq \epsilon_{\mathsf{VCUniq}}(\lambda, L, s, t_{\mathsf{VCUniq}}) \quad \textit{where} \quad t_{\mathsf{VCUniq}} = O(t_{\mathsf{VCBinding}}) \ .$$

Proof. Assume, towards contradiction, the existence of an adversary $A_{\mathsf{VCBinding}}$ that wins with probability $\epsilon_{\mathsf{VCBinding}} > \epsilon_{\mathsf{VCUniq}}$. We construct an adversary A_{VCUniq} with the same winning probability, contradicting the assumed inequality.

$A_{\mathsf{VCUniq}}(\mathsf{pp}, \boldsymbol{\xi})$:
1. Run $(\mathsf{cm}, \mathsf{ans}, \mathsf{ans}', \mathsf{pf}, \mathsf{pf}') \leftarrow A_{\mathsf{VCBinding}}(\mathsf{pp}, \boldsymbol{\xi})$.
2. Let idx be a coordinate in $\mathsf{supp}(\mathsf{ans}) \cap \mathsf{supp}(\mathsf{ans}')$ such that $\mathsf{ans}(\mathsf{idx}) \neq \mathsf{ans}'(\mathsf{idx})$; output `Abort` if none exists. Set $m = \mathsf{ans}(\mathsf{idx})$ and $m' = \mathsf{ans}'(\mathsf{idx})$.
3. Output $(\mathsf{cm}, \mathsf{idx}, m, \mathsf{ans}[\mathsf{idx} \to \perp], \mathsf{pf})$.

$A_{\mathsf{VCUniq}}(\mathcal{A}, \mathcal{O})$:
1. Output $(m', \mathsf{ans}'[\mathsf{idx} \to \perp], \mathsf{pf}')$.

Clearly, A_{VCUniq} wins if and only if $A_{\mathsf{VCBinding}}$ does (making one black-box call to $A_{\mathsf{VCBinding}}$, without any additional quantum operations; in particular, without manipulating the input registers $\mathcal{A}$ or $\mathcal{O}$ in the second stage). □

Proof of Lemma 4.5. Lemma 4.15 proves to the first inequality, $\epsilon_{\mathsf{VCBinding}} \leq \epsilon_{\mathsf{VCUniq}}$. The second follows from Lemma 4.14 ($\epsilon_{\mathsf{VCUniq}} \leq \epsilon_{\mathsf{CMUniq}}$), Lemma 4.13 ($\epsilon_{\mathsf{CMUniq}} \leq 4 \cdot \epsilon_{\mathsf{CMCollapse}}$) and Lemma 4.12 ($\epsilon_{\mathsf{CMCollapse}} \leq \epsilon_{\mathsf{VCCollapse}}$). □

5 Quantum security reduction for the IBCS protocol

Towards proving Theorem 1, we use quantum rewinding to construct an IOP prover whose success probability is close to that of the given argument adversary. In Section 6 we show how this implies soundness and knowledge soundness of the IBCS protocol.

Lemma 5.1 (Security reduction lemma)**.** *For every error bound $\epsilon > 0$, there exists an adversary $\widetilde{\mathbf{P}}$ for IOP such that for every auxiliary input state $\boldsymbol{\xi}$, circuit size bound $t_{\mathsf{ARG}} \in \mathbb{N}$, and t_{ARG}-size quantum adversary $\widetilde{\mathbb{P}}$ for ARG,*

$$\begin{aligned}
&\Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n & \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ \wedge\, b = 1 & (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ & b \leftarrow \left\langle \widetilde{\mathbb{P}}(\mathsf{aux}), \mathbb{V}(\mathsf{pp}, \mathbb{x}) \right\rangle \end{array}\right] \\
&\le \Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n & (\mathbb{x}, \mathbf{aux}) \leftarrow \widetilde{\mathbf{P}}^{\widetilde{\mathbb{P}}}(\boldsymbol{\xi}) \\ \wedge\, b = 1 & b \leftarrow \left\langle \widetilde{\mathbf{P}}^{\widetilde{\mathbb{P}}}(\mathbf{aux}), \mathbf{V}(\mathbb{x}) \right\rangle \end{array}\right] \\
&\quad + \sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \ ,
\end{aligned}$$

where $t_{\mathsf{VCCollapse}} = O(t_{\mathsf{Rewind}}(\frac{2\mathsf{k}\cdot\mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}} + t_{\mathbb{V}})$ and $t_{\mathsf{Rewind}}(\frac{2\mathsf{k}\cdot\mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}})$ is the rewinding time in Equation 1. The circuit size of the IOP prover $\widetilde{\mathbf{P}}$ is $O(\mathsf{k} \cdot (t_{\mathsf{Rewind}}(\frac{2\mathsf{k}\cdot\mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}}))$.

5.1 Construction of the IOP adversary $\widetilde{\mathrm{P}}$

For every round $i \in [\mathsf{k}]$, we construct a *reductor* $\mathfrak{R}_i$, a quantum algorithm that, using the quantum rewinding procedure in Construction 3.17, rewinds $\widetilde{\mathbb{P}}$ multiple times to recover an IOP string for round i. The IOP prover $\widetilde{\mathbf{P}}$ that we construct then runs the reductors $(\mathfrak{R}_i)_{i\in[\mathsf{k}]}$ in sequence.

For each $i \in [\mathsf{k}]$, verifier randomness $(r_j)_{j<i}$, proof strings $(\widetilde{\pi}_j)_{j<i}$ and commitment cm_i, we define the following quantum game $\mathsf{G}_i = \mathsf{G}\big(\mathsf{pp}, \mathbb{x}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j<i}, \mathsf{cm}_i\big)$ (see Definition 3.15) capturing rounds $i, \ldots, \mathsf{k}+1$ of the argument system.

Construction 5.2. Define the quantum game $\mathsf{G}_i = \mathsf{G}\big(\mathsf{pp}, \mathbb{x}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j<i}, \mathsf{cm}_i\big) \coloneqq (C_i, Z_i, f_i)$ as follows.

- The question space is $C_i \coloneqq \{0,1\}^{\mathsf{r}_i} \times \cdots \times \{0,1\}^{\mathsf{r}_\mathsf{k}}$.
- The answer space Z_i is the set of tuples $\big\{\big((\mathsf{cm}_j)_{j\in[i+1,\mathsf{k}]}, (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[i,\mathsf{k}]}\big)\big\}$ where cm_j is the j-th round commitment and $(\mathsf{ans}_j, \mathsf{pf}_j)$ is the j-th answer-opening pair in (the final) round $\mathsf{k}+1$; the commitments are contained in registers $(\mathcal{Z}_j)_{j\in[i+1,\mathsf{k}]}$ (i.e., $\mathcal{Z}_j$ contains cm_j), and the answer-opening pairs are contained in register $\mathcal{Z}_{\mathsf{k}+1} = \otimes_{j\in[\mathsf{k}]}\mathcal{Z}_{\mathsf{k}+1,j}$, where $\mathcal{Z}_{\mathsf{k}+1,j} \coloneqq (\mathcal{A}_j \otimes \mathcal{O}_j)$ contains a superposition of $(\mathsf{ans}_j, \mathsf{pf}_j)$.
- The game predicate $f_i\big((r_j)_{j\in[i,\mathsf{k}]}, (\mathsf{cm}_j)_{j\in[i+1,\mathsf{k}]}, (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[i,\mathsf{k}]}\big)$ is 1 if $\mathbf{V}^{((\widetilde{\pi}_j)_{j<i}, (\mathsf{ans}_j)_{j\ge i})}(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}) = 1$ and, for every $j \in [i+1, \mathsf{k}]$, $\mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}_j, \mathsf{ans}_j, \mathsf{pf}_j) = 1$.

Recall that $\widetilde{\mathbb{P}}$ is described by the sequence of k unitary dilations of the algorithm applied by $\widetilde{\mathbb{P}}$ (to its internal state and verifier message) at each round (see Section 3.1).

Definition 5.3. *We denote by $\widetilde{\mathbb{P}}_{>i} \coloneqq U_{\widetilde{\mathbb{P}}_{\mathsf{k}+1}} \cdots U_{\widetilde{\mathbb{P}}_{i+1}}$ the sequential application of (unitary dilations of) the argument prover's round-j strategy $\widetilde{\mathbb{P}}_j$ for $j \in [i+1, \mathsf{k}+1]$.*

We define $\mathsf{AlgRewind}_{\mathsf{G}_i} := (\mathsf{RewindInit}_{\mathsf{G}_i}, \mathsf{RewindStrat}_{\mathsf{G}_i}, \mathsf{RewindRepair}_{\mathsf{G}_i})$ to be the rewinding algorithm in Construction 3.16 for game G_i. The algorithm $\mathsf{AlgRewind}_{\mathsf{G}_i}$ rewinds the argument adversary $\widetilde{\mathbb{P}}_{>i}$ multiple times to recover the i-th IOP string, and Claim 3.18 shows this process does not significantly affect its success probability. Below, we construct the reductor $\mathfrak{R}_i$ using $\mathsf{AlgRewind}_{\mathsf{G}_i}$.

Construction 5.4. Given $i \in [\mathsf{k}]$ and $\epsilon > 0$, we construct the reductor $\mathfrak{R}_i$ using a sampler S_i (defined next) as a subroutine.

$S_i^{\widetilde{\mathbb{P}}}\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i]}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j<i}, \mathsf{t}, \boldsymbol{\rho}\big)$:

1. Initialize $\mathsf{ANS} := \varnothing$ and $c := 0$.
2. Let $\boldsymbol{\sigma}^{(i,0)} := \boldsymbol{\rho}$ and $\boldsymbol{r} := (r_j)_{j<i}$.
3. Run $\mathbf{aux}_0 \leftarrow \mathsf{RewindInit}_{\mathsf{G}_i}(\widetilde{\mathbb{P}}_{>i}, \mathsf{t}, \epsilon/2\mathsf{k}, \boldsymbol{\sigma}^{(i,0)})$.
4. For $\iota \in [\mathsf{t}]$:
   (a) Sample IOP verifier randomness: $\boldsymbol{r}' := (r'_j)_{j\in[i,\mathsf{k}]} \leftarrow \{0,1\}^{\mathsf{r}_i+\cdots+\mathsf{r}_\mathsf{k}}$.
   (b) Run $\big((\mathcal{Z}_j)_{j\in[i+1,\mathsf{k}+1]}, \mathbf{aux}'_\iota\big) \leftarrow \mathsf{RewindStrat}_{\mathsf{G}_i}(\boldsymbol{r}', \mathbf{aux}_{\iota-1})$.
   (c) Measure $b \leftarrow f_i(\boldsymbol{r}', ((\mathcal{Z}_j)_{j\in[i+1,\mathsf{k}]}, \mathcal{Z}_{\mathsf{k}+1}))$. If $b = 0$, skip to Step 4f.
   (d) Define $Q^{(\mathsf{ANS})} := \cup_{\mathsf{ans}\in\mathsf{ANS}}\mathsf{supp}(\mathsf{ans})$ and initialize $\widetilde{\pi} := \bot^{\mathsf{L}_i}$.
   (e) For $m \in [\mathsf{q}_i]$:
      i. Compute $q \leftarrow \mathsf{Query}^{\widetilde{\pi}}(\mathbb{x}, (\boldsymbol{r}, \boldsymbol{r}'), (\widetilde{\pi}_j)_{j<i}, m)$.
      ii. If $q \in Q^{(\mathsf{ANS})}$, let j be minimal such that $\mathsf{ans}_j \in \mathsf{ANS}$ and $q \in \mathsf{supp}(\mathsf{ans}_j)$. Set $\widetilde{\pi}(q) := \mathsf{ans}_j(q)$.
      iii. If $c < \mathsf{L}_i$ and $q \notin Q^{(\mathsf{ANS})}$, measure $\mathcal{A}_i$ in the computational basis to obtain response ans'_ι, add ans'_ι to ANS and increment $c := c + 1$. Skip to Step 4f.
   (f) Run $\mathbf{aux}_\iota \leftarrow \mathsf{RewindRepair}_{\mathsf{G}_i}((\mathcal{Z}_j)_{j\in[i+1,\mathsf{k}+1]}, b, \mathbf{aux}'_\iota)$.
5. Parse $\mathbf{aux}_\mathsf{t}$ as $(\widetilde{\mathbb{P}}_{>i}, \gamma, \delta, \mathsf{t}, p_\mathsf{t}, \boldsymbol{\sigma}^{(i,\mathsf{t})})$ and output $(\mathsf{ANS}, \boldsymbol{\sigma}^{(i,\mathsf{t})})$.

The sampler S_i's circuit size is dominated by the size for running $\mathsf{Rewind}_{\mathsf{G}_i}(\widetilde{\mathbb{P}}_{\geq i}, \mathsf{t}, \epsilon/2\mathsf{k}, \mathsf{t}, \boldsymbol{\rho})$ (specifically, Steps 3, 4c and 4e, skipping Step 4(e)iii; see Construction 3.17) and thus has circuit size $O\big(t_{\mathsf{Rewind}}(\mathsf{t}, \epsilon/2\mathsf{k}, t_{\mathbb{V}}, t_{\mathsf{ARG}})\big)$. We now describe the reductor; let $T := \frac{2\mathsf{k}\mathsf{L}_{\mathsf{max}}}{\epsilon}$.

$\mathfrak{R}_i^{\widetilde{\mathbb{P}}}\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i]}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j<i}, \boldsymbol{\rho}\big)$:

1. Sample $\mathsf{t} \leftarrow [0, T]$.
2. Run $(\mathsf{ANS}, \boldsymbol{\sigma}^{(i)}) \leftarrow S_i^{\widetilde{\mathbb{P}}}\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i]}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j<i}, \mathsf{t}, \boldsymbol{\rho}\big)$.
3. Initialize $\widetilde{\pi}_i := \bot^{\mathsf{L}_i}$.
4. For every $\mathsf{ans} \in \mathsf{ANS}$ and $q \in \mathsf{supp}(\mathsf{ans})$, set $\widetilde{\pi}_i(q) := \mathsf{ans}(q)$.
5. Output $(\widetilde{\pi}_i, \boldsymbol{\sigma}^{(i)})$.

$\mathfrak{R}_i$'s circuit size is dominated by that of S_i.

Construction 5.5. We construct the IOP adversary $\widetilde{\mathbf{P}}$ by using the reductors $(\mathfrak{R}_i)_{i\in[\mathsf{k}]}$ as follows.

- $\widetilde{\mathbf{P}}^{\widetilde{\mathbb{P}}}(\boldsymbol{\xi})$:
  1. Run $\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)$.
  2. Run $(\mathbb{x}, \boldsymbol{\sigma}^{(0)}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi})$. (Recall that we do not assume any structure on the auxiliary information aux output by $\widetilde{\mathbb{P}}$, and thus denote it as a quantum state $\boldsymbol{\sigma}^{(0)}$.)

3. Set $\mathbf{aux}_0 \coloneqq (\mathsf{pp}, \mathbb{x}, \boldsymbol{\sigma}^{(0)})$.
4. Output $(\mathbb{x}, \mathbf{aux}_0)$.

- $\widetilde{\mathbf{P}}^{\widetilde{\mathbb{P}}}(\mathbf{aux}_0)$:
  1. Parse $\mathbf{aux}_0$ as $(\mathsf{pp}, \mathbb{x}, \boldsymbol{\sigma}^{(0)})$.
  2. Run $(\mathsf{cm}_1, \boldsymbol{\rho}^{(1)}) \leftarrow \widetilde{\mathbb{P}}(\boldsymbol{\sigma}^{(0)})$.
  3. Run $(\widetilde{\pi}_1, \boldsymbol{\sigma}^{(1)}) \leftarrow \mathfrak{R}_1^{\widetilde{\mathbb{P}}}\big(\mathsf{pp}, \mathbb{x}, \mathsf{cm}_1, \boldsymbol{\rho}^{(1)}\big)$.
  4. Set $\mathbf{aux}_1 \coloneqq (\mathsf{pp}, \mathbb{x}, \mathsf{cm}_1, \widetilde{\pi}_1, \boldsymbol{\sigma}^{(1)})$.
  5. Output $(\pi_1, \mathbf{aux}_1)$.

- $\widetilde{\mathbf{P}}^{\widetilde{\mathbb{P}}}(r, \mathbf{aux})$:
  1. Parse $\mathbf{aux}$ as $\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j<i}, (r_j)_{j<i-1}, (\widetilde{\pi}_j)_{j<i}, \boldsymbol{\sigma}^{(i-1)}\big)$.
  2. Run $(\mathsf{cm}_i, \boldsymbol{\rho}^{(i)}) \leftarrow \widetilde{\mathbb{P}}(r, \boldsymbol{\sigma}^{(i-1)})$.
  3. Set $r_{i-1} \coloneqq r$ and run $(\widetilde{\pi}_i, \boldsymbol{\sigma}^{(i)}) \leftarrow \mathfrak{R}_i^{\widetilde{\mathbb{P}}}\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i]}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j<i}, \boldsymbol{\rho}^{(i)}\big)$.
  4. Set $\mathbf{aux}_i \coloneqq \big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i]}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j\in[i]}, \boldsymbol{\sigma}^{(i)}\big)$.
  5. If $i < \mathsf{k}$, output $(\widetilde{\pi}_i, \mathbf{aux}_i)$; output $\widetilde{\pi}_{\mathsf{k}}$ otherwise.

The IOP prover $\widetilde{\mathbf{P}}$'s circuit size is dominated by the size for running the reductor $\mathfrak{R}_i$ and the argument prover $\widetilde{\mathbb{P}}$ for every round $i \in [\mathsf{k}]$. Hence, its circuit size is $O\big(\mathsf{k} \cdot (t_{\mathsf{Rewind}}(T, \epsilon/2\mathsf{k}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}})\big)$.

5.2 Proof of Lemma 5.1

We use a hybrid argument to show that the success probability of $\widetilde{\mathbf{P}}$ is not much smaller than that of $\widetilde{\mathbb{P}}$.

Construction 5.6. For every $i^* \in [0, \mathsf{k}]$, we define a hybrid $\mathsf{H}_{i^*} \coloneqq (\widetilde{\mathcal{P}}_{i^*}, \mathcal{V}_{i^*})$ where the hybrid prover $\widetilde{\mathcal{P}}_{i^*}$ behaves like the IOP prover $\widetilde{\mathbf{P}}$ for the first i^* rounds and the argument prover $\widetilde{\mathbb{P}}$ for the rest; the hybrid verifier $\mathcal{V}_{i^*}$ is adapted accordingly.

- $\widetilde{\mathcal{P}}_{i^*}^{\widetilde{\mathbb{P}}}(\mathsf{pp}, \boldsymbol{\xi})$:
  1. Run $(\mathbb{x}, \boldsymbol{\tau}^{(0)}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi})$.
  2. Set $\mathbf{aux}_0 \coloneqq (\mathsf{pp}, \mathbb{x}, \boldsymbol{\tau}^{(0)})$.
  3. Output $(\mathbb{x}, \mathbf{aux}_0)$.

- $\widetilde{\mathcal{P}}_{i^*}^{\widetilde{\mathbb{P}}}(r, \mathbf{aux})$:
  1. If $r = ()$, skip to Step 4.
  2. Parse $\mathbf{aux}$ as $(i-1, \boldsymbol{\tau}^{(i-1)})$; if it fails, parse as $\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j<i}, (r_j)_{j<i-1}, (\widetilde{\pi}_j)_{j<i}, \boldsymbol{\tau}^{(i-1)}\big)$.
  3. If $i = \mathsf{k}+1$, run $((\mathsf{ans}_j, \mathsf{pf}_j))_{j\in[\mathsf{k}]} \leftarrow \widetilde{\mathbb{P}}(r, \boldsymbol{\tau}^{(\mathsf{k})})$ and output $((\mathsf{ans}_j, \mathsf{pf}_j))_{j\in[i^*+1,\mathsf{k}]}$.
  4. Set $r_{i-1} \coloneqq r$ and run $(\mathsf{cm}_i, \boldsymbol{\rho}^{(i)}) \leftarrow \widetilde{\mathbb{P}}(r_{i-1}, \boldsymbol{\tau}^{(i-1)})$.
  5. If $i > i^*$, set $\mathbf{aux}_i \coloneqq (i, \boldsymbol{\rho}^{(i)})$ and output $(\mathsf{cm}_i, \mathbf{aux}_i)$.
  6. Run $(\widetilde{\pi}_i, \boldsymbol{\sigma}^{(i)}) \leftarrow \mathfrak{R}_i^{\widetilde{\mathbb{P}}}\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i]}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j<i}, \boldsymbol{\rho}^{(i)}\big)$.
  7. Set $\mathbf{aux}_i \coloneqq \big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i]}, (r_j)_{j<i}, (\widetilde{\pi}_j)_{j\in[i]}, \boldsymbol{\sigma}^{(i)}\big)$.
  8. Output $(\widetilde{\pi}_i, \mathbf{aux}_i)$.

- $\mathcal{V}_{i^*}^{(\widetilde{\pi}_j)_{j\in[i^*]}}(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j, \mathsf{ans}_j, \mathsf{pf}_j)_{j\in[i^*+1,\mathsf{k}]}, (r_j)_{j\in[\mathsf{k}]})$:
  1. Check that $\mathbf{V}^{((\widetilde{\pi}_j)_{j\in[i^*]}, (\mathsf{ans}_i)_{j\in[i^*+1,\mathsf{k}]})}(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}) = 1$.

2. Check that $\mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}_j, \mathsf{ans}_j, \mathsf{pf}_j) = 1$ for every $j \in [i^* + 1, \mathsf{k}]$.
3. Output 1 if both checks succeed; output 0 otherwise.

The interaction $b \leftarrow \langle \widetilde{\mathcal{P}}_{i^*}(\boldsymbol{\tau}^{(0)}, \mathbf{aux}_0), \mathcal{V}_{i^*}(\mathsf{pp}, \mathbb{x}) \rangle$ then corresponds to the following experiment:

$$\left[\begin{array}{l} r_0 \coloneqq () \\ \text{For } i \in [i^*] : \\ \quad (\widetilde{\pi}_i, \mathbf{aux}_i) \leftarrow \widetilde{\mathcal{P}}_{i^*}(r_{i-1}, \mathbf{aux}_{i-1}) \\ \quad r_i \leftarrow \{0,1\}^{\mathsf{r}_i} \\ \text{For } i \in [i^* + 1, \mathsf{k}] : \\ \quad (\mathsf{cm}_i, \mathbf{aux}_i) \leftarrow \widetilde{\mathcal{P}}_{i^*}(r_{i-1}, \mathbf{aux}_{i-1}) \\ \quad r_i \leftarrow \{0,1\}^{\mathsf{r}_i} \\ \big((\mathsf{ans}_i, \mathsf{pf}_i)\big)_{i \in [i^*+1, \mathsf{k}]} \leftarrow \widetilde{\mathcal{P}}_{i^*}(r_{\mathsf{k}}, \mathbf{aux}_{\mathsf{k}}) \\ b \leftarrow \mathcal{V}_{i^*}^{(\widetilde{\pi}_j)_{j \in [i^*]}}(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j, \mathsf{ans}_j, \mathsf{pf}_j)_{j \in [i^*+1, \mathsf{k}]}, (r_j)_{j \in [\mathsf{k}]}) \end{array}\right]$$

Remark 5.7 (Notation for states)**.** The argument adversary $\widetilde{\mathbb{P}}$ takes as input, in its setup stage, the public parameter pp and an auxiliary input state $\boldsymbol{\xi}$; then, for each round $i \in [\mathsf{k} + 1]$, it takes an input state $\boldsymbol{\rho}^{(i-1)}$ (when $i = 1$ the state $\boldsymbol{\rho}^{(0)}$ is that output by the setup stage), finishes its computation, and produces an output state $\boldsymbol{\rho}^{(i)}$ for the next round.

For each $i \in [\mathsf{k}]$, our reductor $\mathfrak{R}_i$ rewinds $\widetilde{\mathbb{P}}(\boldsymbol{\rho}^{(i)})$ using the rewinding process in Construction 3.17. Since rewinding disturbs the initial state $\boldsymbol{\rho}^{(i)}$, we use $\boldsymbol{\sigma}^{(i)}$ to denote the state output by the reductor $\mathfrak{R}_i$, and we use $\boldsymbol{\sigma}^{(i,j)}$ to denote the state after j rewinds in S_i.

The state of the hybrid prover at the end of a round i may be either $\boldsymbol{\rho}^{(i)}$ or $\boldsymbol{\sigma}^{(i)}$. Hence, for notational simplicity, we use $\boldsymbol{\tau}^{(i)}$ to represent the state of the hybrid prover $\widetilde{\mathcal{P}}_{i^*}$ at the end of round i.

Remark 5.8 (Notation for black-box access)**.** Throughout the section, for notation simplicity we omit the black-box access to the argument prover $\widetilde{\mathbb{P}}$ except in Construction 5.4, Construction 5.5 and Construction 5.6. For example, we denote the hybrid prover $\widetilde{\mathcal{P}}_{i^*}^{\widetilde{\mathbb{P}}}$ by $\widetilde{\mathcal{P}}_{i^*}$.

We define the value of a hybrid experiment to be the probability that the hybrid prover $\widetilde{\mathcal{P}}_{i^*}$ convinces the hybrid verifier $\mathcal{V}_{i^*}$, that is,

$$\mathsf{val}\,(\mathsf{H}_{i^*}) \coloneqq \Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n & \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ \wedge\, b = 1 & (\mathbb{x}, \mathbf{aux}_0) \leftarrow \widetilde{\mathcal{P}}_{i^*}(\mathsf{pp}, \boldsymbol{\xi}) \\ & b \leftarrow \langle \widetilde{\mathcal{P}}_{i^*}(\mathbf{aux}_0), \mathcal{V}_{i^*}(\mathsf{pp}, \mathbb{x}) \rangle \end{array}\right] .$$

By Construction 5.6 and the definition of the value of a hybrid, we have

$$\mathsf{val}\,(\mathsf{H}_0) = \Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n & \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ \wedge\, b = 1 & (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ & b \leftarrow \big\langle \widetilde{\mathbb{P}}(\mathsf{aux}), \mathbb{V}(\mathsf{pp}, \mathbb{x}) \big\rangle \end{array}\right]$$

and

$$\mathsf{val}\,(\mathsf{H}_\mathsf{k}) = \Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n & (\mathbb{x}, \mathbf{aux}) \leftarrow \widetilde{\mathbf{P}}(\boldsymbol{\xi}) \\ \wedge\, b = 1 & b \leftarrow \big\langle \widetilde{\mathbf{P}}(\mathbf{aux}), \mathbf{V}(\mathbb{x}) \big\rangle \end{array}\right] .$$

Hence, to prove Lemma 5.1, it suffices to show that for every $i^* \in [\mathsf{k}]$,

$$\mathsf{val}\,(\mathsf{H}_{i^*-1}) \le \mathsf{val}\,(\mathsf{H}_{i^*}) + \mathsf{L}_{i^*} \cdot (\mathsf{q}_{i^*} + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) + \epsilon/\mathsf{k} \ . \tag{2}$$

It then follows that

$$\mathsf{val}\left(\mathsf{H}_0\right) \leq \mathsf{val}\left(\mathsf{H}_\mathsf{k}\right) + \sum_{i\in[\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \ ,$$

which completes the proof.

The proof of Eq. 2 proceeds via a sequence of "sub-hybrids" $\mathsf{H}_{i^*,0}, \mathsf{H}_{i^*,1}, \mathsf{H}_{i^*,2}, \mathsf{H}_{i^*,3}$, defined as follows.

- $\mathsf{H}_{i^*,0} = (\widetilde{\mathcal{P}}_{i^*,0}, \mathcal{V}_{i^*,0})$. To simplify notation, we align subscripts and define $(\widetilde{\mathcal{P}}_{i^*,0}, \mathcal{V}_{i^*,0}) \coloneqq (\widetilde{\mathcal{P}}_{i^*-1}, \mathcal{V}_{i^*-1})$. In particular, $\mathsf{H}_{i^*,0}$ is the (i^*-1)-th hybrid H_{i^*-1}.
- $\mathsf{H}_{i^*,1} = (\widetilde{\mathcal{P}}_{i^*,1}, \mathcal{V}_{i^*,1})$. $\widetilde{\mathcal{P}}_{i^*,1}$ is obtained by modifying $\widetilde{\mathcal{P}}_{i^*-1}$ to run a "non-measuring" reductor $\mathfrak{R}_{i^*,1}$ (only) in step i^*, and $\mathcal{V}_{i^*,1} \coloneqq \mathcal{V}_{i^*-1}$.
- $\mathsf{H}_{i^*,2} = (\widetilde{\mathcal{P}}_{i^*,2}, \mathcal{V}_{i^*,2})$. $\widetilde{\mathcal{P}}_{i^*,2}$ is obtained by modifying $\widetilde{\mathcal{P}}_{i^*}$ to output a commitment (instead of the proof string obtained by the reductor) at round i^*, and $\mathcal{V}_{i^*,2} \coloneqq \mathcal{V}_{i^*-1}$.
- $\mathsf{H}_{i^*,3} = (\widetilde{\mathcal{P}}_{i^*,3}, \mathcal{V}_{i^*,3})$. To simplify notation, we align subscripts and define $(\widetilde{\mathcal{P}}_{i^*,3}, \mathcal{V}_{i^*,3}) \coloneqq (\widetilde{\mathcal{P}}_{i^*}, \mathcal{V}_{i^*})$. In particular, $\mathsf{H}_{i^*,3}$ is the i^*-th hybrid H_{i^*}.

We bound the difference in success probability between adjacent hybrids via the following three claims. The first claim, relating $\mathsf{H}_{i^*-1} = \mathsf{H}_{i^*,0}$ and $\mathsf{H}_{i^*,1}$, states that additional rewinding does not significantly reduce the success probability of the argument adversary, and is proved using Claim 3.18.

Claim 5.9. $\mathsf{val}\left(\mathsf{H}_{i^*,0}\right) \leq \mathsf{val}\left(\mathsf{H}_{i^*,1}\right) + \epsilon/2\mathsf{k}$ for every $i^* \in [\mathsf{k}]$.

The second claim, relating $\mathsf{H}_{i^*,1}$ and $\mathsf{H}_{i^*,2}$, states that the measurement of the prover responses during the rewinding process does not noticeably disturb the success probability of the argument adversary. This is due to the collapsing property of the IBCS protocol as established in Lemma 5.14.

Claim 5.10. $\mathsf{val}\left(\mathsf{H}_{i^*,1}\right) \leq \mathsf{val}\left(\mathsf{H}_{i^*,2}\right) + \mathsf{L}_{i^*} \cdot \mathsf{q}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}})$ for every $i^* \in [\mathsf{k}]$.

The third and final claim relates $\mathsf{H}_{i^*,2}$ and $\mathsf{H}_{i^*,3} = \mathsf{H}_{i^*}$, and requires some new ideas. The difference between these two hybrids is that in $\mathsf{H}_{i^*,2}$ the verifier obtains its query answers in round i^* from the commitment openings, whereas in $\mathsf{H}_{i^*,3}$, the verifier obtains those answers from the extracted oracle. To prove that the success probability is maintained, we "couple" the two hybrids, and bound the probability that the verifier gets *different* answers in each hybrid.

This can happen for two reasons: either the adversary opens an already-seen location to a different value, or the verifier queries a "missing" position that has not yet been added to the oracle. The former case clearly violates position binding; bounding the latter is more involved. In the classical setting, the bound is proven roughly as follows [CDGS23]. Let δ_i be the probability that the i-th entry in the oracle is queried by the verifier and correctly revealed by the prover; the probability that the i-th entry is missing is then $(1-\delta_i)^T\delta_i \leq 1/T$, where T is the number of rewinds.

In the quantum setting, the prover's state changes after each rewind, and so the value of δ_i may increase or decrease during the procedure. To address this, we observe that regardless of how the prover's strategy changes, any valid rewind in which there is a missing position will fill in that position in the oracle. Hence at most L_{i^*} rewinds can have missing positions. It follows that if we stop at a *random* rewind between 0 and T the probability that there is a missing position is at most L_{i^*}/T, which is the same as the classical bound in [CDGS23]. Formalizing this intuition is delicate because the experiments we are coupling contain incompatible measurements.

Claim 5.11. $\mathsf{val}\left(\mathsf{H}_{i^*,2}\right) \leq \mathsf{val}\left(\mathsf{H}_{i^*,3}\right) + 4 \cdot \mathsf{L}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) + \epsilon/2\mathsf{k}$ for every $i^* \in [\mathsf{k}]$.

For notational simplicity, we define two subroutines, Before_{i^*} and After_{i^*}, which capture the common interaction between two successive hybrid interactions (i.e., $\langle \widetilde{\mathcal{P}}_{i^*-1}, \mathcal{V}_{i^*-1} \rangle$ and $\langle \widetilde{\mathcal{P}}_{i^*}, \mathcal{V}_{i^*} \rangle$).

Construction 5.12. Before_{i^*} is an algorithm that simulates the interaction $\langle \widetilde{\mathcal{P}}_i, \mathcal{V}_i \rangle$, for $i \in \{i^*-1, i^*\}$, up to the commitment generation in round i^* (where $\langle \widetilde{\mathcal{P}}_{i^*-1}, \mathcal{V}_{i^*-1} \rangle$ and $\langle \widetilde{\mathcal{P}}_{i^*}, \mathcal{V}_{i^*} \rangle$ both behave as their IOP counterparts) and outputs the partial transcript along with the prover's internal state.

$\mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})$:
1. Run $(\mathbb{x}, \mathbf{aux}_0) \leftarrow \widetilde{\mathcal{P}}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})$ and set $r_0 := ()$.
2. For $i \in [i^* - 1]$:
   (a) Run $(\widetilde{\pi}_i, \mathbf{aux}_i) \leftarrow \widetilde{\mathcal{P}}_{i^*}(r_{i-1}, \mathbf{aux}_{i-1})$.
   (b) Sample $r_i \leftarrow \{0,1\}^{\mathsf{r}_i}$.
3. Parse $\mathbf{aux}_{i^*-1}$ as $\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j<i^*}, (r_j)_{j<i^*-1}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\tau}^{(i-1)}\big)$.
4. Run $(\mathsf{cm}_{i^*}, \boldsymbol{\rho}^{(i^*)}) \leftarrow \widetilde{\mathbb{P}}(r_{i^*-1}, \boldsymbol{\tau}^{(i^*-1)})$.
5. Output $(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)})$.

After_{i^*} is an algorithm which, given a prover state $\boldsymbol{\tau}^{(i^*)}$, finishes the simulation of the interaction $\langle \widetilde{\mathcal{P}}_i, \mathcal{V}_i \rangle$, for $i \in \{i^*-1, i^*\}$, from round i^*+1 round to $\mathsf{k}+1$ (where $\langle \widetilde{\mathcal{P}}_{i^*-1}, \mathcal{V}_{i^*-1} \rangle$ and $\langle \widetilde{\mathcal{P}}_{i^*}, \mathcal{V}_{i^*} \rangle$ both behave as their argument counterparts) and outputs the last-round *argument* prover message.

$\mathsf{After}_{i^*}(\boldsymbol{\tau}^{(i^*)})$:
1. Sample $r_{i^*} \leftarrow \{0,1\}^{\mathsf{r}_{i^*}}$.
2. Set $\mathbf{aux}_{i^*} := (i^*, \boldsymbol{\tau}^{(i^*)})$.
3. For $i \in [i^*+1, \mathsf{k}]$:
   (a) Run $(\mathsf{cm}_i, \mathbf{aux}_i) \leftarrow \widetilde{\mathcal{P}}_{i^*}(r_{i-1}, \mathbf{aux}_{i-1})$.
   (b) Sample $r_i \leftarrow \{0,1\}^{\mathsf{r}_i}$.
4. Parse $\mathbf{aux}_{\mathsf{k}}$ as $(\mathsf{k}, \boldsymbol{\tau}^{(\mathsf{k})})$.
5. Run $\big((\mathsf{ans}_i, \mathsf{pf}_i)\big)_{i\in[\mathsf{k}]} \leftarrow \widetilde{\mathbb{P}}(r_{\mathsf{k}}, \boldsymbol{\tau}^{(\mathsf{k})})$.
6. Output $\big((\mathsf{cm}_i)_{i\in[i^*+1,\mathsf{k}]}, (\mathsf{ans}_i, \mathsf{pf}_i)_{i\in[\mathsf{k}]}, (r_i)_{i\in[i^*,\mathsf{k}]}\big)$.

Observe that $(\mathbb{x}, b)$ is identically distributed in

$$\left[\begin{array}{l} \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ (\mathbb{x}, \mathbf{aux}) \leftarrow \widetilde{\mathcal{P}}_{i^*}(\mathsf{pp}, \boldsymbol{\xi}) \\ b \leftarrow \langle \widetilde{\mathcal{P}}_{i^*}(\mathbf{aux}), \mathcal{V}_{i^*}(\mathsf{pp}, \mathbb{x}) \rangle \end{array}\right]$$

and

$$\mathsf{ReductorExp} := \left[\begin{array}{l} \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)}) \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi}) \\ (\widetilde{\pi}_{i^*}, \boldsymbol{\sigma}^{(i^*)}) \leftarrow \mathfrak{R}_{i^*}\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)}\big) \\ \big((\mathsf{cm}_j)_{j\in[i^*+1,\mathsf{k}]}, (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[\mathsf{k}]}, (r_j)_{j\in[i^*,\mathsf{k}]}\big) \leftarrow \mathsf{After}_{i^*}(\boldsymbol{\sigma}^{(i^*)}) \\ b \leftarrow \mathcal{V}_{i^*}^{(\widetilde{\pi}_j)_{j\in[i^*]}}(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j, \mathsf{ans}_j, \mathsf{pf}_j)_{j\in[i^*+1,\mathsf{k}]}, (r_j)_{j\in[\mathsf{k}]}) \end{array}\right] \tag{3}$$

for all $i^* \in [0, \mathsf{k}]$.

Note, moreover, that the only difference between $\langle \widetilde{\mathcal{P}}_{i^*-1}, \mathcal{V}_{i^*-1} \rangle$ and $\langle \widetilde{\mathcal{P}}_{i^*}, \mathcal{V}_{i^*} \rangle$ is the latter's (i) inclusion of an execution $\mathfrak{R}_{i^*}$ between Before_{i^*} and After_{i^*}; and (ii) replacing of $\mathcal{V}_{i^*-1}$ by $\mathcal{V}_{i^*}$ (with appropriate inputs).

5.3 Proof of Claim 5.9

Fix $i^* \in [\mathsf{k}]$. Let $\mathfrak{R}_{i^*,1}$ be the reductor that runs the modified sampler $S_{i^*,1}$ (instead of S_{i^*}) which corresponds to S_{i^*} with Step 4e removed. (Note that the proof string $\widetilde{\pi}_{i^*}$ output by $S_{i^*,1}$ is unmodified from its initial value of $\bot^{\mathsf{L}_{i^*}}$.) Define $\widetilde{\mathcal{P}}_{i^*,1}$ by replacing Step 5 of $\widetilde{\mathcal{P}}_{i^*-1}$ with the following:

5. If $i \geq i^*$:
   (a) If $i = i^*$, run $(\widetilde{\pi}_{i^*}, \hat{\boldsymbol{\sigma}}^{(i^*)}) \leftarrow \mathfrak{R}_{i^*,1}\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)}\big)$ and set $\mathbf{aux}_i \coloneqq (i, \hat{\boldsymbol{\sigma}}^{(i)})$.
   (b) If $i > i^*$, set $\mathbf{aux}_i \coloneqq (i, \boldsymbol{\rho}^{(i)})$.
   (c) Output $(\mathsf{cm}_i, \mathbf{aux}_i)$.

Fix a prover state $\boldsymbol{\tau}^{(i^*)}$ and game $\mathsf{G}_{i^*} = \mathsf{G}\big(\mathsf{pp}, \mathbb{x}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \mathsf{cm}_{i^*}\big)$. By Definition 3.15 and Construction 5.2, the value of the quantum strategy $(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\tau}^{(i^*)})$ in G_{i^*} is

$$\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\tau}^{(i^*)})$$
$$= \Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & \\ \wedge\, \mathcal{V}_{i^*-1}^{(\widetilde{\pi}_j)_{j<i^*}}(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j, \mathsf{ans}_j, \mathsf{pf}_j)_{j\in[i^*,\mathsf{k}]}, (r_j)_{j\in[\mathsf{k}]}) = 1 & \left(\begin{array}{c}(\mathsf{cm}_j)_{j\in[i^*+1,\mathsf{k}]},\\ (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[\mathsf{k}]}, (r_j)_{j\in[i^*,\mathsf{k}]}\end{array}\right) \leftarrow \mathsf{After}_{i^*}(\boldsymbol{\tau}^{(i^*)}) \end{array}\right]$$

Similarly, let $\hat{\boldsymbol{\sigma}}^{(i^*)}$ be the internal state of hybrid prover $\widetilde{\mathcal{P}}_{i^*,1}$ after sending its i^*-th round message in $\big\langle \widetilde{\mathcal{P}}_{i^*,1}, \mathcal{V}_{i^*,1}\big\rangle$ (i.e., after running the non-measuring reductor and sending the i^*-th commitment). Notice that the prover messages of $\widetilde{\mathcal{P}}_{i^*,1}$ are the same as those of $\widetilde{\mathcal{P}}_{i^*,0}$.

Define the experiment $\mathsf{Before}_{i^*,1}$ as follows:

$\mathsf{Before}_{i^*,1}(\mathsf{pp}, \boldsymbol{\xi})$:
1. Run $(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)}) \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})$.
2. Run $(\widetilde{\pi}_{i^*}, \hat{\boldsymbol{\sigma}}^{(i^*)}) \leftarrow \mathfrak{R}_{i^*,1}\big(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)}\big)$.
3. Output $(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \hat{\boldsymbol{\sigma}}^{(i^*)})$.

By Claim 3.18 and the law of total probability for conditional probability,

$$\begin{aligned}
&\mathsf{val}\,(\mathsf{H}_{i^*,1})\\
&= \mathbb{E}\left[\omega_{\mathsf{G}_{i^*}}(\mathsf{Strat}_{i^*}, \hat{\boldsymbol{\sigma}}^{(i^*)}) \;\middle|\; \begin{array}{l}\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)\\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \hat{\boldsymbol{\sigma}}^{(i^*)}) \leftarrow \mathsf{Before}_{i^*,1}(\mathsf{pp}, \boldsymbol{\xi})\end{array}\right]\\
&\geq \mathbb{E}\left[\omega_{\mathsf{G}_{i^*}}(\mathsf{Strat}_{i^*}, \boldsymbol{\tau}^{(i^*)}) \;\middle|\; \begin{array}{l}\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)\\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)}) \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})\end{array}\right] - \epsilon/2\mathsf{k}\\
&= \mathsf{val}\,(\mathsf{H}_{i^*,0}) - \epsilon/2\mathsf{k}\ .
\end{aligned}$$

5.4 Proof of Claim 5.10

Fix $i^* \in [\mathsf{k}]$. Define $\widetilde{\mathcal{P}}_{i^*,2}$ by outputting $(\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[i^*,\mathsf{k}]}$ in Step 3 of $\widetilde{\mathcal{P}}_{i^*}$ (i.e., adding $(\mathsf{ans}_{i^*}, \mathsf{pf}_{i^*})$ to the output) and replacing Step 8 with the following:

8. If $i = i^*$, output $\big(\mathsf{cm}_{i^*}, (\boldsymbol{\rho}^{(i^*)}, i^*+1)\big)$. Otherwise, output $(\widetilde{\pi}_i, \mathbf{aux}_i)$.

For every $\ell \in [0, \mathsf{L}_{i^*}]$, let $S_{i^*,2,\ell}$ be the modified sampler that measures no more than $\ell \leq \mathsf{L}_{i^*}$ answers; that is, S_{i^*} with Step 4(e)iii of Construction 5.4 replaced by

4. (e) iii. If $c < \ell$ and $q \notin Q^{(\mathsf{ANS})}$, measure $\mathcal{A}_{i^*}$ in the computational basis to obtain response ans'_ι, add ans'_ι to ANS and increment $c := c + 1$. Skip to Step 4f.

Define $\mathsf{Before}_{i^*,2}(\ell, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})$ as be the algorithm that simulates $\langle \widetilde{\mathcal{P}}_{i^*,1}, \mathcal{V}_{i^*,1} \rangle$ up to the commitment in round i^*, then runs $S_{i^*,2,\ell}$ (which caps the number of samples to a maximum of ℓ) for t iterations:

$\mathsf{Before}_{i^*,2}(\ell, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})$:
1. Run $(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)}) \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})$.
2. Run $(\mathsf{ANS}, \boldsymbol{\mu}^{(\ell,\mathsf{t})}) \leftarrow S_{i^*,2,\ell}(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \mathsf{t}, \boldsymbol{\rho}^{(i^*)})$.
3. Output $(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\mu}^{(\ell,\mathsf{t})})$.

We will use the following key claim, whose proof is deferred to the section, to conclude Claim 5.10.

Claim 5.13. For every $\mathsf{t} \in [T]$ and $\ell \in [\mathsf{L}_{i^*}]$,

$$\begin{aligned}
&\mathbb{E}\left[\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell-1,\mathsf{t})}) \;\middle|\; \begin{array}{l}\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)\\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\mu}^{(\ell-1,\mathsf{t})}) \leftarrow \mathsf{Before}_{i^*,2}(\ell-1, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})\end{array}\right]\\
&\le \mathbb{E}\left[\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell,\mathsf{t})}) \;\middle|\; \begin{array}{l}\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)\\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\mu}^{(\ell,\mathsf{t})}) \leftarrow \mathsf{Before}_{i^*,2}(\ell, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})\end{array}\right]\\
&\quad + \mathsf{q}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) \;,
\end{aligned}$$

where $t_{\mathsf{VCCollapse}} = O(t_{\mathsf{Rewind}}(\frac{2\mathsf{k}\cdot\mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}} + t_{\mathbb{V}})$.

With Claim 5.13, we deduce (for every $\mathsf{t} \in [T]$) that

$$\begin{aligned}
&\mathbb{E}\left[\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(0,\mathsf{t})}) \;\middle|\; \begin{array}{l}\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)\\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\mu}^{(0,\mathsf{t})}) \leftarrow \mathsf{Before}_{i^*,2}(0, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})\end{array}\right]\\
&\le \mathbb{E}\left[\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\mathsf{L}_{i^*},\mathsf{t})}) \;\middle|\; \begin{array}{l}\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)\\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\mu}^{(\mathsf{L}_{i^*},\mathsf{t})}) \leftarrow \mathsf{Before}_{i^*,2}(\ell, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})\end{array}\right]\\
&\quad + \mathsf{L}_{i^*} \cdot \mathsf{q}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) \;.
\end{aligned}$$

Finally, we obtain the desired bound as follows:

$$\begin{aligned}
&\mathsf{val}\,(\mathsf{H}_{i^*,1})\\
&= \mathbb{E}\left[\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(0,\mathsf{t})}) \;\middle|\; \begin{array}{l}\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)\\ \mathsf{t} \leftarrow [T]\\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\mu}^{(0,\mathsf{t})}) \leftarrow \mathsf{Before}_{i^*,2}(0, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})\end{array}\right]\\
&= \frac{1}{T} \cdot \sum_{\mathsf{t}\in[T]} \mathbb{E}\left[\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(0,\mathsf{t})}) \;\middle|\; \begin{array}{l}\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)\\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\mu}^{(0,\mathsf{t})}) \leftarrow \mathsf{Before}_{i^*,2}(0, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})\end{array}\right]\\
&\le \frac{1}{T} \cdot \sum_{\mathsf{t}\in[T]} \mathbb{E}\left[\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\mathsf{L}_{i^*},\mathsf{t})}) \;\middle|\; \begin{array}{l}\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)\\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\mu}^{(\mathsf{L}_{i^*},\mathsf{t})}) \leftarrow \mathsf{Before}_{i^*,2}(\mathsf{L}_{i^*}, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})\end{array}\right]\\
&\quad + \mathsf{L}_{i^*} \cdot \mathsf{q}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}})\\
&= \mathsf{val}\,(\mathsf{H}_{i^*,2}) + \mathsf{L}_{i^*} \cdot \mathsf{q}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) \;.
\end{aligned}$$

The equalities follow from the fact that $\mathfrak{R}_{i^*}$ samples $\mathsf{t} \leftarrow [T]$; and that $\mathsf{Before}_{i^*,2}(0, \mathsf{t})$ and $\mathsf{Before}_{i^*,2}(\mathsf{L}_{i^*}, \mathsf{t})$ simulate the non-measuring reductor $\mathfrak{R}_{i^*,1}$ and the "regular" reductor $\mathfrak{R}_{i^*}$, respectively. The inequality is the one shown above (which follows from Claim 5.13 and Lemma 5.14).

Proof of Claim 5.13. We define a function g as follows.

$$g\big(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, (\widetilde{\pi}_j)_{j<i^*}, \mathsf{ans}_{i^*}\big) \coloneqq \mathsf{ans}_{i^*}(Q_{i^*}) \ ,$$

where $Q_{i^*} \leftarrow \mathsf{QuerySet}^{\mathsf{ans}_{i^*}}(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, (\widetilde{\pi}_j)_{j<i^*})$.

Lemma 5.14. *For every circuit size bound $t_{\mathsf{pcol}} \in \mathbb{N}$, auxiliary information $\mathbf{aux}$ (classical or quantum), and t_{pcol}-size quantum circuit A_{pcol},*

$$\begin{aligned}&\left|\begin{array}{l}\Pr\left[\mathsf{PCollapseExp}(0, g, A_{\mathsf{pcol}}, \mathbf{aux}) = 1\right] \\ -\Pr\left[\mathsf{PCollapseExp}(1, g, A_{\mathsf{pcol}}, \mathbf{aux}) = 1\right]\end{array}\right| \\ &\leq \mathsf{q}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) \quad \textit{where} \quad t_{\mathsf{VCCollapse}} = O(t_{\mathsf{pcol}} + t_{\mathbb{V}}) \ .\end{aligned}$$

For $b \in \{0,1\}$, define the partially collapsing experiment as follows.

$\mathsf{PCollapseExp}(b, g, A_{\mathsf{pcol}}, \mathbf{aux})$*:*

1. *Sample public parameters* $\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)$.
2. *Run* $\big(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, \mathsf{cm}_{i^*}, (\widetilde{\pi}_j)_{j<i^*}, (\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}\big) \leftarrow A_{\mathsf{pcol}}(\mathsf{pp}, \mathbf{aux})$ *to obtain (classically) an instance $\mathbb{x}$, all verifier messages $(r_j)_{j\in[\mathsf{k}]}$, prover message cm_{i^*} at the i^*-th round, IOP proofs $(\widetilde{\pi}_j)_{j<i^*}$, and registers $(\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}$ containing superpositions of prover messages for rounds i^*+1 to $\mathsf{k}+1$. In particular, $\mathcal{Z}_j$ contains cm_j, and the answer-opening pairs are contained in register $\mathcal{Z}_{\mathsf{k}+1} = \otimes_{j\in[\mathsf{k}]}\mathcal{Z}_{\mathsf{k}+1,j}$, where $\mathcal{Z}_{\mathsf{k}+1,j} \coloneqq (\mathcal{A}_j \otimes \mathcal{O}_j)$ contains a superposition of $(\mathsf{ans}_j, \mathsf{pf}_j)$. Output* $\mathtt{Abort}$ *if $|\mathbb{x}| > n$.*
3. *Measure* $\mathcal{V}_{i^*-1}^{(\widetilde{\pi}_j)_{j<i^*}}\big(\mathsf{pp}, \mathbb{x}, \big((\mathsf{cm}_{i^*}, (\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}]}), \otimes_{j\in[i^*,\mathsf{k}]}\mathcal{Z}_{\mathsf{k}+1,j}\big), (r_j)_{j\in[\mathsf{k}]}\big)$ *and output* $\mathtt{Abort}$ *if the outcome is* 0.
4. *If $b = 1$, measure $g\big(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, \big((\widetilde{\pi}_j)_{j<i^*}, \mathcal{A}_{i^*}\big)\big)$ and discard the outcome.*
5. *Output* $b' \leftarrow A_{\mathsf{pcol}}\big((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}\big)$.

Before we prove Lemma 5.14, we first prove Claim 5.13 using Lemma 5.14. That is, we aim to show that for every $\mathsf{t} \in [T]$ and $\ell \in [\mathsf{L}_{i^*}]$, the difference in values $\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell-1,\mathsf{t})})$ and $\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell,\mathsf{t})})$ is at most the probability that A_{pcol} outputs 1, which is bounded by $\mathsf{q}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}})$.

For $\mathbf{aux} \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})$, we define an adversary $A_{\mathsf{pcol}} \coloneqq (A_{\mathsf{pcol}}^{(1)}, A_{\mathsf{pcol}}^{(2)})$ that runs $S_{i^*,2,\ell}$ and then After_{i^*} except at the places we highlight in blue. It also breaks the sampler $S_{i^*,2,\ell}$ into two parts: before the ℓ-th measurement, A_{pcol} stops and outputs the answer registers $(\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}$; upon receiving the answer registers back, A_{pcol} finishes simulating $S_{i^*,2,\ell}$, runs After_{i^*}, and in the end outputs the hybrid verifier $\mathcal{V}_{i^*-1}$'s decision. The construction is as follows:

- $A_{\mathsf{pcol}}^{(1)}(\mathsf{pp}, \mathbf{aux})$:
  1. Parse $\mathbf{aux}$ as $(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)})$.
  2. Run sampler $S_{i^*,2,\ell}$ for at most ℓ measurements at Step 4(e)iii of Construction 5.4.
     (a) Initialize $\mathsf{ANS} \coloneqq \varnothing$ and $c \coloneqq 0$.
     (b) Let $\boldsymbol{\sigma}^{(i^*,0)} \coloneqq \boldsymbol{\rho}^{(i^*)}$ and $\boldsymbol{r} \coloneqq (r_j)_{j<i^*}$.
     (c) Run $\mathbf{aux}_0 \leftarrow \mathsf{RewindInit}_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \mathsf{t}, \epsilon/2\mathsf{k}, \boldsymbol{\sigma}^{(i^*,0)})$.
     (d) For $\iota \in [\mathsf{t}]$:
        i. Sample IOP verifier randomness: $\boldsymbol{r}' \coloneqq (r'_j)_{j\in[i^*,\mathsf{k}]} \leftarrow \{0,1\}^{\mathsf{r}_{i^*}+\cdots+\mathsf{r}_{\mathsf{k}}}$.
        ii. Run $\big((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, \mathbf{aux}'_\iota\big) \leftarrow \mathsf{RewindStrat}_{\mathsf{G}_{i^*}}(\boldsymbol{r}', \mathbf{aux}_{\iota-1})$.
        iii. Measure $b \leftarrow f_{i^*}(\boldsymbol{r}', ((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}]}, \mathcal{Z}_{\mathsf{k}+1}))$. If $b = 0$, skip to Step 2(d)vi.

iv. Define $Q^{(\mathsf{ANS})} := \cup_{\mathsf{ans}\in\mathsf{ANS}}\mathsf{supp}(\mathsf{ans})$ and initialize $\widetilde{\pi} := \bot^{\mathsf{L}_{i^*}}$.

v. For $m \in [\mathsf{q}_{i^*}]$:

A. Compute $q \leftarrow \mathsf{Query}^{\widetilde{\pi}}(\mathbb{x}, (\boldsymbol{r}, \boldsymbol{r}'), (\widetilde{\pi}_j)_{j<i^*}, m)$.

B. If $q \in Q^{(\mathsf{ANS})}$, let j be minimal such that $\mathsf{ans}_j \in \mathsf{ANS}$ and $q \in \mathsf{supp}(\mathsf{ans}_j)$. Set $\widetilde{\pi}(q) := \mathsf{ans}_j(q)$.

C. If $c < \ell - 1$ and $q \notin Q^{(\mathsf{ANS})}$, measure $\mathcal{A}_{i^*}$ in the computational basis to obtain response ans'_ι, add ans'_ι to ANS and increment $c := c + 1$. Skip to Step 2(d)vi.

D. Else if $c < \ell$ and $q \notin Q^{(\mathsf{ANS})}$, output $\left(\mathbb{x}, (\boldsymbol{r}, \boldsymbol{r}'), \mathsf{cm}_{i^*}, (\widetilde{\pi}_j)_{j<i^*}, (\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}\right)$ and pass out internal state $(\mathbf{aux}'_\iota, \iota)$ to $A^{(2)}_{\mathsf{pcol}}$.

vi. Run $\mathbf{aux}_\iota \leftarrow \mathsf{RewindRepair}_{\mathsf{G}_{i^*}}((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, b, \mathbf{aux}'_\iota)$.

3. Output $\left(\mathbb{x}, (\boldsymbol{r}, \boldsymbol{r}'), \mathsf{cm}_{i^*}, (\widetilde{\pi}_j)_{j<i^*}, (\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}\right)$ and pass out internal state $(\mathbf{aux}_\mathsf{t}, \mathsf{t}+1)$ to $A^{(2)}_{\mathsf{pcol}}$.

- $A^{(2)}_{\mathsf{pcol}}((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, \mathbf{aux}, \iota')$:

1. If $\iota' = \mathsf{t}+1$, set $\mathbf{aux}_\mathsf{t} := \mathbf{aux}$ and skip to Step 3. Otherwise, set $\mathbf{aux}'_{\iota'} := \mathbf{aux}$.
2. Run the remaining iterations of $S_{i^*,2,\ell}$ without any measurement at Step 4(e)iii of Construction 5.4:
   (a) Run $\mathbf{aux}_{\iota'} \leftarrow \mathsf{RewindRepair}_{\mathsf{G}_{i^*}}((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, b, \mathbf{aux}'_{\iota'})$.
   (b) For $\iota \in [\iota'+1, \mathsf{t}]$:
      i. Sample IOP verifier randomness: $\boldsymbol{r}' := (r'_j)_{j\in[i^*,\mathsf{k}]} \leftarrow \{0,1\}^{\mathsf{r}_{i^*}+\cdots+\mathsf{r}_\mathsf{k}}$.
      ii. Run $\left((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, \mathbf{aux}'_\iota\right) \leftarrow \mathsf{RewindStrat}_{\mathsf{G}_{i^*}}(\boldsymbol{r}', \mathbf{aux}_{\iota-1})$.
      iii. Measure $b \leftarrow f_{i^*}(\boldsymbol{r}', ((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}]}, \mathcal{Z}_{\mathsf{k}+1}))$.
      iv. Run $\mathbf{aux}_\iota \leftarrow \mathsf{RewindRepair}_{\mathsf{G}_{i^*}}((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, b, \mathbf{aux}'_\iota)$.
3. Parse $\mathbf{aux}_\mathsf{t}$ as $(\widetilde{\mathbb{P}}_{>i^*}, \gamma, \delta, \mathsf{t}, p_\mathsf{t}, \boldsymbol{\sigma}^{(i^*,\mathsf{t})})$.
4. Run $\left((\mathsf{cm}_j)_{j\in[i^*+1,\mathsf{k}]}, (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[\mathsf{k}]}, (r_j)_{j\in[i^*,\mathsf{k}]}\right) \leftarrow \mathsf{After}_{i^*}(\boldsymbol{\sigma}^{(i^*,\mathsf{t})})$.
5. Output $\mathcal{V}^{(\widetilde{\pi}_j)_{j<i^*}}_{i^*-1}(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j, \mathsf{ans}_j, \mathsf{pf}_j)_{j\in[i^*,\mathsf{k}]}, (r_j)_{j\in[\mathsf{k}]}) \wedge |\mathbb{x}| \le n$.

Observe that the adversary A_{pcol} runs the sampler $S_{i^*,2,\ell}$, except that it depends on the partially collapsing challenger to perform the ℓ-th measurement on $\mathcal{A}_{i^*} \in \mathcal{Z}_{\mathsf{k}+1}$ or not. In other words, if the challenger measures, then A_{pcol} runs $S_{i^*,2,\ell}$, and otherwise it actually runs $S_{i^*,2,\ell-1}$.

Further observe that $S_{i^*,2,\ell}$ and $S_{i^*,2,\ell-1}$ only differ in whether Step 2(d)vD is executed. If $c = \ell - 1$ is never true, then $S_{i^*,2,\ell}$ and $S_{i^*,2,\ell-1}$ are the same algorithm. In that case, $\boldsymbol{\mu}^{(\ell-1,\mathsf{t})} \leftarrow \mathsf{Before}_{i^*,2}(\ell-1, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})$ and $\boldsymbol{\mu}^{(\ell,\mathsf{t})} \leftarrow \mathsf{Before}_{i^*,2}(\ell, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi})$ have the same distribution, which implies that $\omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell-1,\mathsf{t})}) = \omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell,\mathsf{t})})$.

Now assume that $c = \ell - 1$ is true at some iteration ι'. If the challenger does not measure $\mathcal{A}_{i^*}$ upon receiving $\mathcal{Z}_{\mathsf{k}+1,i^*} = \mathcal{A}_{i^*} \otimes \mathcal{O}_{i^*}$, then the adversary A_{pcol} is in the experiment $\mathsf{PCollapseExp}(0, g, A_{\mathsf{pcol}}, \mathbf{aux})$. A_{pcol} almost runs $S_{i^*,2,\ell-1}$ and then After_{i^*}, except that there is one more projective measurement to check if $b = 1$ in the partially collapsing experiment at Step 3 (another one is at Step 2(d)iii of $A^{(1)}_{\mathsf{pcol}}$ at iteration ι'.). Since two consecutive applications of the same projective measurement yield the same outcome (and post-measurement state) as one,

$$\begin{aligned}
&\Pr\left[\begin{array}{l} |\mathbb{x}| \le n \\ \wedge\, \mathcal{V}^{(\widetilde{\pi}_j)_{j<i^*}}_{i^*-1}(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j, \mathsf{ans}_j, \mathsf{pf}_j)_{j\in[i^*,\mathsf{k}]}, (r_j)_{j\in[\mathsf{k}]}) = 1 \end{array}\right] \\
&= \omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell-1,\mathsf{t})}) \\
&= \Pr\left[\mathsf{PCollapseExp}(0, g, A_{\mathsf{pcol}}, \mathbf{aux}) = 1 \mid \mathbf{aux} \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})\right]
\end{aligned}$$

over the distribution

$$\left[\begin{array}{l} \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\mu}^{(\ell-1,\mathsf{t})}) \leftarrow \mathsf{Before}_{i^*,2}(\ell-1, \mathsf{t}, \mathsf{pp}, \boldsymbol{\xi}) \\ \left(\begin{array}{c} (\mathsf{cm}_j)_{j\in[i^*+1,\mathsf{k}]}, \\ (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[\mathsf{k}]}, (r_j)_{j\in[i^*,\mathsf{k}]} \end{array}\right) \leftarrow \mathsf{After}_{i^*}(\boldsymbol{\mu}^{(\ell-1,\mathsf{t})}) \end{array}\right] .$$

Similarly, if the challenger measures $\mathcal{A}_{i^*}$, then A_{pcol} is in the experiment $\mathsf{PCollapseExp}(1, g, A_{\mathsf{pcol}}, \mathbf{aux})$. Since ℓ measurements are performed, A_{pcol} runs $S_{i^*,2,\ell}$ and then After_{i^*}. As a result,

$$\Pr\left[\mathsf{PCollapseExp}(1, g, A_{\mathsf{pcol}}, \mathbf{aux}) = 1 \mid \mathbf{aux} \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})\right] = \omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell,\mathsf{t})}) \ .$$

To conclude, by Lemma 5.14,

$$\begin{aligned} &\left|\begin{array}{l} \Pr\left[\mathsf{PCollapseExp}(0, g, A_{\mathsf{pcol}}, \mathbf{aux}) = 1 \mid \mathbf{aux} \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})\right] \\ -\Pr\left[\mathsf{PCollapseExp}(1, g, A_{\mathsf{pcol}}, \mathbf{aux}) = 1 \mid \mathbf{aux} \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})\right] \end{array}\right| \\ &= \omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell-1,\mathsf{t})}) - \omega_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \boldsymbol{\mu}^{(\ell,\mathsf{t})}) \\ &\leq \mathsf{q}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) \ . \end{aligned}$$

The circuit size of A_{pcol} is $t_{\mathsf{pcol}} = O(t_{\mathsf{Rewind}}(\frac{2\mathsf{k}\cdot\mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}} + t_{\mathbb{V}})$ because in the worst case it simulates the sampler S_{i^*} and the argument prover $\widetilde{\mathbb{P}}$ with an additional verifier check in the end. □

Proof of Lemma 5.14. We prove the lemma via a hybrid argument. Let $q_1, q_2, \ldots, q_{\mathsf{q}_{i^*}}$ be the coordinates of Q_{i^*} in query order, i.e., $q_\iota = \mathsf{Query}^{\mathsf{ans}_{i^*}}\left(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, (\widetilde{\pi}_j)_{j<i^*}, \iota\right)$ for every $\iota \in [\mathsf{q}_{i^*}]$. (Note that $q_\iota \neq \mathtt{Abort}$ for all ι unless PCollapseExp aborts.) Define the function

$$\mathsf{PartialQuerySet}_\iota(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, (\mathsf{ans}_j)_{j\in[\iota]}) \coloneqq \{q_j : j \leq \iota\} \ .$$

For $\iota \in [0, \mathsf{q}_{i^*}]$, we define hybrid H_ι as follows.

H_ι:

1. Sample public parameters $\mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n)$.
2. Run $\left(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, \mathsf{cm}_{i^*}, (\widetilde{\pi}_j)_{j<i^*}, (\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}\right) \leftarrow A_{\mathsf{pcol}}(\mathsf{pp}, \mathbf{aux})$. Output $\mathtt{Abort}$ if $|\mathbb{x}| > n$.
3. Measure $\mathcal{V}_{i^*-1}^{(\widetilde{\pi}_j)_{j<i^*}}\left(\mathsf{pp}, \mathbb{x}, \left(\left(\mathsf{cm}_{i^*}, (\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}]}\right), \otimes_{j\in[i^*,\mathsf{k}]}\mathcal{Z}_{\mathsf{k}+1,j}\right), (r_j)_{j\in[\mathsf{k}]}\right)$ and output $\mathtt{Abort}$ if the outcome is 0.
4. Measure $Q_{i^*,\iota} \leftarrow \mathsf{PartialQuerySet}_\iota(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, ((\widetilde{\pi}_j)_{j<i^*}, \mathcal{A}_{i^*}))$ and $\mathsf{ans}_{i^*,\iota} \leftarrow \mathcal{A}_{i^*}(Q_{i^*,\iota})$.
5. Output $b \leftarrow A_{\mathsf{pcol}}\left((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}\right)$.

Note that $\mathsf{H}_0 = \mathsf{PCollapseExp}(0, g, A_{\mathsf{pcol}}, \mathbf{aux})$, as $\mathsf{PartialQuerySet}_0$ is identically $\varnothing$ (so Step 4 does nothing); and $\mathsf{H}_{\mathsf{q}_{i^*}} = \mathsf{PCollapseExp}(1, g, A_{\mathsf{pcol}}, \mathbf{aux})$, as $\mathsf{PartialQuerySet}_{\mathsf{q}_{i^*}}$ is QuerySet when applied to the i^*-th IOP string (so Step 4 measures g).

Therefore, it suffices to argue, for every $\iota \in [0, \mathsf{q}_{i^*} - 1]$, that

$$|\Pr[b = 1|\mathsf{H}_{\iota+1}] - \Pr[b = 1|\mathsf{H}_\iota]| \leq \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) \ .$$

Suppose, towards contradiction, that there exists some ι for which the above inequality does not hold. We construct an adversary $A_{\mathsf{VCCollapse}}$ for the collapse position binding experiment as follows.

- $A_{\mathsf{VCCollapse}}(\mathsf{pp}, \mathbf{aux})$:
  1. Get $\big(\mathsf{tr}, (\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}\big) \leftarrow A_{\mathsf{pcol}}(\mathsf{pp}, \mathbf{aux})$ and parse $\mathsf{tr} = (\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, \mathsf{cm}_{i^*}, (\widetilde{\pi}_j)_{j<i^*})$.
  2. Measure $\mathcal{V}_{i^*-1}^{(\widetilde{\pi}_j)_{j<i^*}}\big(\mathsf{pp}, \mathbb{x}, \big((\mathsf{cm}_{i^*}, (\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}]}\big), \otimes_{j\in[i^*,\mathsf{k}]}\mathcal{Z}_{\mathsf{k}+1,j}\big), (r_j)_{j\in[\mathsf{k}]}\big)$ and output $\mathtt{Abort}$ if the outcome is 0.
  3. Measure $Q_{i^*,\iota} \leftarrow \mathsf{PartialQuerySet}_\iota\big(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, ((\widetilde{\pi}_j)_{j<i^*}, \mathcal{A}_{i^*})\big)$ and $\mathsf{ans}_{i^*,\iota} \leftarrow \mathcal{A}_{i^*}(Q_{i^*,\iota})$.
  4. Compute $q_{\iota+1} \leftarrow \mathsf{Query}^{\mathsf{ans}_{i^*,\iota}}\big(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, (\widetilde{\pi}_j)_{j<i^*}, \iota+1\big)$.
  5. Output the classical messages $(\mathsf{cm}_{i^*}, q_{\iota+1})$ as well as the quantum registers $(\mathcal{A}_{i^*}, \mathcal{O}_{i^*})$.

- $A_{\mathsf{VCCollapse}}(\mathcal{A}_{i^*}, \mathcal{O}_{i^*})$:
  1. Output $b \leftarrow A_{\mathsf{pcol}}\big((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}\big)$.

The adversary $A_{\mathsf{VCCollapse}}$ has circuit size $t_{\mathsf{VCCollapse}} = O(t_{\mathsf{pcol}} + t_{\mathbb{V}})$, as it runs the partially collapsing adversary A_{pcol} and the verification algorithm $\mathcal{V}_{i^*-1}$ whose runtime is at most that of $\mathbb{V}$.

$A_{\mathsf{VCCollapse}}$ measures $\mathsf{ans}_{i^*,\iota} \leftarrow \mathcal{A}_{i^*}(Q_{i^*,\iota})$, and then compute the next query position classically $q_{\iota+1} \leftarrow \mathsf{Query}^{\mathsf{ans}_{i^*,\iota}}\big(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, (\widetilde{\pi}_j)_{j<i^*}, \iota+1\big)$. Since an additional measurement of $\mathcal{A}_{i^*}(q_{\iota+1})$ is made in the experiment $\mathsf{VCCollapseExp}(1, \lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, A_{\mathsf{VCCollapse}}, \mathbf{aux})$, a measurement of $\mathcal{A}_{i^*}(Q_{i^*,\iota+1})$ occurs between the two stages of A_{pcol}, which corresponds to an execution of $\mathsf{H}_{\iota+1}$.

This additional measurement is absent in $\mathsf{VCCollapseExp}(0, \lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, A_{\mathsf{VCCollapse}}, \mathbf{aux})$, so that it corresponds to an execution of H_ι.

Therefore, a distinguishing probability of more than $\epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}})$ for A_{pcol} implies the same advantage for $A_{\mathsf{VCCollapse}}$ (contradicting the collapse position binding error of VC). □

5.5 Proof of Claim 5.11

Fix $i^* \in [\mathsf{k}]$. Throughout this subsection, the probability distributions are over experiment $\mathsf{ReductorExp}$ in Equation 3 unless otherwise stated. By Construction 5.4, we define a distribution over $\mathsf{ReductorExp}$ as follows.

$$D \coloneqq \left[\begin{array}{l|l} \mathsf{cm}_{i^*}, \mathsf{ans}_{i^*}, & (\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)}) \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi}) \\ \mathsf{pf}_{i^*}, \widetilde{\pi}_{i^*}, \mathsf{ANS} & \mathsf{t} \leftarrow [0, T] \\ & (\mathsf{ANS}, \boldsymbol{\sigma}^{(i^*)}) \leftarrow S_{i^*}(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \mathsf{t}, \boldsymbol{\rho}^{(i^*)}) \\ & \widetilde{\pi}_{i^*} \coloneqq \perp^{\mathsf{L}_{i^*}} \\ & \widetilde{\pi}_{i^*}(q) = \mathsf{ans}'(q), \forall \mathsf{ans}' \in \mathsf{ANS}, q \in \mathsf{supp}(\mathsf{ans}') \\ & \big((\mathsf{cm}_j)_{j\in[i^*+1,\mathsf{k}]}, (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[\mathsf{k}]}, (r_j)_{j\in[i^*,\mathsf{k}]}\big) \leftarrow \mathsf{After}_{i^*}(\boldsymbol{\sigma}^{(i^*)}) \end{array}\right]$$

where $|\mathsf{ANS}| = \mathsf{t}$ and ANS contains the set of accepting answers $(\mathsf{ans}'_j)_{j\in[\mathsf{t}]}$ sampled by the reductor $\mathfrak{R}_{i^*}$. Throughout this section, probability statements are over the same experiment unless otherwise stated.

Then we define E to be the disjunction of the following two events:

(i) $\exists \mathsf{ans}', \mathsf{ans}'' \in \mathsf{ANS}$ and $q \in \mathsf{supp}(\mathsf{ans}') \cap \mathsf{supp}(\mathsf{ans}'')$ such that $\mathsf{ans}'(q) \neq \mathsf{ans}''(q)$; or

(ii) $\widetilde{\pi}_{i^*}$ and ans_{i^*} disagree at a position $q \in \mathsf{supp}(\mathsf{ans}_{i^*}) \cap \mathsf{supp}(\widetilde{\pi})$; or

(iii) there is a coordinate $q \in [\mathsf{L}_{i^*}]$ in $\mathsf{supp}(\mathsf{ans}_{i^*}) \setminus \mathsf{supp}(\widetilde{\pi})$.

Notice that if Item (i) does not happen, then it implies that at Step 4(e)ii of Construction 5.4, when $q \in Q^{(\mathsf{ANS})}$, sampler S_{i^*} is able to correctly simulate the answer at query q without measuring $\mathcal{A}_{i^*}$ using previously measured answers in ANS. Since the IOP string at round i^* has length at most L_{i^*}, when all query answers are simulated correctly, it suffices to measure at most L_{i^*} times, and the check $c < \mathsf{L}_{i^*}$ does not affect the number of times that answer register $\mathcal{A}_{i^*}$ should have been measured.

Next, note that if $\mathcal{P}_{i^*,2}$ convinces $\mathcal{V}_{i^*,2}$ and event E does not happen, then the proof string $\widetilde{\pi}_{i^*}$ is consistent with ans_{i^*}, thus $\mathcal{P}_{i^*,3}$ convinces $\mathcal{V}_{i^*,3}$. Hence, $\mathsf{val}\left(\mathsf{H}_{i^*,2}\right) \leq \mathsf{val}\left(\mathsf{H}_{i^*,3}\right) + \Pr\left[E\right]$.

To conclude the proof of Claim 5.11, it remains to show the following:

$$\Pr\left[E\right] \leq \mathsf{L}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) + \epsilon/2\mathsf{k} \ .$$

We consider Item (i) and Item (ii) as one sub-event, and consider the two sub-events of E separately.

(i) and (ii): valid openings with disagreeing answers. Our goal is to prove the following:

$$\begin{aligned}\Pr&\left[\begin{array}{l}\exists\, \mathsf{ans}', \mathsf{ans}'' \in \mathsf{ANS}, q \in \mathsf{supp}(\mathsf{ans}') \cap \mathsf{supp}(\mathsf{ans}'') : \mathsf{ans}'(q) \neq \mathsf{ans}''(q) \\ \text{or} \\ \exists\, q \in \mathsf{supp}(\mathsf{ans}_{i^*}) \cap \mathsf{supp}(\widetilde{\pi}_{i^*}) : \mathsf{ans}_{i^*}(q) \neq \widetilde{\pi}_{i^*}(q) \\ \wedge\ \mathsf{VC.Check}(\mathsf{pp}, \mathsf{cm}_{i^*}, \mathsf{ans}_{i^*}, \mathsf{pf}_{i^*}) = 1\end{array}\right] \\ &\leq 4 \cdot \mathsf{L}_{i^*} \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}}) \ ,\end{aligned}$$

where $t_{\mathsf{VCCollapse}} = O(t_{\mathsf{Rewind}}(\frac{2\mathsf{k}\cdot\mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}})$.

For $\mathbf{aux} \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})$, consider the following adversary $A_{\mathsf{VCUniq}} := (A^{(1)}_{\mathsf{VCUniq}}, A^{(2)}_{\mathsf{VCUniq}})$ against the vector commitment scheme, which simulates $\langle \widetilde{\mathcal{P}}_{i^*,3}, \mathcal{V}_{i^*,3}\rangle$ and runs the sampler S_{i^*} to find disagreeing answers among sampler answers ANS, or between sampler constructed proof $\widetilde{\pi}_{i^*}$ and the argument prover's answer ans_{i^*}. We highlight in blue the modifying parts in running the sampler S_{i^*} in order to output disagreeing answers to the unique-message position binding challenger.

- $A^{(1)}_{\mathsf{VCUniq}}(\mathsf{pp}, \mathbf{aux})$:
  1. Parse $\mathbf{aux}$ as $(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)})$.
  2. Sample $\mathsf{t} \in [0, T]$, and $\mathsf{idx} \in [\mathsf{L}_{i^*}]$.
  3. Run sampler S_{i^*} until it finds an answer at position idx:
     (a) Initialize $\mathsf{ANS} := \varnothing$ and $c := 0$.
     (b) Let $\boldsymbol{\sigma}^{(i^*,0)} := \boldsymbol{\rho}^{(i^*)}$ and $\boldsymbol{r} := (r_j)_{j<i^*}$.
     (c) Run $\mathbf{aux}_0 \leftarrow \mathsf{RewindInit}_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \mathsf{t}, \epsilon/2\mathsf{k}, \boldsymbol{\sigma}^{(i^*,0)})$.
     (d) For $\iota \in [\mathsf{t}]$:
        i. Sample IOP verifier randomness: $\boldsymbol{r}' := (r'_j)_{j\in[i^*,\mathsf{k}]} \leftarrow \{0,1\}^{\mathsf{r}_{i^*}+\cdots+\mathsf{r}_{\mathsf{k}}}$.
        ii. Run $\left((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, \mathbf{aux}'_\iota\right) \leftarrow \mathsf{RewindRepair}_{\mathsf{G}_{i^*}}(\boldsymbol{r}', \mathbf{aux}_{\iota-1})$.
        iii. Measure $b \leftarrow f_{i^*}(\boldsymbol{r}', ((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}]}, \mathcal{Z}_{\mathsf{k}+1}))$. If $b = 0$, skip to Step 3(d)vi.
        iv. Define $Q^{(\mathsf{ANS})} := \cup_{\mathsf{ans}\in\mathsf{ANS}}\mathsf{supp}(\mathsf{ans})$ and initialize $\widetilde{\pi} := \bot^{\mathsf{L}_{i^*}}$.
        v. For $m \in [\mathsf{q}_{i^*}]$:
           A. Compute $q \leftarrow \mathsf{Query}^{\widetilde{\pi}}(\mathbb{x}, (\boldsymbol{r}, \boldsymbol{r}'), (\widetilde{\pi}_j)_{j<i^*}, m)$.
           B. If $q \in Q^{(\mathsf{ANS})}$, let j be minimal such that $\mathsf{ans}'_j \in \mathsf{ANS}$ and $q \in \mathsf{supp}(\mathsf{ans}'_j)$. Set $\widetilde{\pi}(q) := \mathsf{ans}'_j(q)$.
           C. If $c < \mathsf{L}_{i^*}$ and $q \notin Q^{(\mathsf{ANS})}$, measure $\mathcal{A}_{i^*}$ in the computational basis to obtain response ans'_ι, add ans'_ι to ANS and increment $c := c + 1$.
           D. If Step 3(d)vC was true (measurement was performed): if $\mathsf{idx} \in \mathsf{supp}(\mathsf{ans}'_\iota)$, output $\left(\mathsf{cm}_{i^*}, \mathsf{idx}, \mathsf{ans}'_\iota(\mathsf{idx}), \mathsf{ans}'_\iota, \mathcal{O}_{i^*}\right)$ where $\mathcal{O}_{i^*}$ stores the opening proof pf'_ι. Also pass the internal state $(\mathsf{idx}, \mathbf{aux}'_\iota, \mathsf{ANS}, \iota)$ to $A^{(2)}_{\mathsf{VCUniq}}$; otherwise, skip to Step 3(d)vi.
        vi. Run $\mathbf{aux}_\iota \leftarrow \mathsf{RewindRepair}_{\mathsf{G}_{i^*}}((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, b, \mathbf{aux}'_\iota)$.

4. Output $\left(\mathsf{cm}_{i^*}, \mathsf{idx}, \mathsf{ans}'_{\mathsf{t}}(\mathsf{idx}), \mathsf{ans}'_{\mathsf{t}}, \mathcal{O}_{i^*}\right)$ where $\mathcal{O}_{i^*}$ stores the opening proof $\mathsf{pf}'_{\mathsf{t}}$. Pass the internal state $(\mathsf{idx}, \mathbf{aux}_{\mathsf{t}}, \mathsf{ANS}, \mathsf{t}+1)$ to $A^{(2)}_{\mathsf{VCUniq}}$.

- $A^{(2)}_{\mathsf{VCUniq}}(\mathcal{O}_{i^*}, \mathsf{idx}, \mathbf{aux}, \mathsf{ANS}, \iota')$:
  1. If $\iota' = \mathsf{t}+1$, set $\mathbf{aux}_{\mathsf{t}} := \mathbf{aux}$ and skip to Step 3. Otherwise, set $\mathbf{aux}'_{\iota'} := \mathbf{aux}$.
  2. Run the remaining iterations of S_{i^*} until it finds a disagreeing answer at position idx:
     (a) Run $\mathbf{aux}_{\iota'} \leftarrow \mathsf{RewindRepair}_{\mathsf{G}_{i^*}}((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, b, \mathbf{aux}'_{\iota'})$.
     (b) For $\iota \in [\iota'+1, \mathsf{t}]$:
        i. Sample IOP verifier randomness: $\boldsymbol{r}' := (r'_j)_{j\in[i^*,\mathsf{k}]} \leftarrow \{0,1\}^{\mathsf{r}_{i^*}+\cdots+\mathsf{r}_{\mathsf{k}}}$.
        ii. Run $\left((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, \mathbf{aux}'_{\iota}\right) \leftarrow \mathsf{RewindStrat}_{\mathsf{G}_{i^*}}(\boldsymbol{r}', \mathbf{aux}_{\iota-1})$.
        iii. Measure $b \leftarrow f_{i^*}(\boldsymbol{r}', ((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}]}, \mathcal{Z}_{\mathsf{k}+1}))$. If $b=0$, skip to Step 2(b)vi.
        iv. Define $Q^{(\mathsf{ANS})} := \cup_{\mathsf{ans}\in\mathsf{ANS}}\mathsf{supp}(\mathsf{ans})$ and initialize $\widetilde{\pi} := \perp^{\mathsf{L}_{i^*}}$.
        v. For $m \in [\mathsf{q}_{i^*}]$:
           A. Compute $q \leftarrow \mathsf{Query}^{\widetilde{\pi}}(\mathbb{x}, (\boldsymbol{r}, \boldsymbol{r}'), (\widetilde{\pi}_j)_{j<i^*}, m)$.
           B. If $q \in Q^{(\mathsf{ANS})}$, let j be minimal such that $\mathsf{ans}'_j \in \mathsf{ANS}$ and $q \in \mathsf{supp}(\mathsf{ans}'_j)$. Set $\widetilde{\pi}(q) := \mathsf{ans}'_j(q)$.
           C. If $c < \mathsf{L}_{i^*}$ and $q \notin Q^{(\mathsf{ANS})}$, measure $\mathcal{A}_{i^*}$ in the computational basis to obtain response ans'_{ι}, add ans'_{ι} to ANS and increment $c := c+1$.
           D. If Step 2(b)vC was true: if $\mathsf{idx} \in \mathsf{supp}(\mathsf{ans}'_{\iota})$ and there exists $\mathsf{ans}'' \in \mathsf{ANS}$ such that $\mathsf{ans}'_{\iota}(\mathsf{idx}) \neq \mathsf{ans}''(\mathsf{idx})$, output $\left(\mathsf{ans}'_{\iota}(\mathsf{idx}), \mathsf{ans}'_{\iota}, \mathcal{O}_{i^*}\right)$ where $\mathcal{O}_{i^*}$ stores the opening proof pf'_{ι}; otherwise, skip to Step 2(b)vi.
        vi. Run $\mathbf{aux}_{\iota} \leftarrow \mathsf{RewindRepair}_{\mathsf{G}_{i^*}}((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, b, \mathbf{aux}'_{\iota})$.
  3. Parse $\mathbf{aux}_{\mathsf{t}}$ as $(\widetilde{\mathbb{P}}_{>i^*}, \gamma, \delta, \mathsf{t}, p_{\mathsf{t}}, \boldsymbol{\sigma}^{(i^*,\mathsf{t})})$.
  4. Run $\left((\mathsf{cm}_j)_{j\in[i^*+1,\mathsf{k}]}, (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[\mathsf{k}]}, (r_j)_{j\in[i^*,\mathsf{k}]}\right) \leftarrow \mathsf{After}_{i^*}(\boldsymbol{\sigma}^{(i^*,\mathsf{t})})$.
  5. Output $(\mathsf{ans}_{i^*}(\mathsf{idx}), \mathsf{ans}_{i^*}, \mathsf{pf}_{i^*})$.

By the construction of $\widetilde{\pi}_{i^*}$, there exists q such that $\mathsf{ans}_{i^*}(q) \neq \widetilde{\pi}_{i^*}(q)$ if and only if the reductor queried a position q at j'-th iteration and got valid queried answer $\mathsf{ans}'_{j'}$ such that $\mathsf{ans}_{i^*}(q) \neq \mathsf{ans}'_{j'}(q)$. In addition, there exists $j', j'' \in [\mathsf{t}]$ and $q \in \mathsf{supp}(\mathsf{ans}'_{j'}) \cap \mathsf{supp}(\mathsf{ans}'_{j''})$ such that $\mathsf{ans}'_{j'}(q) \neq \mathsf{ans}'_{j''}(q)$ if and only if the reductor queried a position q at j'-th iteration and queried q again at j''-th iteration such that $\mathsf{ans}'_{j'}(q) \neq \mathsf{ans}'_{j''}(q)$.

Observe that $A^{(1)}_{\mathsf{VCUniq}}$ outputs ans'_j where $\mathsf{idx} \in \mathsf{supp}(\mathsf{ans}'_j)$. If Item (i) or Item (ii) happens, A_{VCUniq} succeeds if $q = \mathsf{idx}$, because if so, $\mathsf{ans}'_j(\mathsf{idx}) = \mathsf{ans}'_{j'}(\mathsf{idx}) \neq \mathsf{ans}_{i^*}(\mathsf{idx})$, or $\mathsf{ans}'_j(\mathsf{idx}) = \mathsf{ans}'_{j'}(\mathsf{idx}) \neq \mathsf{ans}'_{j''}(\mathsf{idx})$. As a result, we obtain

$$\Pr\left[\begin{array}{l} \left(\begin{array}{l}\exists\, \mathsf{ans}', \mathsf{ans}'' \in \mathsf{ANS}, q \in \mathsf{supp}(\mathsf{ans}') \cap \mathsf{supp}(\mathsf{ans}'') : \\ \mathsf{ans}'(q) \neq \mathsf{ans}''(q)\end{array}\right) \\ \text{or} \\ \left(\begin{array}{l}\exists\, q \in \mathsf{supp}(\mathsf{ans}_{i^*}) \cap \mathsf{supp}(\widetilde{\pi}_{i^*}) : \\ \mathsf{ans}_{i^*}(q) \neq \widetilde{\pi}_{i^*}(q) \\ \wedge\, \mathsf{VC.Check}\left(\mathsf{pp}, \mathsf{cm}_{i^*}, \mathsf{ans}_{i^*}, \mathsf{pf}_{i^*}\right) = 1\end{array}\right) \end{array} \middle| \; (\mathsf{cm}_{i^*}, \mathsf{ans}_{i^*}, \mathsf{pf}_{i^*}, \widetilde{\pi}_{i^*}, \mathsf{ANS}) \leftarrow D \right]$$

$$\leq \Pr\left[\begin{array}{l} \left(\begin{array}{l}\exists\, j', j'' \in [\mathsf{t}], q \in \mathsf{supp}(\mathsf{ans}'_{j'}) \cap \mathsf{supp}(\mathsf{ans}'_{j''}) : \\ \mathsf{ans}'_{j'}(q) \neq \mathsf{ans}'_{j''}(q)\end{array}\right) \\ \text{or} \\ \left(\begin{array}{l}\exists\, j' \in [\mathsf{t}], q \in \mathsf{supp}(\mathsf{ans}_{i^*}) \cap \mathsf{supp}(\mathsf{ans}'_{j'}) : \\ \mathsf{ans}_{i^*}(q) \neq \mathsf{ans}'_{j'}(q) \\ \wedge\, \mathsf{VC.Check}\big(\mathsf{pp}, \mathsf{cm}_{i^*}, \mathsf{ans}_{i^*}, \mathsf{pf}_{i^*}\big) = 1\end{array}\right) \end{array} \middle| \begin{array}{l} \\ (\mathsf{cm}_{i^*}, \mathsf{ans}_{i^*}, \mathsf{pf}_{i^*}, \widetilde{\pi}_{i^*}, \mathsf{ANS}) \leftarrow D \\ \mathsf{ANS} = (\mathsf{ans}'_j)_{j\in[\mathsf{t}]} \end{array}\right]$$

$$\leq \mathsf{L}_{i^*} \cdot \Pr\left[\begin{array}{l} \left(\begin{array}{l} \left(\begin{array}{l}\exists\, j', j'' \in [\mathsf{t}], q \in \mathsf{supp}(\mathsf{ans}'_{j'}) \cap \mathsf{supp}(\mathsf{ans}'_{j''}) : \\ \mathsf{ans}'_{j'}(q) \neq \mathsf{ans}'_{j''}(q)\end{array}\right) \\ \text{or} \\ \left(\begin{array}{l}\exists\, j' \in [\mathsf{t}], q \in \mathsf{supp}(\mathsf{ans}_{i^*}) \cap \mathsf{supp}(\mathsf{ans}'_{j'}) : \\ \mathsf{ans}_{i^*}(q) \neq \mathsf{ans}'_{j'}(q) \\ \wedge\, \mathsf{VC.Check}\big(\mathsf{pp}, \mathsf{cm}_{i^*}, \mathsf{ans}_{i^*}, \mathsf{pf}_{i^*}\big) = 1\end{array}\right) \end{array}\right) \\ \wedge\, \mathsf{idx} = q \end{array} \middle| \begin{array}{l} \\ \mathsf{idx} \leftarrow [\mathsf{L}_{i^*}] \\ \left(\begin{array}{l}\mathsf{cm}_{i^*}, \mathsf{ans}_{i^*}, \mathsf{pf}_{i^*}, \\ \widetilde{\pi}_{i^*}, \mathsf{ANS}\end{array}\right) \leftarrow D \\ \mathsf{ANS} = (\mathsf{ans}'_j)_{j\in[\mathsf{t}]} \end{array}\right]$$

$$\leq \mathsf{L}_{i^*} \cdot \Pr\left[\begin{array}{l} \mathsf{ans}(\mathsf{idx}) \neq \mathsf{ans}'_j(\mathsf{idx}) \\ \wedge\, \mathsf{VC.Check}\big(\mathsf{pp}, \mathsf{cm}_{i^*}, \mathsf{ans}, \mathcal{O}\big) = 1 \\ \wedge\, \mathsf{idx} \in \mathsf{supp}(\mathsf{ans}) \end{array} \middle| \begin{array}{l} \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ \mathbf{aux} \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi}) \\ \left(\begin{array}{l}\mathsf{cm}_{i^*}, \mathsf{idx}, \mathsf{ans}'_j(\mathsf{idx}), \mathsf{ans}'_j, \\ \mathcal{O}_{i^*}, (\mathbf{aux}', \mathsf{ANS}, j')\end{array}\right) \leftarrow A^{(1)}_{\mathsf{VCUniq}}(\mathsf{pp}, \mathbf{aux}) \\ (\mathsf{ans}(\mathsf{idx}), \mathsf{ans}, \mathcal{O}) \leftarrow A^{(2)}_{\mathsf{VCUniq}}(\mathcal{O}_{i^*}, (\mathbf{aux}', \mathsf{ANS}, j')) \end{array}\right]$$

$$\leq \mathsf{L}_{i^*} \cdot \Pr\left[\mathsf{VCUniqExp}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, A_{\mathsf{VCUniq}}, \mathbf{aux}) = 1\right]$$

$$\leq \mathsf{L}_{i^*} \cdot \epsilon_{\mathsf{VCUniq}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCUniq}}) \ .$$

A_{VCUniq} simulates S_{i^*} and After_{i^*} (which runs $\widetilde{\mathbb{P}}$ only), so the circuit size of A_{VCUniq} is at most $t_{\mathsf{VCUniq}} = O(t_{\mathsf{Rewind}}(\frac{2\mathsf{k}\cdot\mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}})$. By Lemma 4.5, $\epsilon_{\mathsf{VCUniq}}$ is upper bounded by $4\cdot\mathsf{L}_{i^*}\cdot\epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_{i^*}, \mathsf{q}_{i^*}, t_{\mathsf{VCCollapse}})$ where $t_{\mathsf{VCCollapse}} = O(t_{\mathsf{VCUniq}})$.

(iii): missing positions in $\widetilde{\pi}_{i^*}$. We upper bound the probability that there is a query q in $\mathsf{supp}(\mathsf{ans}_{i^*}) \setminus \mathsf{supp}(\widetilde{\pi}_{i^*})$; we claim that

$$\Pr\left[\mathsf{supp}(\mathsf{ans}_{i^*}) \setminus \mathsf{supp}(\widetilde{\pi}_{i^*}) \neq \varnothing\right] \leq \epsilon/2\mathsf{k} \ .$$

For every $\iota \in [T]$, let $\mathsf{SampQuerySets}_\iota$ be an algorithm that simulates the interaction of $\langle \widetilde{\mathcal{P}}_{i^*,3}, \mathcal{V}_{i^*,3}\rangle$ until the beginning of the i^*-th round and then simulates the sampler S_{i^*} for $\iota+1$ iterations of Step 4 in Construction 5.4. We highlight in blue the places where we slightly modify S_{i^*}. Concretely, $\mathsf{SampQuerySets}_\iota$ works as follows:

$\mathsf{SampQuerySets}_\iota$:

1. Run $(\mathsf{pp}, \mathbb{x}, (\mathsf{cm}_j)_{j\in[i^*]}, (r_j)_{j<i^*}, (\widetilde{\pi}_j)_{j<i^*}, \boldsymbol{\rho}^{(i^*)}) \leftarrow \mathsf{Before}_{i^*}(\mathsf{pp}, \boldsymbol{\xi})$.
2. Initialize $I_0 := \varnothing$.
3. Run S_{i^*} for ι iterations:
   (a) Initialize $\mathsf{ANS} := \varnothing$ and $c := 0$.
   (b) Let $\boldsymbol{\sigma}^{(i^*,0)} := \boldsymbol{\rho}^{(i^*)}$ and $\boldsymbol{r} := (r_j)_{j<i^*}$.
   (c) Run $\mathbf{aux}_0 \leftarrow \mathsf{RewindInit}_{\mathsf{G}_{i^*}}(\widetilde{\mathbb{P}}_{>i^*}, \iota, \epsilon/2\mathsf{k}, \boldsymbol{\sigma}^{(i^*,0)})$.
   (d) For $i \in [\iota]$:
      i. Sample $\boldsymbol{r}' := (r'_j)_{j\in[i^*,\mathsf{k}]} \leftarrow \{0,1\}^{\mathsf{r}_{i^*}+\cdots+\mathsf{r}_\mathsf{k}}$.
      ii. Run $\big((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, \mathbf{aux}'_i\big) \leftarrow \mathsf{RewindStrat}_{\mathsf{G}_{i^*}}(\boldsymbol{r}', \mathbf{aux}_{i-1})$.
      iii. Measure $b \leftarrow f_{i^*}(\boldsymbol{r}', ((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}]}, \mathcal{Z}_{\mathsf{k}+1}))$. If $b = 0$, set $I_i := I_{i-1}$ and skip to Step 3(d)vi.
      iv. Define $Q^{(\mathsf{ANS})} := \cup_{\mathsf{ans}\in\mathsf{ANS}}\mathsf{supp}(\mathsf{ans})$ and initialize $\widetilde{\pi} := \bot^{\mathsf{L}_{i^*}}$.

v. For $m \in [\mathsf{q}_{i^*}]$:
   A. Compute $q \leftarrow \mathsf{Query}^{\widetilde{\pi}}(\mathbb{x}, (\boldsymbol{r}, \boldsymbol{r}'), (\widetilde{\pi}_j)_{j<i^*}, m)$.
   B. If $q \in Q^{(\mathsf{ANS})}$, let j be minimal such that $\mathsf{ans}'_j \in \mathsf{ANS}$ and $q \in \mathsf{supp}(\mathsf{ans}'_j)$. Set $\widetilde{\pi}(q) \coloneqq \mathsf{ans}'_j(q)$.
   C. If $c < \mathsf{L}_{i^*}$ and $q \notin Q^{(\mathsf{ANS})}$, measure $\mathcal{A}_{i^*}$ in the computational basis to obtain response ans'_i, add ans'_i to ANS and increment $c \coloneqq c + 1$. Also obtain the query set $Q'_i \leftarrow \mathsf{QuerySet}^{\mathsf{ans}'_i}(\mathbb{x}, (\boldsymbol{r}, \boldsymbol{r}'), (\widetilde{\pi}_j)_{j<i^*})$. Set $I_i \coloneqq I_{i-1} \cup Q'_i$. Skip to Step 3(d)vi.

vi. Run $\mathbf{aux}_i \leftarrow \mathsf{RewindRepair}_{\mathsf{G}_{i^*}}((\mathcal{Z}_j)_{j\in[i^*+1,\mathsf{k}+1]}, b, \mathbf{aux}'_i)$.

4. Parse $\mathbf{aux}_\iota$ as $(\widetilde{\mathbb{P}}_{>i^*}, \gamma, \delta, \iota, p_\iota, \boldsymbol{\sigma}^{(i^*,\iota)})$.
5. Run $\left((\mathsf{cm}_j)_{j\in[i^*+1,\mathsf{k}]}, (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[\mathsf{k}]}, (r_j)_{j\in[i^*,\mathsf{k}]}\right) \leftarrow \mathsf{After}_{i^*}(\boldsymbol{\sigma}^{(i^*,\iota)})$.
6. Compute the hybrid verifier $\mathcal{V}_{i^*}$'s i^*-th round query set $Q_{i^*} \leftarrow \mathsf{QuerySet}^{\mathsf{ans}_{i^*}}(\mathbb{x}, (r_j)_{j\in[\mathsf{k}]}, (\widetilde{\pi}_j)_{j<i^*})$.
7. If $f_{i^*}((r_j)_{j\in[i^*,\mathsf{k}]}, (\mathsf{cm}_j)_{j\in[i^*+1,\mathsf{k}]}, (\mathsf{ans}_j, \mathsf{pf}_j)_{j\in[i^*,\mathsf{k}]}) = 1$, let $I_{\iota+1} \coloneqq I_\iota \cup Q_{i^*}$.
8. Output $(I_1, \ldots, I_{\iota+1})$.

We claim that

$$p \coloneqq \Pr\left[Q_{i^*} \setminus \mathsf{supp}(\widetilde{\pi}_{i^*}) \neq \varnothing\right] = \Pr\left[I_{\mathsf{t}} \neq I_{\mathsf{t}+1} \;\middle|\; \begin{array}{r} \mathsf{t} \leftarrow [0,T] \\ (I_1, \ldots, I_{\mathsf{t}+1}) \leftarrow \mathsf{SampQuerySets}_{\mathsf{t}} \end{array}\right] .$$

Notice that I_{t} and $\mathsf{supp}(\widetilde{\pi}_{i^*})$ have the same distribution since $\mathsf{supp}(\widetilde{\pi}_{i^*})$ is the union of all queries after S_{i^*} runs for t iterations. Moreover, it is clear that Q_{i^*} follows the same distribution as $Q'_{\mathsf{t}+1}$. Hence, $Q_{i^*} \setminus \mathsf{supp}(\widetilde{\pi}_{i^*}) \neq \varnothing$ happens if and only if $Q'_{\mathsf{t}+1} \setminus I_{\mathsf{t}} \neq \varnothing$, i.e. $I_{\mathsf{t}} \neq I_{\mathsf{t}+1}$ as desired.

It remains to bound p. For every j such that $0 \leq j \leq \iota \leq T$, let $\zeta_{\iota,j}$ be the indicator random variable for the event that $I_j \neq I_{j+1}$ in an execution of $\mathsf{SampQuerySets}_\iota$.

$$\zeta_{\iota,j} \coloneqq \mathbb{1}\{I_j \neq I_{j+1} \mid (I_1, \ldots, I_{\iota+1}) \leftarrow \mathsf{SampQuerySets}_\iota\} .$$

Note that

$$p = \mathbb{E}\left[\zeta_{\mathsf{t},\mathsf{t}}\right] = \frac{1}{T} \cdot \sum_{j=0}^{T} \mathbb{E}\left[\zeta_{j,j}\right] ,$$

where the expectation is taken over both t and $\mathsf{SampQuerySets}$.

Note that $|I_j| \leq \mathsf{L}_{i^*}$ for every $j \in \{0, \ldots, \iota\}$. Therefore, taking $\iota = T$,

$$\sum_{j=0}^{T} \zeta_{T,j} \leq \mathsf{L}_{i^*}$$

with probability 1, so that

$$\sum_{j=0}^{T} \mathbb{E}\left[\zeta_{T,j}\right] \leq \mathsf{L}_{i^*} .$$

Crucially, we observe that for every $j \in [0,T]$,

$$\mathbb{E}\left[\zeta_{j,j}\right] = \mathbb{E}\left[\zeta_{T,j}\right] .$$

Clearly the distribution of I_j is the same in both $\mathsf{SampQuerySets}_j$ and $\mathsf{SampQuerySets}_T$, as those experiments generate I_j in the same way. For I_{j+1}, we note that Steps 3(d)i and 3(d)ii of $\mathsf{SampQuerySets}_T$

constitute a coherent execution of Step 5 to Step 7 of $\mathsf{SampQuerySets}_j$. It follows that I_{j+1} is identically distributed in both cases.

Putting these facts together, we obtain

$$p = \frac{1}{T} \cdot \sum_{j=0}^{T} \mathbb{E}\left[\zeta_{j,j}\right] \;\; = \frac{1}{T} \cdot \sum_{j=0}^{T} \mathbb{E}\left[\zeta_{T,j}\right] \leq \frac{1}{T} \cdot \mathsf{L}_{i^*} \leq \epsilon/2\mathsf{k} \;,$$

as desired.

6 Security analysis of the IBCS protocol

We prove our main theorem: a bound on the post-quantum soundness and knowledge soundness errors of the IBCS protocol.

Theorem 6.1. *Consider these two ingredients:*

- $\mathsf{IOP} = (\mathbf{P}, \mathbf{V})$*, a public-coin semi-adaptive IOP system for a relation* R *with round complexity* k*, alphabet* Σ*, total proof length* L*, maximum proof length* $\mathsf{L}_{\mathsf{max}}$*, and maximum query complexity* $\mathsf{q}_{\mathsf{max}}$*; and*
- $\mathsf{VC} = (\mathsf{Gen}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Check})$*, a vector commitment scheme over alphabet* Σ*.*

Then $\mathsf{ARG} = (\mathbb{G}, \mathbb{P}, \mathbb{V}) := \mathsf{IBCS}[\mathsf{IOP}, \mathsf{VC}]$ *is a* **post-quantum** *interactive argument system for* R *whose soundness error* ϵ_{ARG} *and knowledge soundness error* κ_{ARG} *satisfy the following for every* $\epsilon > 0$*:*

$$\epsilon_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) \leq \epsilon_{\mathsf{IOP}}(n) + \sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \quad \textit{and}$$
$$\kappa_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) \leq \kappa_{\mathsf{IOP}}(n) + \sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon$$

where $t_{\mathsf{VCCollapse}} = O(t_{\mathsf{Rewind}}(\frac{2\mathsf{k} \cdot \mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}} + t_{\mathbb{V}})$ *and* t_{Rewind} *is the rewinding time in Equation 1. Moreover, the argument extraction circuit size bound is* $t_{\mathcal{E}} = O(\mathsf{k} \cdot (t_{\mathsf{Rewind}}(\frac{2\mathsf{k} \cdot \mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}}) + t_{\mathbf{E}})$.

We prove Theorem 6.1 in Sections 6.1 and 6.2. Below we prove that the theorem implies a negligible loss in soundness error and knowledge soundness error if VC's collapse position binding error is negligible.

Corollary 6.2. *Let* ARG *be as in Theorem 6.1. Assume that* $\epsilon_{\mathsf{VCCollapse}}(\lambda, L, s, t_{\mathsf{VCCollapse}}) = \mathrm{negl}(\lambda)$ *when* $L, s, t_{\mathsf{VCCollapse}} = \mathsf{poly}(\lambda)$*. Then, for* $t_{\mathsf{ARG}} = \mathsf{poly}(\lambda)$ *it holds that*

$$\epsilon_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) \leq \epsilon_{\mathsf{IOP}}(n) + \mathrm{negl}(\lambda) \quad \textit{and}$$
$$\kappa_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) \leq \kappa_{\mathsf{IOP}}(n) + \mathrm{negl}(\lambda) \ .$$

Proof. We argue the bound for ϵ_{ARG}; an analogous argument holds for κ_{ARG}. Let $p(\lambda)$ be any polynomial and $\epsilon := \frac{1}{2p(\lambda)} > 0$. By Equation 1, $t_{\mathsf{Rewind}}(\frac{2\mathsf{k} \cdot \mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) = \mathsf{poly}(\frac{\mathsf{k} \cdot \mathsf{L}_{\mathsf{max}}}{\epsilon}) \cdot (t_{\mathbb{V}} + t_{\mathsf{ARG}})$. Hence

$$\begin{aligned} t_{\mathsf{VCCollapse}} &= O(t_{\mathsf{Rewind}}(\frac{2\mathsf{k} \cdot \mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}} + t_{\mathbb{V}}) \\ &= O(\mathsf{poly}(\frac{\mathsf{k} \cdot \mathsf{L}_{\mathsf{max}}}{\epsilon}) \cdot (t_{\mathbb{V}} + t_{\mathsf{ARG}}) + t_{\mathsf{ARG}} + t_{\mathbb{V}}) \\ &= \mathsf{poly}(\frac{\mathsf{k} \cdot \mathsf{L}_{\mathsf{max}}}{\epsilon}) \cdot (t_{\mathsf{ARG}} + t_{\mathbb{V}}) \\ &= \mathsf{poly}(\lambda) \end{aligned}$$

and hence $\epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) = \mathrm{negl}(\lambda)$ for every $i \in [\mathsf{k}]$. Therefore, for large enough λ,

$$\begin{aligned} \epsilon_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) &\leq \epsilon_{\mathsf{IOP}}(n) + \sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \qquad \text{(by Theorem 6.1)} \\ &\leq \epsilon_{\mathsf{IOP}}(n) + \mathrm{negl}(\lambda) + \frac{1}{2p(\lambda)} \end{aligned}$$

$$< \epsilon_{\mathsf{IOP}}(n) + \frac{1}{p(\lambda)} \ .$$

Since p is an arbitrary polynomial, we conclude that

$$\epsilon_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) \leq \epsilon_{\mathsf{IOP}}(n) + \mathrm{negl}(\lambda) \ .$$

□

6.1 Soundness analysis

By Definition 3.10, we want to upper-bound the following probability:

$$\begin{aligned}
&\Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ \wedge\, \mathbb{x} \notin L(R) & (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ \wedge\, b = 1 & b \leftarrow \left\langle \widetilde{\mathbb{P}}(\mathsf{aux}), \mathbb{V}(\mathsf{pp}, \mathbb{x}) \right\rangle \end{array}\right] \\
&\leq \Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ \wedge\, \mathbb{x} \notin L(R) & (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ \wedge\, b = 1 & b \leftarrow \left\langle \widetilde{\mathbf{P}}^{\tilde{\mathbb{P}}}(\mathsf{aux}), \mathbf{V}(\mathbb{x}) \right\rangle \end{array}\right] && \text{(by Construction 5.5 and Lemma 5.1)} \\
&\quad + \sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \\
&= \Pr\left[\begin{array}{l|l} |\mathbb{x}| \leq n & \\ \wedge\, \mathbb{x} \notin L(R) & (\mathbb{x}, \mathbf{aux}) \leftarrow \widetilde{\mathbf{P}}^{\tilde{\mathbb{P}}}(\boldsymbol{\xi}) \\ \wedge\, b = 1 & b \leftarrow \left\langle \widetilde{\mathbf{P}}^{\tilde{\mathbb{P}}}(\mathbf{aux}), \mathbf{V}(\mathbb{x}) \right\rangle \end{array}\right] \\
&\quad + \sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \\
&\leq \epsilon_{\mathsf{IOP}}(n) + \sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \ . && \text{(by Definition 3.2)}
\end{aligned}$$

Notice that there is one extra predicate $\mathbb{x} \notin L(R)$ in the probabilities when applying Lemma 5.1 to the second inequality. Nevertheless, Lemma 5.1 still applies since predicates on the instance $\mathbb{x}$ have no effect when $\mathbb{x}$ is generated in the same way.

Finally, we conclude that

$$\epsilon_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) \leq \epsilon_{\mathsf{IOP}}(n) + \sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \ .$$

6.2 Knowledge soundness analysis

First we construct the knowledge extractor $\mathcal{E}$ for ARG. Let $\mathbf{E}$ be the extractor for IOP.

$\mathcal{E}^{\tilde{\mathbb{P}}}(\mathbb{x}, \mathsf{aux})$:
1. Define $\widetilde{\mathbf{P}}^{\tilde{\mathbb{P}}}$ as in Construction 5.5.
2. Run $\mathbb{w} \leftarrow \mathbf{E}^{\widetilde{\mathbf{P}}^{\tilde{\mathbb{P}}}}(\mathbb{x}, \mathsf{aux})$.
3. Output $\mathbb{w}$.

By Definition 3.11, we want to lower-bound the following probability:

$$
\begin{aligned}
&\Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n \\ \wedge\, (\mathbb{x}, \mathbb{w}) \in R & \begin{array}{l} \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ \mathbb{w} \leftarrow \mathcal{E}^{\widetilde{\mathbb{P}}}(\mathbb{x}, \mathsf{aux}) \end{array} \end{array}\right] \\
&= \Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n \\ \wedge\, (\mathbb{x}, \mathbb{w}) \in R & \begin{array}{l} (\mathbb{x}, \mathbf{aux}) \leftarrow \widetilde{\mathbf{P}}^{\widetilde{\mathbb{P}}}(\boldsymbol{\xi}) \\ \mathbb{w} \leftarrow \mathbf{E}^{\widetilde{\mathbf{P}}}(\mathbb{x}, \mathbf{aux}) \end{array} \end{array}\right] && \text{(from construction of } \mathcal{E}) \\
&= \Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n \\ \wedge\, (\mathbb{x}, \mathbb{w}) \in R & \begin{array}{l} \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ \mathbb{w} \leftarrow \mathbf{E}^{\widetilde{\mathbf{P}}}(\mathbb{x}, \mathsf{aux}) \end{array} \end{array}\right] \\
&\ge \Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n \\ \wedge\, b = 1 & \begin{array}{l} \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ b \leftarrow \left\langle \widetilde{\mathbf{P}}^{\widetilde{\mathbb{P}}}(\mathsf{aux}), \mathbf{V}(\mathbb{x}) \right\rangle \end{array} \end{array}\right] - \kappa_{\mathsf{IOP}}(n) && \text{(by Definition 3.3)} \\
&\ge \Pr\left[\begin{array}{l|l} |\mathbb{x}| \le n \\ \wedge\, b = 1 & \begin{array}{l} \mathsf{pp} \leftarrow \mathbb{G}(1^\lambda, n) \\ (\mathbb{x}, \mathsf{aux}) \leftarrow \widetilde{\mathbb{P}}(\mathsf{pp}, \boldsymbol{\xi}) \\ b \leftarrow \left\langle \widetilde{\mathbb{P}}(\mathsf{aux}), \mathbb{V}(\mathsf{pp}, \mathbb{x}) \right\rangle \end{array} \end{array}\right] \\
&\quad - \kappa_{\mathsf{IOP}}(n) - \left(\sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \right) . && \text{(by Lemma 5.1)}
\end{aligned}
$$

Hence, we conclude that

$$
\kappa_{\mathsf{ARG}}(\lambda, n, t_{\mathsf{ARG}}) \le \kappa_{\mathsf{IOP}}(n) + \sum_{i \in [\mathsf{k}]} \mathsf{L}_i \cdot (\mathsf{q}_i + 5) \cdot \epsilon_{\mathsf{VCCollapse}}(\lambda, \mathsf{L}_i, \mathsf{q}_i, t_{\mathsf{VCCollapse}}) + \epsilon \ .
$$

The circuit size of the IOP prover $\widetilde{\mathbf{P}}$ is $O(\mathsf{k} \cdot (t_{\mathsf{Rewind}}(\frac{2\mathsf{k} \cdot \mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}}))$, so the circuit size of our argument knowledge extractor is $t_{\mathcal{E}} = O(\mathsf{k} \cdot (t_{\mathsf{Rewind}}(\frac{2\mathsf{k} \cdot \mathsf{L}_{\mathsf{max}}}{\epsilon}, \frac{\epsilon}{2\mathsf{k}}, t_{\mathbb{V}}, t_{\mathsf{ARG}}) + t_{\mathsf{ARG}}) + t_{\mathbf{E}})$.

Acknowledgments

Alessandro Chiesa, Zijing Di, and Ziyi Guan are partially supported by the Ethereum Foundation.

References

[ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. "STIR: Reed-Solomon Proximity Testing with Fewer Queries". In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO '24. 2024, pp. 380–413.

[ACLMT22] Martin R. Albrecht, Valerio Cini, Russell W. F. Lai, Giulio Malavolta, and Sri Aravinda Krishnan Thyagarajan. "Lattice-Based SNARKs: Publicly Verifiable, Preprocessing, and Recursively Composable". In: *Proceedings of the 42nd Annual International Cryptology Conference*. CRYPTO '22. 2022, pp. 102–132.

[ACY23] Gal Arnon, Alessandro Chiesa, and Eylon Yogev. "IOPs with Inverse Polynomial Soundness Error". In: *Proceedings of the 64th Annual IEEE Symposium on Foundations of Computer Science*. FOCS '23. 2023, pp. 752–761.

[AHIKV17] Benny Applebaum, Naama Haramaty, Yuval Ishai, Eyal Kushilevitz, and Vinod Vaikuntanathan. "Low-Complexity Cryptographic Hash Functions". In: *Proceedings of the 8th Innovations in Theoretical Computer Science Conference*. ITCS '17. 2017, 7:1–7:31.

[BBHR19] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. "Scalable Zero Knowledge with No Trusted Setup". In: *Proceedings of the 39th Annual International Cryptology Conference*. CRYPTO '19. 2019, pp. 733–764.

[BCG20] Jonathan Bootle, Alessandro Chiesa, and Jens Groth. "Linear-Time Arguments with Sublinear Verification from Tensor Codes". In: *Proceedings of the 18th Theory of Cryptography Conference*. TCC '20. 2020, pp. 19–46.

[BCGRS17] Eli Ben-Sasson, Alessandro Chiesa, Ariel Gabizon, Michael Riabzev, and Nicholas Spooner. "Interactive Oracle Proofs with Constant Rate and Query Complexity". In: *Proceedings of the 44th International Colloquium on Automata, Languages and Programming*. ICALP '17. 2017, 40:1–40:15.

[BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. "Interactive Oracle Proofs". In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC '16-B. 2016, pp. 31–60.

[BG08] Boaz Barak and Oded Goldreich. "Universal Arguments and their Applications". In: *SIAM Journal on Computing* 38.5 (2008). Preliminary version appeared in CCC '02., pp. 1661–1694.

[BLNS20] Jonathan Bootle, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Gregor Seiler. "A Non-PCP Approach to Succinct Quantum-Safe Zero-Knowledge". In: *Proceedings of the 40th Annual International Cryptology Conference*. CRYPTO '20. 2020, pp. 441–469.

[Bar+22] James Bartusek, Yael Tauman Kalai, Alex Lombardi, Fermi Ma, Giulio Malavolta, Vinod Vaikuntanathan, Thomas Vidick, and Lisa Yang. "Succinct Classical Verification of Quantum Computation". In: *Proceedings of the 42nd Annual International Cryptology Conference*. CRYPTO '22. 2022, pp. 195–211.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. "HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates". In: *Proceedings of the 42nd Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT '23. 2023, pp. 499–530.

[CCHLRR18] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. *Fiat–Shamir From Simpler Assumptions*. Cryptology ePrint Archive, Report 2018/1004. 2018.

[CCY21] Nai-Hui Chia, Kai-Min Chung, and Takashi Yamakawa. "A Black-Box Approach to Post-Quantum Zero-Knowledge in Constant Rounds". In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO '21. 2021, pp. 315–345.

[CDGS23] Alessandro Chiesa, Marcel Dall'Agnol, Ziyi Guan, and Nicholas Spooner. *On the Security of Succinct Interactive Arguments from Vector Commitments*. Cryptology ePrint Archive, Report 2023/1737. 2023.

[CDGSY24] Alessandro Chiesa, Marcel Dall'Agnol, Ziyi Guan, Nicholas Spooner, and Eylon Yogev. "Untangling the Security of Kilian's Protocol: Upper and Lower Bounds". In: *Proceedings of the 22nd Theory of Cryptography Conference*. TCC '24. 2024.

[CF13] Dario Catalano and Dario Fiore. "Vector Commitments and Their Applications". In: *Proceedings of the 16th International Conference on Practice and Theory in Public Key Cryptography*. PKC '13. 2013, pp. 55–72.

[CGKY25] Alessandro Chiesa, Ziyi Guan, Christian Knabenhans, and Zihan Yu. *On the Fiat–Shamir Security of Succinct Arguments from Functional Commitments*. Cryptology ePrint Archive, Paper 2025/902. 2025.

[CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. "Succinct Arguments in the Quantum Random Oracle Model". In: *Proceedings of the 17th Theory of Cryptography Conference*. TCC '19. 2019, pp. 1–29.

[CMSZ21] Alessandro Chiesa, Fermi Ma, Nicholas Spooner, and Mark Zhandry. "Post-Quantum Succinct Arguments: Breaking the Quantum Rewinding Barrier". In: *Proceedings of the 62nd Annual IEEE Symposium on Foundations of Computer Science*. FOCS '21. 2021, pp. 49–58.

[DFS24] Thomas Debris-Alazard, Pouria Fallahpour, and Damien Stehlé. "Quantum Oblivious LWE Sampling and Insecurity of Standard Model Lattice-Based SNARKs". In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. STOC '24. 2024, pp. 423–434.

[GLSTW23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. "Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS". In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO '23. 2023, pp. 193–226.

[GMNO18] Rosario Gennaro, Michele Minelli, Anca Nitulescu, and Michele Orrù. "Lattice-Based zk-SNARKs from Square Span Programs". In: *Proceedings of the 25th ACM Conference on Computer and Communications Security*. CCS '18. 2018, pp. 556–573.

[GNS23] Chaya Ganesh, Anca Nitulescu, and Eduardo Soria-Vazquez. "Rinocchio: SNARKs for Ring Arithmetic". In: *Journal of Cryptology* 36.4 (2023), p. 41.

[GW11] Craig Gentry and Daniel Wichs. "Separating Succinct Non-Interactive Arguments From All Falsifiable Assumptions". In: *Proceedings of the 43rd Annual ACM Symposium on Theory of Computing*. STOC '11. 2011, pp. 99–108.

[HLP24] Ulrich Haböck, David Levit, and Shahar Papini. *Circle STARKs*. Cryptology ePrint Archive, Report 2024/278. 2024.

[HR22] Justin Holmgren and Ron D. Rothblum. "Faster Sounder Succinct Arguments and IOPs". In: *Proceedings of the 42nd Annual International Cryptology Conference*. CRYPTO '22. 2022, pp. 474–503.

[ISW21] Yuval Ishai, Hang Su, and David J. Wu. "Shorter and Faster Post-Quantum Designated-Verifier zkSNARKs from Lattices". In: *Proceedings of the 28th ACM Conference on Computer and Communications Security*. CCS '21. 2021, pp. 212–234.

[Kil92] Joe Kilian. "A note on efficient zero-knowledge proofs and arguments". In: *Proceedings of the 24th Annual ACM Symposium on Theory of Computing*. STOC '92. 1992, pp. 723–732.

[LMS22a] Russell W. F. Lai, Giulio Malavolta, and Nicholas Spooner. "Quantum Rewinding for Many-Round Protocols". In: *Proceedings of the 20th Theory of Cryptography Conference*. TCC '22. 2022, pp. 80–109.

[LMS22b] Alex Lombardi, Fermi Ma, and Nicholas Spooner. "Post-Quantum Zero Knowledge, Revisited or: How to Do Quantum Rewinding Undetectably". In: *Proceedings of the 63rd Annual IEEE Symposium on Foundations of Computer Science*. FOCS '22. 2022, pp. 851–859.

[MNZ24] Tony Metger, Anand Natarajan, and Tina Zhang. "Succinct arguments for QMA from standard assumptions via compiled nonlocal games". In: *Proceedings of the 65th Annual IEEE Symposium on Foundations of Computer Science*. FOCS '24. 2024, pp. 1193–1201.

[RR20] Noga Ron-Zewi and Ron Rothblum. "Local Proofs Approaching the Witness Length". In: *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*. FOCS '20. 2020.

[RR22] Noga Ron-Zewi and Ron D. Rothblum. "Proving as fast as computing: succinct arguments with constant prover overhead". In: *Proceedings of the 54th Annual ACM Symposium on Theory of Computing*. STOC '22. 2022, pp. 1353–1363.

[RRR16] Omer Reingold, Ron Rothblum, and Guy Rothblum. "Constant-Round Interactive Proofs for Delegating Computation". In: *Proceedings of the 48th ACM Symposium on the Theory of Computing*. STOC '16. 2016, pp. 49–62.

[STW23] Srinath Setty, Justin Thaler, and Riad Wahby. *Customizable constraint systems for succinct arguments*. Cryptology ePrint Archive, Report 2023/552. 2023.

[Unr12] Dominique Unruh. "Quantum Proofs of Knowledge". In: *Proceedings of the 31st Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT '12. 2012, pp. 135–152.

[Unr16a] Dominique Unruh. "Collapse-Binding Quantum Commitments Without Random Oracles". In: *Proceedings of the 22rd International Conference on the Theory and Applications of Cryptology and Information Security*. ASIACRYPT '16. 2016.

[Unr16b] Dominique Unruh. "Computationally Binding Quantum Commitments". In: *Proceedings of the 35th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT '16. 2016, pp. 497–527.

[WB15] Michael Walfish and Andrew J. Blumberg. "Verifying Computations Without Reexecuting Them". In: *Communications of the ACM* 58.2 (Jan. 2015), pp. 74–84.

[WW23] Hoeteck Wee and David J. Wu. "Lattice-Based Functional Commitments: Fast Verification and Cryptanalysis". In: *Proceedings of the 29th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT '23. 2023, pp. 201–235.

[Wat09] John Watrous. "Zero-Knowledge against Quantum Attacks". In: *SIAM Journal on Computing* 39.1 (2009). Preliminary version appeared in STOC '06., pp. 25–58.

[ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes". In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO '24. 2024, pp. 138–169.